

CyberDudeBivash - GCP Data Engineer

GCP Data Engineer Real-Time On-Job Mastery Program (2026 Edition)

LEVELS: Beginner to Senior to Lead

For Working Professionals & Domain Switchers

Make Yourself Fully Productive, Confident & Promotable as a GCP Data Engineer

GCP DATA ENGINEER

PROFESSIONAL ROLES & RESPONSIBILITIES

- DATA PLATFORM OWNERSHIP
  - **Design, build & operate scalable GCP data platforms**
  - **Own data pipelines end-to-end (ingestion → analytics)**
- DATA INGESTION & PROCESSING
  - **Build batch & streaming pipelines (GCS, Pub/Sub, Dataflow)**
  - **Handle real-time, incremental & CDC data patterns**
- BIGQUERY & ANALYTICS ENGINEERING

- **Model data for analytics, reporting & BI**
- **Optimize BigQuery for performance & cost**
- **Implement partitioning, clustering & data marts**

- ORCHESTRATION & AUTOMATION
  - **Design reliable workflows using Cloud Composer / Airflow**
  - **Manage scheduling, dependencies & backfills**

- DATA QUALITY & RELIABILITY
  - **Implement data validation, freshness & quality checks**
  - **Ensure trustworthy, timely & accurate data delivery**

- SECURITY & GOVERNANCE

- **Apply IAM, least-privilege access & service accounts**
- **Protect PII using masking, row-level & column-level security**
- **Ensure audit & compliance readiness**

- CI/CD & DEVOPS FOR DATA
  - **Version control pipelines & infrastructure**
  - **Automate deployments (Dev → Stage → Prod)**
  - **Use Terraform for infrastructure as code**

- MONITORING, INCIDENTS & ON-CALL

- **Monitor pipelines, cost & performance**
- **Respond to incidents calmly & professionally**
- **Maintain runbooks & post-incident reviews**

- COST & PERFORMANCE OPTIMIZATION
  - **Control BigQuery, Dataflow & storage costs**
  - **Optimize pipelines at scale without over-engineering**
- SRE & OPERATIONAL EXCELLENCE
  - **Define SLIs, SLOs & error budgets**
  - **Make release decisions based on reliability metrics**
- COLLABORATION & LEADERSHIP

- **Work with stakeholders & business teams**
- **Communicate risks, tradeoffs & impact clearly**
- **Mentor engineers & uphold engineering standards**

🏆 CyberDudeBivash – GCP Data Engineer

Real-Time On-Job Mastery Program (2026 Edition)

Level: Beginner → Senior → Lead
Audience: Working professionals & domain switchers
Goal: Make the learner fully productive, confident, and promotable as a GCP Data Engineer

This is written like internal enterprise training, not a YouTube course.

📘 PROGRAM OVERVIEW

What This Training Will Do

- Train exactly what GCP Data Engineers do at work
- Teach real pipelines, failures, incidents, fixes
- Build muscle memory for daily tasks

- Make the learner job-safe + job-excellent

What This Training Will NOT Do

- ❌ No shallow theory
- ❌ No copy-paste labs
- ❌ No interview-only nonsense

🔰 PHASE 0 – ROLE CLARITY & FOUNDATION (Mandatory)

Module 0.1 – GCP Data Engineer Role Deep Dive

- What a Data Engineer actually does daily
- Responsibilities vs expectations
- KPIs used to measure a DE
- Interaction with:
    - Business
    - Analysts
    - ML teams
    - DevOps
- Junior vs Senior vs Lead DE

Module 0.2 – Data Engineering Fundamentals (Practical)

- What is data engineering in real companies
- Batch vs Streaming (business context)
- OLTP vs OLAP
- Data lakes vs warehouses vs lakehouses
- Why bad data engineering kills businesses

☁️ PHASE 1 – GCP CORE FOR DATA ENGINEERS

Module 1.1 – GCP Account, Project & Org Setup

- Enterprise GCP org structure
- Folder & project strategy
- Environment separation (dev / stage / prod)
- Naming conventions (real-world)

Module 1.2 – IAM for Data Engineers (Critical)

- IAM concepts simplified
- Service accounts (real usage)
- Least privilege in pipelines

- IAM mistakes that break pipelines
- IAM debugging at work

Module 1.3 – Networking Basics for Data Pipelines

- VPC basics (only what DE needs)
- Private services access
- Secure data movement
- Common network failures in data jobs

📥 PHASE 2 – DATA INGESTION (DAILY JOB WORK)

Module 2.1 – Ingestion Patterns

- File-based ingestion
- API ingestion
- Streaming ingestion
- CDC ingestion
- Vendor data feeds

Module 2.2 – Cloud Storage (GCS) Deep Dive

- Buckets & lifecycle rules

- Folder structures
- File formats (CSV, JSON, Parquet, Avro)
- Cost optimization
- Data validation

Module 2.3 – Pub/Sub for Real-Time Data

- Topics & subscriptions
- Push vs Pull
- Ordering & retries
- Dead-letter queues
- Real production patterns

Projects

- Vendor file ingestion pipeline
- API → GCS ingestion system
- Pub/Sub real-time event ingestion

🧠 PHASE 3 – BIGQUERY (CORE SKILL)

Module 3.1 – BigQuery Architecture (How It REALLY Works)

- Storage vs compute
- Slots & reservations
- Cost model (critical at work)
- Why queries are slow

Module 3.2 – Data Modeling for Analytics

- Star schema
- Snowflake schema
- Fact & dimension tables
- Slowly changing dimensions (SCD)

Module 3.3 – BigQuery Optimization

- Partitioning
- Clustering
- Query rewriting
- Cost reduction strategies
- Real troubleshooting cases

Projects

- Raw → Curated BigQuery pipeline

- Query optimization challenge
- Cost reduction exercise

🔄 PHASE 4 – DATA PROCESSING

Module 4.1 – Dataflow & Apache Beam

- When to use Dataflow
- Beam concepts explained clearly
- Batch pipelines
- Streaming pipelines
- Windowing & triggers
- Late data handling

Module 4.2 – Dataproc & Spark

- Spark fundamentals for DE
- PySpark transformations
- Performance tuning
- Handling skew & failures

Projects

- Streaming pipeline → BigQuery
- Spark-based transformation job
- Deduplication & enrichment pipeline

⏱ PHASE 5 – ORCHESTRATION & AUTOMATION

Module 5.1 – Cloud Composer (Airflow)

- DAG fundamentals
- Scheduling strategies
- Backfills
- Reruns
- SLA monitoring

Module 5.2 – Workflow Design

- Idempotent pipelines
- Dependency management
- Failure recovery patterns

Projects

- End-to-end Airflow DAG

- Production rerun scenario

🚀 PHASE 6 – DEVOPS FOR DATA ENGINEERS

Module 6.1 – Git & Version Control

- Branching strategy
- Code reviews
- Release management

Module 6.2 – CI/CD for Data Pipelines

- Pipeline testing
- Deployment automation
- Environment promotion

Module 6.3 – Terraform Basics for DE

- Infrastructure as code
- Managing datasets, buckets, jobs
- Safe deployments

🚨 PHASE 7 – MONITORING, INCIDENTS & ON-CALL LIFE

Module 7.1 – Monitoring & Logging

- Cloud Monitoring
- Logs Explorer
- Custom metrics

Module 7.2 – Incident Handling (REAL SCENARIOS)

- Pipeline failure at midnight
- Missing data in dashboards
- Cost spikes
- Wrong data published

This is senior-level training.

🔐 PHASE 8 – SECURITY, GOVERNANCE & COMPLIANCE

- Data security best practices
- PII handling
- DLP
- Audit logging

- Access reviews
- Compliance basics

📈 PHASE 9 – PERFORMANCE, COST & SCALE

- Cost optimization strategies
- Query & pipeline tuning
- Scaling data platforms
- Tradeoffs in architecture

🧭 PHASE 10 – CAREER & REAL-WORLD EXCELLENCE

- Communicating with stakeholders
- Designing data platforms
- Migration projects
- Becoming Lead / Principal DE
- Avoiding burnout

📦 FINAL CAPSTONE PROJECT

Enterprise-Grade GCP Data Platform

- Multiple data sources
- Batch + streaming
- Orchestration
- Monitoring
- Security
- Cost control

Exactly what companies run in production.

🟢 PHASE 0

ROLE CLARITY & FOUNDATION

(For Working Professionals & Domain Switchers)

📘 MODULE 0.1

What a GCP Data Engineer ACTUALLY Does (Real Job Reality)

0.1.1 Why This Module Matters (Read This First)

Most people fail in Data Engineering not because they lack tools, but because they:

- Don’t understand what the job actually expects
- Confuse Data Engineering with Analytics or ML
- Focus on learning tools instead of delivering business outcomes

This module removes confusion, false expectations, and career risk.

0.1.2 The Core Responsibility of a GCP Data Engineer

A GCP Data Engineer is responsible for:

Designing, building, operating, securing, and maintaining data pipelines that deliver reliable, timely, and accurate data to the business.

That’s it.

Everything else is a sub-task.

0.1.3 Daily Responsibilities (REALISTIC, NOT THEORY)

Every Day, a GCP Data Engineer Does Some Combination Of:

1️⃣ Data Pipeline Operations

- Monitor pipelines
- Fix failed jobs
- Rerun workflows
- Validate data freshness

2️⃣ Data Ingestion & Processing

- Add new data sources

- Modify existing pipelines
- Handle schema changes
- Process batch & streaming data

3️⃣ Debugging & Troubleshooting

- Broken Airflow DAGs
- Dataflow job failures
- BigQuery query issues
- IAM permission errors

4️⃣ Data Quality Assurance

- Check row counts
- Detect missing data
- Validate transformations
- Investigate anomalies

5️⃣ Cost & Performance Optimization

- Reduce BigQuery cost
- Optimize queries
- Tune Dataflow jobs
- Control storage growth

0.1.4 Weekly Responsibilities

- Release new pipelines
- Review pipeline performance
- Participate in sprint planning
- Review pull requests
- Attend architecture discussions

0.1.5 Monthly / Periodic Responsibilities

- Cost audits
- Access reviews
- Platform improvements
- Migration tasks
- Incident postmortems

0.1.6 What a GCP Data Engineer Is NOT

❌ Not a Data Analyst
❌ Not a Dashboard Builder
❌ Not a Machine Learning Engineer
❌ Not a DevOps Engineer (but overlaps)

Understanding this protects your career path.

0.1.7 Role Comparison (Critical for Switchers)

Data Engineer vs Data Analyst

Data Engineer	Data Analyst
Builds pipelines	Consumes data
Owns infrastructure	Owns reports
Writes backend code	Writes SQL
Focuses on reliability	Focuses on insights

Data Engineer vs ML Engineer

Data Engineer	ML Engineer
Data movement & quality	Model building
Pipelines & orchestration	Training & inference
Platforms	Algorithms

Data Engineer vs DevOps

Data Engineer	DevOps
Data pipelines	Application pipelines
BigQuery, Dataflow	Kubernetes, CI/CD
Data reliability	App reliability

0.1.8 Levels of a GCP Data Engineer

Junior Data Engineer

- Executes tasks
- Follows instructions
- Fixes simple issues

Mid-Level Data Engineer

- Designs pipelines
- Owns components
- Handles incidents

Senior Data Engineer

- Designs platforms
- Makes architectural decisions
- Mentors others
- Owns reliability & cost

Lead / Principal

- Owns strategy
- Talks to business
- Designs scalable systems

0.1.9 How Performance Is Measured at Work

You are NOT measured by:

- Certifications
- Fancy tools
- Complex code

You ARE measured by:

- Pipeline uptime
- Data freshness
- Incident reduction
- Cost control
- Reliability

📘 MODULE 0.2

Data Engineering Fundamentals (Real-World, Practical)

0.2.1 What Data Engineering Really Is

Data Engineering = Data Plumbing for the Business

If data:

- Is late → business fails

- Is wrong → decisions fail
- Is missing → trust fails

Your job is to prevent failure.

0.2.2 Batch vs Streaming (Business Context)

Batch Processing

- Scheduled
- Predictable
- Cost-effective
- Used for reports, analytics

Examples:

- Daily sales reports
- Financial reconciliation
- Historical analytics

Streaming Processing

- Real-time
- Continuous
- More expensive

- Used for alerts & events

Examples:

- Clickstream
- Fraud detection
- IoT data

0.2.3 OLTP vs OLAP (Why BigQuery Exists)

OLTP Systems

- Transactions
- Small queries
- Frequent writes

Examples:

- MySQL
- PostgreSQL

OLAP Systems

- Analytics
- Large scans
- Complex queries

Examples:

- BigQuery
- Snowflake

0.2.4 Data Lake vs Data Warehouse

Data Lake

- Raw data
- Cheap storage
- Schema-on-read

Data Warehouse

- Cleaned data
- Structured
- Business-ready

Most GCP platforms use both.

0.2.5 Why Data Engineering Fails in Companies

Common reasons:

- No ownership
- No monitoring
- No cost control

- Poor data quality
- Weak security

A strong Data Engineer prevents all of these.

0.2.6 The Data Engineering Mindset (Very Important)

You must think like:

- An operator
- A reliability engineer
- A business enabler

Not like:

- A script writer
- A tutorial follower

0.2.7 Core Principles You Will Follow in This Program

- Reliability > Cleverness
- Simplicity > Complexity
- Automation > Manual work
- Observability > Guessing
- Cost awareness > Unlimited usage

✅ PHASE 0 COMPLETION OUTCOME

After Phase 0, the learner will:

- Understand the real GCP Data Engineer job
- Know daily, weekly, monthly responsibilities
- Avoid role confusion
- Be mentally prepared for real-world work
- Think like a professional engineer

🟦 PHASE 1

GCP Core for Data Engineers

(Real-World Cloud Foundation – Job Critical)

📘 MODULE 1.1

GCP Organization, Projects & Environment Setup (Enterprise Reality)

1.1.1 Why This Module Is Critical

Most production failures, access issues, cost overruns, and security incidents happen not because of code, but because of:

- Poor project structure

- Bad environment separation
- Wrong IAM boundaries

A strong Data Engineer understands the cloud layout, not just pipelines.

1.1.2 How Real Companies Structure GCP (Not Tutorials)

Typical Enterprise GCP Hierarchy

```
Organization
└── Folders
    ├── Non-Prod
    │   ├── dev
    │   ├── qa
    │   └── stage
    └── Prod
        ├── prod-data
        ├── prod-analytics
        └── prod-ml
```

Why This Matters

- Prevents accidental production damage
- Controls cost
- Enforces access discipline
- Enables safe deployments

As a Data Engineer, you must understand this even if you don't create it.

1.1.3 Projects: The Real Boundary of Responsibility

In GCP, projects are the real security and billing boundary.

Typical Data Engineering Projects

- data-ingestion-dev
- data-processing-dev
- analytics-dev
- data-ingestion-prod
- analytics-prod

Each project has:

- Separate IAM
- Separate billing
- Separate datasets

1.1.4 Environment Separation (MANDATORY)

Why Environments Exist

Environment	Purpose
Dev	Build & experiment
QA	Validate logic
Stage	Production-like testing
Prod	Business-critical

What Happens Without Separation

- Data corruption
- Cost explosions
- Business outages

A Data Engineer never experiments in prod. Ever.

1.1.5 Naming Conventions (Looks Boring – Saves Careers)

Examples:

- Buckets: company-env-domain-purpose
- Datasets: raw_sales_dev
- Tables: orders_daily_partitioned

Consistency allows:

- Debugging
- Automation
- On-call sanity

📘 MODULE 1.2

IAM for Data Engineers (This Breaks Pipelines Daily)

1.2.1 IAM Reality for Data Engineers

Most pipeline failures at work look like:

❌ "Permission denied"

IAM issues are normal, not embarrassing.

Your job is to diagnose and fix them fast.

1.2.2 Core IAM Concepts (Simplified)

- Principal → Who (user, service account)
- Role → What permissions
- Resource → Where (project, dataset, bucket)

IAM = Who can do what, where.

1.2.3 Service Accounts (Your Pipelines' Identity)

What Service Accounts Are

- Non-human identities
- Used by Dataflow, Composer, scripts
- Represent your pipelines

Why They Matter

- Security
- Auditability
- Least privilege

1.2.4 Typical Data Engineer Service Accounts

- dataflow-sa
- composer-sa
- ingestion-sa

Each has only the permissions it needs.

1.2.5 Common IAM Mistakes (Real Incidents)

❌ Using default service accounts
❌ Giving Editor role everywhere
❌ Mixing human and pipeline access
❌ Granting dataset access at project level

These mistakes cause:

- Security risk
- Audit failures
- Accidental data deletion

1.2.6 IAM Debugging Workflow (On-Job Skill)

When a pipeline fails:

1. Identify the service account
2. Identify the resource
3. Check required permissions

4. Grant minimal role
5. Retry pipeline

This workflow is used weekly in real jobs.

📘 MODULE 1.3

Networking Basics for Data Engineers (Only What You Need)

1.3.1 Why Networking Matters to DEs

You don't design networks — but your pipelines depend on them.

Common failures:

- API unreachable
- Private IP restrictions
- On-prem connectivity issues

1.3.2 Key Concepts You MUST Know

- VPC (virtual network)
- Subnets
- Private Service Access
- VPC Service Controls (advanced orgs)

You don't need deep networking — but you must recognize network-related failures.

1.3.3 Secure Data Movement Patterns

- On-prem → GCS
- API → GCP
- Cross-project access

All require:

- Correct IAM
- Correct network permissions

📘 MODULE 1.4

Billing & Cost Awareness (Silent Career Killer)

1.4.1 Why Cost Matters to Data Engineers

Data Engineers can:

- Generate massive BigQuery bills
- Run expensive Dataflow jobs
- Store unnecessary data forever

Cost awareness = job safety.

1.4.2 Common Cost Mistakes

- No partitioning
- Full table scans
- Streaming when batch is enough
- Long-running Dataflow jobs

1.4.3 Your Responsibility

You may not own billing — but you are responsible for cost-efficient design.

Senior engineers are trusted because they don't burn money.

📘 MODULE 1.5

Daily GCP Tasks for a Data Engineer

Every Day, You Will:

- Check pipeline health
- Review logs
- Validate data freshness
- Fix permission issues
- Respond to alerts

This is normal, not failure.

🧪 PHASE 1 PRACTICAL EXERCISES (Job-Simulated)

1. Identify correct project for a pipeline
2. Decide service account usage
3. Debug a permission failure
4. Explain cost impact of a query
5. Identify environment misuse

🟩 PHASE 2

DATA INGESTION – REAL PIPELINES, REAL WORK, REAL FAILURES

Core Job Truth:
If data ingestion fails, everything downstream is irrelevant.

Dashboards, ML models, analytics — all depend on ingestion reliability.

📘 MODULE 2.1

Data Ingestion Patterns (What You ACTUALLY See at Work)

2.1.1 What "Data Ingestion" Means in Real Companies

Data ingestion is not:

- Copying files once
- Running a script manually

- Uploading CSVs

Data ingestion is a system that must:

- Run automatically
- Handle failures
- Scale with volume
- Preserve data integrity
- Be auditable

2.1.2 The 5 Real-World Ingestion Patterns

Every company uses some combination of these.

🔹 Pattern 1: File-Based Ingestion (MOST COMMON)

Sources:

- Vendors
- Partners
- Legacy systems
- SFTP servers

Formats:

- CSV
- JSON

- Parquet
- Avro

Typical Flow:

External Source → GCS → BigQuery (raw)

Real Challenges:

- Late files
- Missing files
- Partial uploads
- Schema changes
- Duplicate files

🔹 Pattern 2: API-Based Ingestion

Sources:

- SaaS platforms
- Internal microservices
- Third-party vendors

Typical Flow:

API → Python Ingestion → GCS → BigQuery

Real Challenges:

- Rate limits
- Authentication expiry
- Pagination
- Inconsistent responses
- API downtime

🔹 Pattern 3: Streaming Ingestion (Pub/Sub)

Sources:

- Clickstream
- Events
- Logs
- IoT

Typical Flow:

Producer → Pub/Sub → Dataflow → BigQuery

Real Challenges:

- Message ordering
- Late events
- Duplicates
- Backpressure

- Cost control

🔹 Pattern 4: CDC (Change Data Capture)

Sources:

- OLTP databases

Typical Flow:

DB → CDC Tool → GCS / PubSub → BigQuery

Real Challenges:

- Schema evolution
- Deletes
- Reprocessing
- Consistency

🔹 Pattern 5: Manual / One-Time Loads (Still Real)

Yes, this still happens.

Examples:

- Backfills
- Migrations
- Historical corrections

Your job is to make them safe and repeatable.

📘 MODULE 2.2

Google Cloud Storage (GCS) – Your Ingestion Backbone

2.2.1 How GCS Is Used by Data Engineers

GCS is:

- A landing zone
- A buffer
- A recovery layer
- A cheap archive

It is not just “storage”.

2.2.2 Real-World Bucket Design

Typical Bucket Structure

```
gs://company-env-raw-data/
 ├── source_name/
 │    ├── year=2026/
 │    │    ├── month=01/
 │    │    │    ├── day=15/
 │    │    │    │    └── data_file.parquet
```

Why this matters:

- Partition-friendly

- Easy backfills
- Clear ownership

2.2.3 Lifecycle Rules (Critical for Cost)

Every raw bucket must have:

- Retention policies
- Auto-deletion rules
- Archival strategies

Failing this = exploding storage cost.

2.2.4 File Validation (On-Job Reality)

Before loading:

- Check file size
- Check format
- Check row count
- Check schema

Never assume input is clean.

📘 MODULE 2.3

API Ingestion – Python-Based Real Pipelines

2.3.1 Why APIs Break Pipelines Constantly

APIs fail due to:

- Rate limits
- Token expiry
- Vendor bugs
- Network issues

Your code must expect failure.

2.3.2 Real API Ingestion Workflow

Scheduler (Airflow)

→ Python Ingestion Script

→ API Calls (Paginated)

→ Raw Files in GCS

→ Load to BigQuery

2.3.3 Core Responsibilities of the Ingestion Script

- Authentication handling
- Pagination logic
- Retry with backoff
- Logging

- Idempotency (no duplicates)

2.3.4 Example (Conceptual Python Logic)

```
def fetch_and_store(api_client, bucket, date):
    data = []
    page = 1
    while True:
        response = api_client.get(page=page)
        if not response:
            break
        data.extend(response)
        page += 1

    upload_to_gcs(bucket, data, date)
```

At work, this code includes:

- Error handling
- Timeouts
- Metrics
- Alerts

📘 MODULE 2.4

Pub/Sub for Streaming Ingestion (Production View)

2.4.1 What Pub/Sub Really Does

Pub/Sub:

- Buffers events
- Decouples producers & consumers
- Enables scalability

It does NOT:

- Guarantee ordering by default
- Clean your data
- Deduplicate automatically

2.4.2 Real Streaming Architecture

App → Pub/Sub Topic
→ Subscription
→ Dataflow
→ BigQuery

2.4.3 Production Considerations

- Dead-letter topics
- Retry policies

- Message retention
- Throughput control

Most juniors ignore this.
Seniors don't.

📘 MODULE 2.5

BigQuery Raw Layer (Ingestion Target)

2.5.1 Why Raw Data Must Stay Raw

Raw layer principles:

- No transformations
- No business logic
- Append-only

This allows:

- Reprocessing
- Debugging
- Auditing

2.5.2 Schema Handling in Ingestion

You must handle:

- New columns
- Missing fields
- Data type changes

Ignoring schema evolution causes:

- Pipeline breaks
- Data loss
- Business distrust

📘 MODULE 2.6

Data Quality Checks at Ingestion Time

Every ingestion pipeline should include:

- Row count validation
- File completeness check
- Basic schema validation
- Freshness check

Data quality starts at ingestion, not analytics.

🧪 PHASE 2 – REAL-WORLD PROJECTS

Project 1: Vendor File Ingestion

- Daily CSV drop
- GCS landing
- Validation
- Load to BigQuery raw

Project 2: API-Based Ingestion

- Authenticated API
- Pagination
- Error handling
- Incremental loads

Project 3: Streaming Events Pipeline

- Pub/Sub topic
- Dataflow job
- BigQuery sink
- Dead-letter handling

These are exact replicas of real jobs.

🚨 COMMON PHASE 2 FAILURES (Seen at Work)

- Assuming input is clean

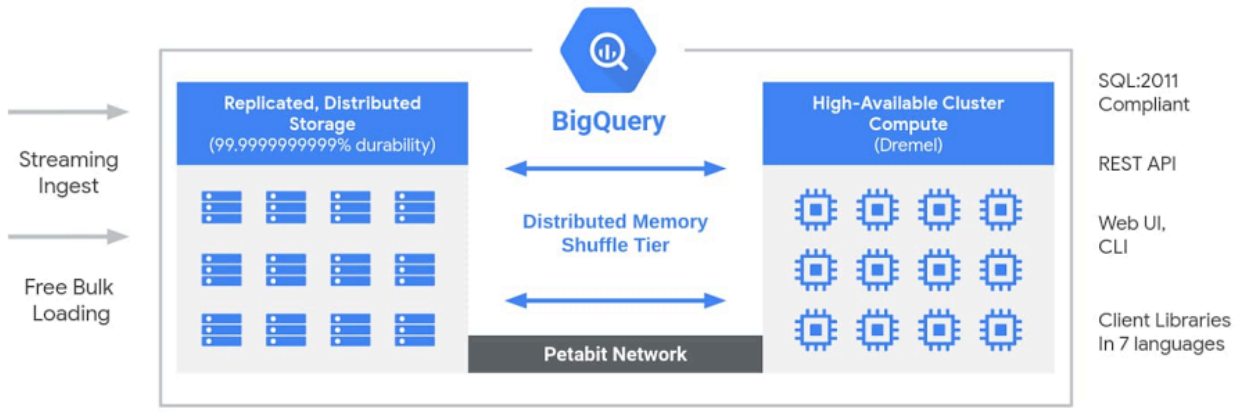
- Hardcoding schemas
- No retries
- No logging
- No backfill strategy

Avoid these and you are already above average.

✅ PHASE 2 OUTCOME

After Phase 2, the learner can:

- Design ingestion pipelines
- Handle batch & streaming data
- Debug ingestion failures
- Manage schemas safely
- Build production-ready ingestion systems



Clustered and Partitioned Tables

Orders table
Not Clustered; Not partitioned

Order_Date	Country	Status
2022-08-02	US	Shipped
2022-08-04	JP	Shipped
2022-08-05	UK	Canceled
2022-08-06	KE	Shipped
2022-08-02	KE	Canceled
2022-08-05	US	Processing
2022-08-04	JP	Processing
2022-08-04	KE	Shipped
2022-08-06	UK	Canceled
2022-08-02	UK	Processing
2022-08-05	JP	Canceled
2022-08-06	UK	Processing
2022-08-05	US	Shipped
2022-08-06	JP	Processing
2022-08-02	KE	Shipped
2022-08-04	US	Shipped

Orders table
Clustered by Country; Not partitioned

Order_Date	Country	Status
2022-08-04	JP	Shipped
2022-08-04	JP	Processing
2022-08-05	JP	Canceled
2022-08-06	JP	Processing
2022-08-06	KE	Shipped
2022-08-02	KE	Canceled
2022-08-04	KE	Shipped
2022-08-02	KE	Shipped
2022-08-05	UK	Processing
2022-08-06	UK	Canceled
2022-08-02	UK	Canceled
2022-08-06	UK	Processing
2022-08-02	US	Shipped
2022-08-05	US	Processing
2022-08-05	US	Shipped
2022-08-04	US	Shipped

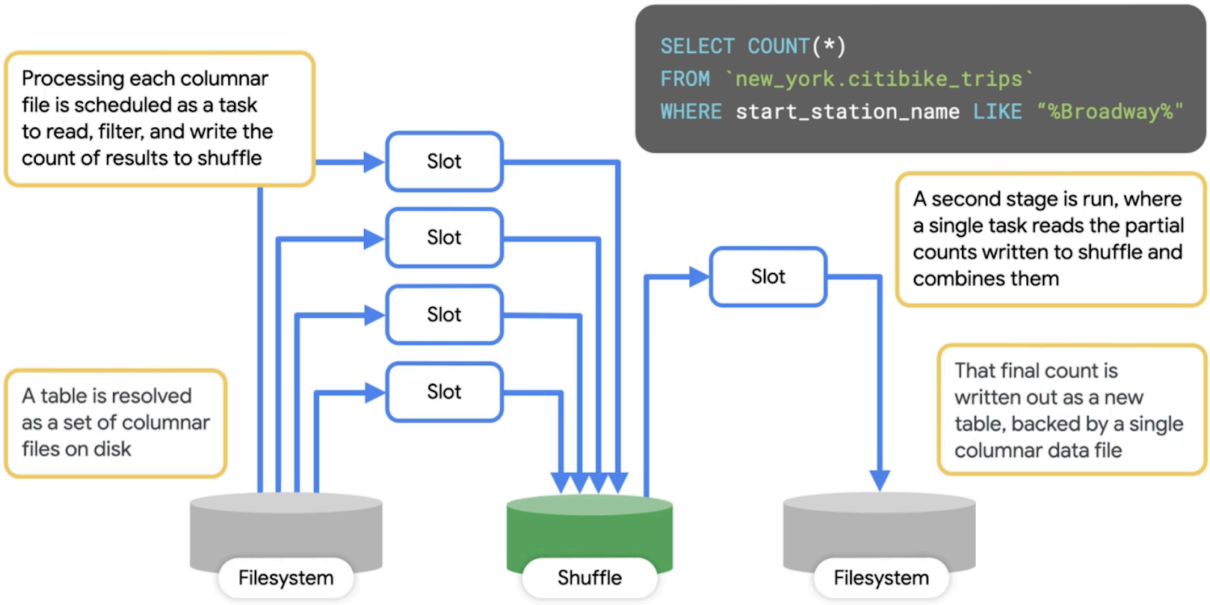
Orders table
Clustered by Country; Partitioned by Order_Date (Daily)

	Order_Date	Country	Status
Partition: 2022-08-02	2022-08-02	KE	Shipped
	2022-08-02	KE	Canceled
Clusters: Country	2022-08-02	UK	Processing
	2022-08-02	US	Shipped

	Order_Date	Country	Status
Partition: 2022-08-04	2022-08-04	JP	Shipped
	2022-08-04	JP	Processing
Cluster: Country	2022-08-04	KE	Shipped
	2022-08-04	US	Shipped

	Order_Date	Country	Status
Partition: 2022-08-05	2022-08-05	JP	Canceled
	2022-08-05	UK	Canceled
Cluster: Country	2022-08-05	US	Shipped
	2022-08-05	US	Processing

	Order_Date	Country	Status
Partition: 2022-08-06	2022-08-06	JP	Processing
	2022-08-06	KE	Shipped
Cluster: Country	2022-08-06	UK	Canceled
	2022-08-06	UK	Processing



5

🟨 PHASE 3

BIGQUERY – THE CORE SKILL OF A GCP DATA ENGINEER

Career Reality:
If you are weak at BigQuery, you will struggle as a GCP Data Engineer — no matter how good your pipelines are.

BigQuery is where:

- Data becomes business-ready
- Costs explode or get controlled
- Trust is built or destroyed

📘 MODULE 3.1

BigQuery Architecture (How It Actually Works)

3.1.1 Why Understanding Architecture Matters

Most BigQuery mistakes happen because engineers:

- Think like traditional databases
- Ignore how BigQuery scans data
- Don't understand cost mechanics

Architecture knowledge = fewer incidents + lower bills.

3.1.2 BigQuery Is Serverless (But Not Free)

Key components:

- Storage layer
- Compute (slots)
- Execution engine

Important implication:

You don't manage servers — but you are fully responsible for cost and performance.

3.1.3 Storage vs Compute (Critical Concept)

Storage

- Columnar
- Highly compressed
- Cheap

Compute

- Slot-based
- Charged per query / reservation
- Where money burns

If a query scans unnecessary columns or rows → cost spikes.

3.1.4 How a Query Executes (Simplified)

1. SQL parsed
2. Execution plan created
3. Slots allocated
4. Data scanned
5. Results aggregated
6. Cost calculated

As a Data Engineer, you must ask:

"How much data will this query scan?"

📘 MODULE 3.2

BigQuery Data Layers (Enterprise Standard)

3.2.1 Why Data Layers Exist

Real companies never query raw data directly for analytics.

Standard layers:

RAW → STAGING → CURATED → MART

3.2.2 Raw Layer

- Ingested data
- Append-only
- No transformations

Purpose: Recovery, audit, reprocessing

3.2.3 Staging Layer

- Light cleaning
- Type casting
- Deduplication

Purpose: Prepare for business logic

3.2.4 Curated Layer

- Business logic applied
- Clean schemas
- Trusted data

Purpose: Analytics, dashboards, ML

3.2.5 Data Marts

- Domain-specific views
- Optimized for queries

Purpose: Performance & usability

📘 MODULE 3.3

Data Modeling for Analytics (This Separates Juniors from Seniors)

3.3.1 Why Data Modeling Matters

Bad modeling causes:

- Slow dashboards
- Confusing metrics
- Business mistrust

Good modeling:

- Speeds up queries
- Reduces cost
- Improves adoption

3.3.2 Star Schema (Most Common)

- Fact table (metrics)
- Dimension tables (context)

Example:

- fact_orders
- dim_customer
- dim_date

BigQuery LOVES this structure.

3.3.3 Snowflake Schema (When Needed)

- Normalized dimensions
- Less duplication

- Slightly complex queries

Used when dimensions are large or reused heavily.

3.3.4 Slowly Changing Dimensions (SCD)

You must handle:

- Attribute changes over time
- Historical correctness

Types you will see at work:

- SCD Type 1 (overwrite)
- SCD Type 2 (history)

📘 MODULE 3.4

Partitioning & Clustering (Cost Control Weapons)

3.4.1 Partitioning (MANDATORY)

Partition by:

- Date (most common)
- Ingestion time

Why:

- Limits scanned data
- Massive cost reduction

Never create large unpartitioned tables.

3.4.2 Clustering (Performance Booster)

Cluster by:

- High-cardinality columns

- Common filter columns

Partition → reduces data
Clustering → speeds access inside partitions

3.4.3 Common Mistakes

❌ Partitioning on low-cardinality fields
❌ Over-clustering
❌ Ignoring query patterns

These increase cost instead of reducing it.

📘 MODULE 3.5

Query Optimization (Daily Job Skill)

3.5.1 How Queries Become Expensive

- SELECT *
- Missing filters
- Joining raw tables
- Cartesian joins

3.5.2 Optimization Techniques

- Select only needed columns
- Filter early
- Use partition filters
- Pre-aggregate when needed
- Avoid unnecessary joins

3.5.3 Cost Awareness Rule

If you don't know how much data your query scans, you are not production-ready.

📘 MODULE 3.6

Incremental Loads & Idempotency

3.6.1 Why Full Reloads Fail at Scale

- Slow
- Expensive
- Risky

3.6.2 Incremental Patterns

- Load only new data
- Use watermark columns
- Merge safely

Common tools:

- MERGE
- INSERT OVERWRITE
- Partition-based loads

3.6.3 Idempotent Design

Rerunning a job should:

- Not duplicate data
- Not corrupt tables

This is non-negotiable in production.

📘 MODULE 3.7

BigQuery Cost Management (Career Safety)

3.7.1 Where Costs Come From

- Query scans
- Streaming inserts
- Storage growth
- Poor design

3.7.2 What Data Engineers Control

- Table design
- Query patterns
- Retention policies
- Aggregation strategies

Senior engineers are trusted because they control cost.

🧪 PHASE 3 – REAL-WORLD PROJECTS

Project 1: Raw → Curated BigQuery Pipeline

- Partitioned tables
- Schema evolution
- Incremental loads

Project 2: Query Optimization Challenge

- Reduce scan size

- Improve runtime
- Cut cost by 70%+

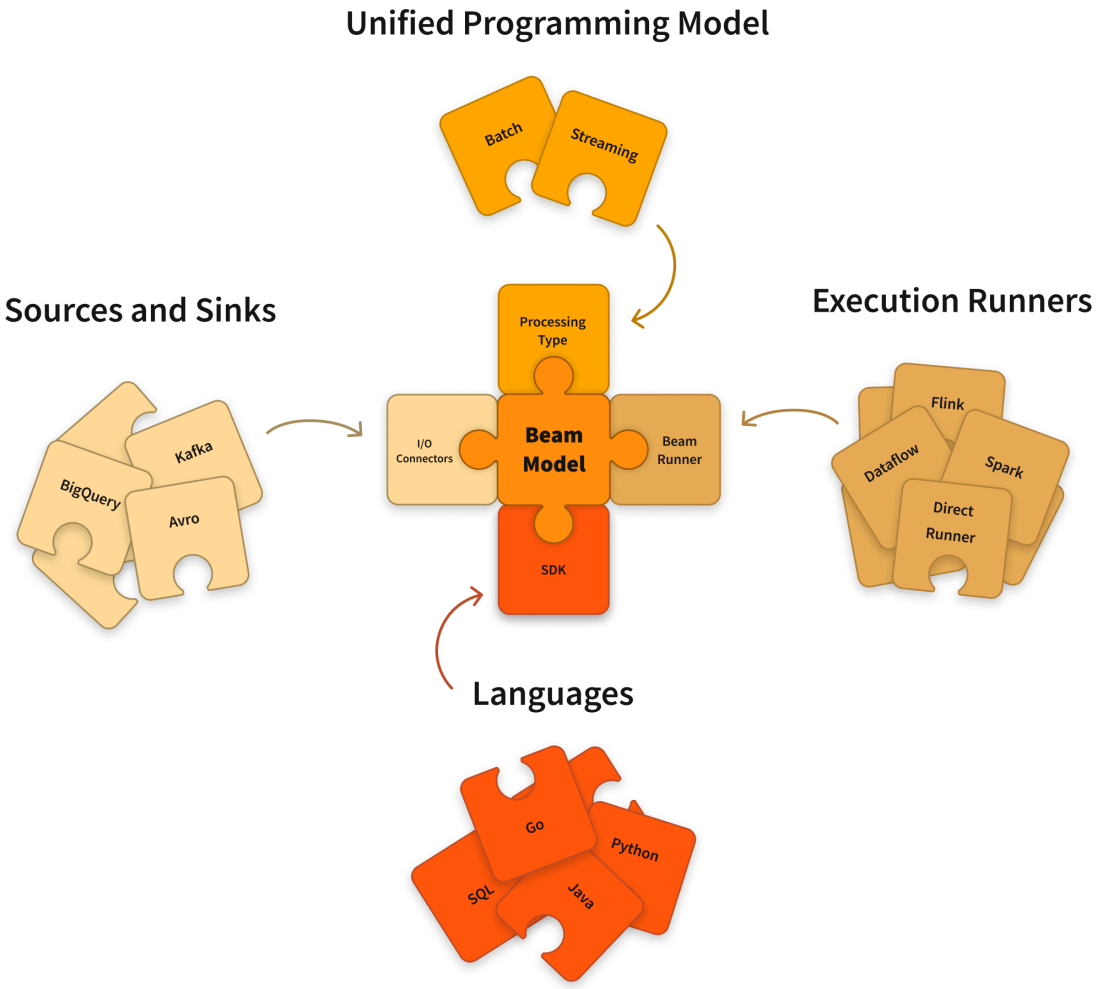
Project 3: Data Modeling Exercise

- Build star schema
- Create marts
- Optimize for dashboards

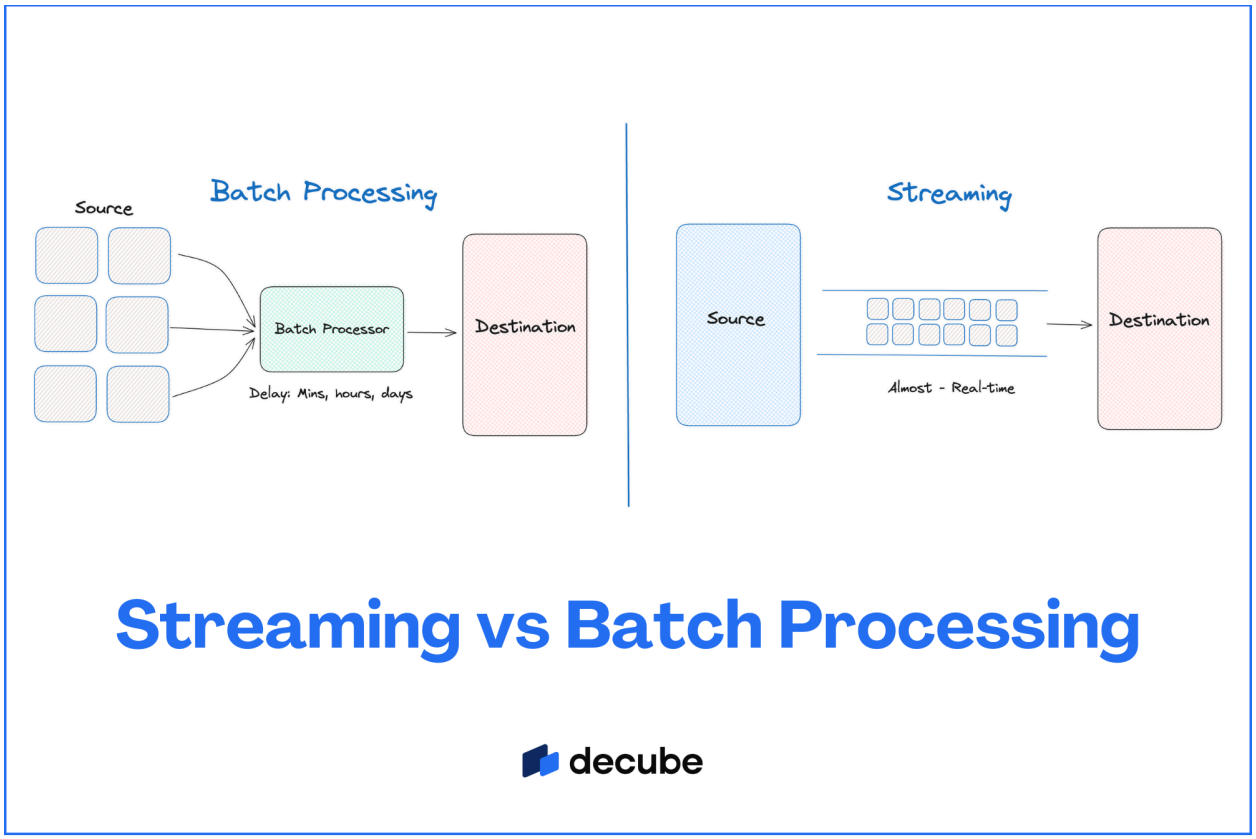
🚨 COMMON BIGQUERY FAILURES (Seen in Real Jobs)

- No partitioning
- SELECT *
- Business logic in raw layer
- No incremental strategy
- Ignoring cost alerts

Avoiding these already puts you above many professionals.


Unified Programming Model
Batch
Streaming
Sources and Sinks
Execution Runners
Processing Type
I/O Connectors
Beam Model
Beam Runner
SDK
Kafka
BigQuery
Avro
Flink
Dataflow
Spark
Direct Runner
Languages
Go
Python
SQL
Java





5

🟧 PHASE 4

DATAFLOW & DATA PROCESSING (BATCH + STREAMING – REAL PRODUCTION)

Career Reality:
Dataflow is used when BigQuery SQL is not enough.
If you misuse it, costs explode. If you master it, you become valuable.

📘 MODULE 4.1

Why Dataflow Exists (And When NOT to Use It)

4.1.1 The Biggest Misconception

❌ “Dataflow is for everything”
❌ “Dataflow replaces BigQuery SQL”

Both are wrong.

4.1.2 When Dataflow IS the Right Tool

Use Dataflow when you need:

- Complex transformations
- Streaming processing
- Event-time logic
- Deduplication at scale
- Stateful processing
- Late data handling

If SQL can do it cheaply and safely, prefer SQL.

4.1.3 Batch vs Streaming (Operational Difference)

Batch Dataflow

- Finite input
- Scheduled
- Predictable cost
- Easier debugging

Streaming Dataflow

- Infinite input
- Always running
- Cost-sensitive
- Requires discipline

Streaming jobs are production services, not scripts.

📘 MODULE 4.2

Apache Beam Fundamentals (Only What You Need)

4.2.1 Beam Mental Model (Simplified)

Apache Beam =
Data + Transforms + Execution Engine

You write:

- Pipeline logic

Dataflow handles:

- Scaling
- Parallelism
- Fault tolerance

4.2.2 Core Concepts You MUST Know

- PCollection → Data
- PTransform → Operation
- Pipeline → Workflow

Ignore advanced theory. Focus on behavior.

4.2.3 Typical Pipeline Structure (Python)

```
with beam.Pipeline(options=pipeline_options) as p:
  (
    p
    | "Read" >> ReadFromSource()
    | "Transform" >> beam.Map(transform_fn)
```

```
    | "Write" >> WriteToSink()
)
```

This pattern appears everywhere in production.

📘 MODULE 4.3

Batch Processing with Dataflow (REAL USE CASES)

4.3.1 Common Batch Scenarios

- Large file transformations
- Enrichment joins
- Deduplication
- Backfills

4.3.2 Batch Design Rules

- Idempotent processing
- Safe retries
- Partition-aware logic
- Output validation

If batch jobs are not idempotent, reruns will corrupt data.

📘 MODULE 4.4

Streaming Processing (Where Most Engineers Fail)

4.4.1 Streaming Is NOT Just “Faster Batch”

Streaming requires understanding:

- Event time vs processing time
- Windowing
- Triggers
- Late data

Ignoring these causes:

- Wrong metrics
- Silent data loss
- Business distrust

4.4.2 Event Time vs Processing Time

- Event time → When event occurred
- Processing time → When system received it

Correct analytics require event time.

4.4.3 Windowing (Critical Concept)

Windows group events in time:

- Fixed windows (most common)
- Sliding windows
- Session windows

Without windowing, streaming analytics are meaningless.

4.4.4 Triggers (Advanced but Real)

Triggers decide when results are emitted.

Common patterns:

- Early results

- On-time results
- Late results

Used for:

- Dashboards
- Alerts
- SLAs

📘 MODULE 4.5

Handling Late & Duplicate Data (Real Pain Point)

4.5.1 Why Late Data Happens

- Network delays
- Mobile apps offline
- IoT devices

Late data is normal, not a bug.

4.5.2 Deduplication Strategies

- Unique event IDs
- State-based dedupe
- Time-bounded dedupe

Failing this causes:

- Double counting
- Incorrect metrics
- Executive escalations

📘 MODULE 4.6

Dataflow Performance & Cost Control

4.6.1 Where Costs Come From

- Worker count
- Streaming job runtime
- Inefficient transforms
- Excessive reshuffles

4.6.2 Cost Control Rules

- Use autoscaling wisely
- Avoid unnecessary state
- Batch when possible
- Stop unused streaming jobs

Senior engineers are trusted because they control Dataflow cost.

📘 MODULE 4.7

Debugging Dataflow Jobs (On-Call Reality)

4.7.1 Common Failures

- Schema mismatch
- Serialization errors
- Memory pressure
- Permission issues

4.7.2 Debugging Workflow

1. Check job status
2. Inspect logs
3. Identify failing transform
4. Fix code or config
5. Rerun safely

This is weekly work in real teams.

🧪 PHASE 4 – REAL-WORLD PROJECTS

Project 1: Batch Transformation Pipeline

- Read from GCS
- Transform data
- Write to BigQuery curated layer

Project 2: Streaming Events Pipeline

- Pub/Sub source
- Windowed aggregation
- BigQuery sink

Project 3: Deduplication Pipeline

- Handle duplicates
- Late data
- Correct counts

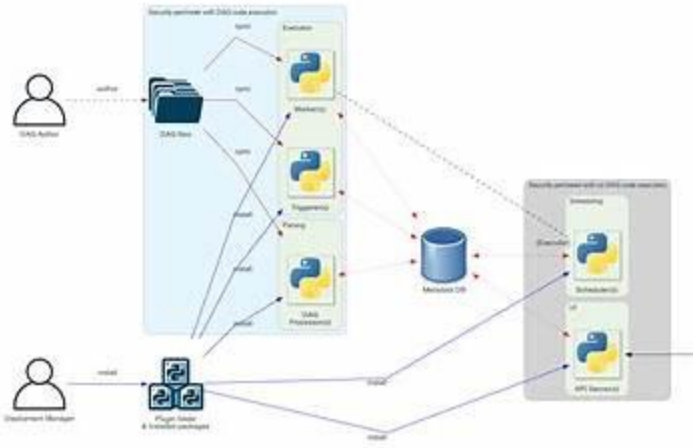
These projects mirror actual enterprise pipelines.

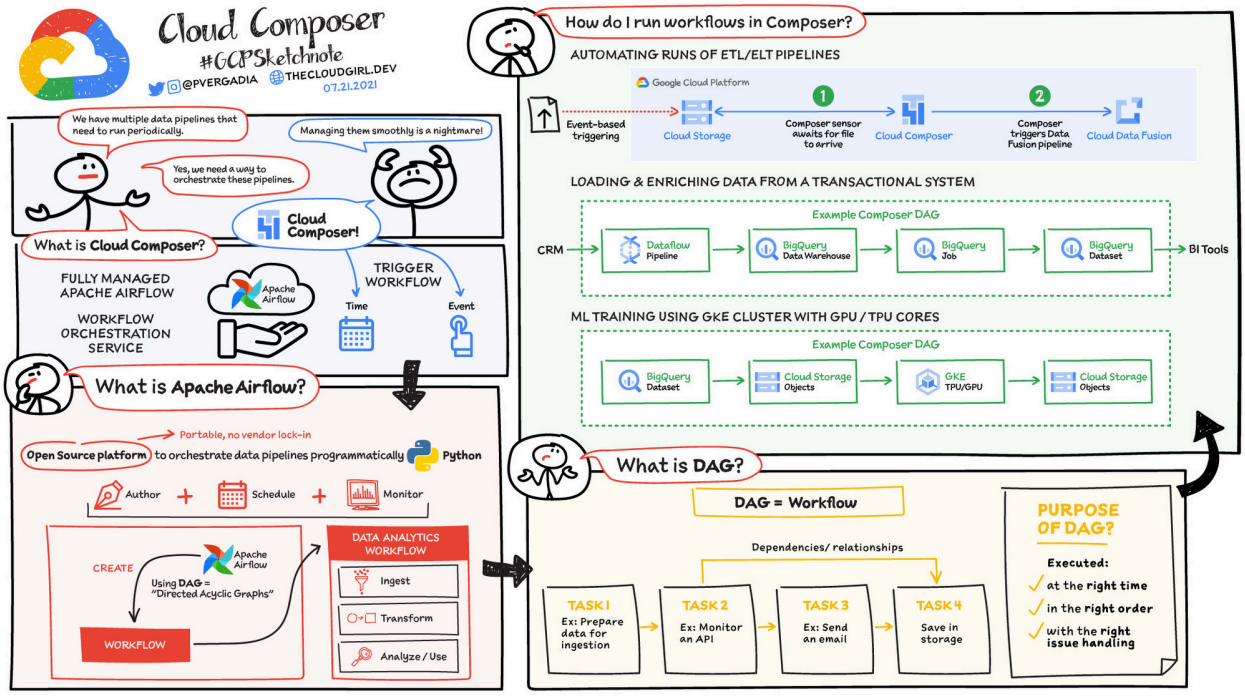
🚨 COMMON DATAFLOW FAILURES (Seen in Jobs)

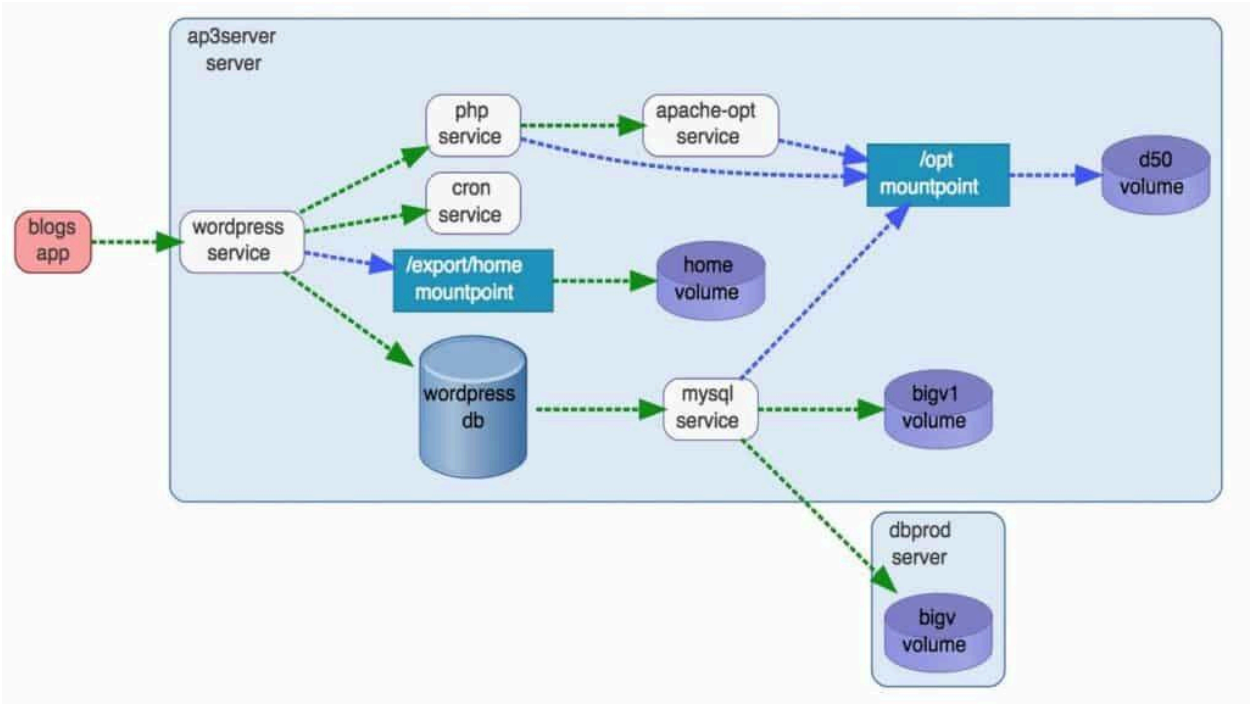
- Using Dataflow when SQL is enough

- No windowing in streaming
- Ignoring late data
- Letting streaming jobs run forever
- No cost monitoring

Avoiding these makes you senior-capable.




Cloud Composer
#GCPSketchnote
@PVERGADIA
THECLOUDGIRL.DEV
07.21.2021
We have multiple data pipelines that need to run periodically.
Managing them smoothly is a nightmare!
Yes, we need a way to orchestrate these pipelines.
Cloud Composer!
What is Cloud Composer?
FULLY MANAGED APACHE AIRFLOW
WORKFLOW ORCHESTRATION SERVICE
TRIGGER WORKFLOW
Time
Event
What is Apache Airflow?
Open Source platform
Portable, no vendor lock-in
to orchestrate data pipelines programmatically
Python
Author
Schedule
Monitor
CREATE
Using DAG = "Directed Acyclic Graphs"
WORKFLOW
DATA ANALYTICS WORKFLOW
Ingest
Transform
Analyze / Use
How do I run workflows in Composer?
AUTOMATING RUNS OF ETL/ELT PIPELINES
Google Cloud Platform
Event-based triggering
Cloud Storage
1
Composer sensor awaits for file to arrive
Cloud Composer
2
Composer triggers Data Fusion pipeline
Cloud Data Fusion
LOADING & ENRICHING DATA FROM A TRANSACTIONAL SYSTEM
Example Composer DAG
CRM
Dataflow Pipeline
BigQuery Data Warehouse
BigQuery Job
BigQuery Dataset
BI Tools
ML TRAINING USING GKE CLUSTER WITH GPU / TPU CORES
Example Composer DAG
BigQuery Dataset
Cloud Storage Objects
GKE TPU/GPU
Cloud Storage Objects
What is DAG?
DAG = Workflow
Dependencies/ relationships
TASK 1
Ex: Prepare data for ingestion
TASK 2
Ex: Monitor an API
TASK 3
Ex: Send an email
TASK 4
Save in storage
PURPOSE OF DAG?
Executed:
at the right time
in the right order
with the right issue handling


ap3server
server
blogs app
wordpress service
php service
cron service
/export/home mountpoint
wordpress db
apache-opt service
mysql service
/opt mountpoint
home volume
bigv1 volume
d50 volume
dbprod server
bigv volume

🟦 PHASE 5

ORCHESTRATION WITH CLOUD COMPOSER (APACHE AIRFLOW)

Career Reality:
Most data failures in companies are not code bugs —
they are orchestration failures.

If you master Airflow, you become the engineer everyone depends on.

📘 MODULE 5.1

Why Orchestration Exists (And Why Cron Jobs Fail)

5.1.1 The Problem Orchestration Solves

Without orchestration:

- Jobs run in wrong order
- Failures go unnoticed
- Reruns corrupt data
- Dependencies are unclear

Cron ≠ Orchestration.

5.1.2 What Airflow Actually Does

Airflow:

- Defines what runs
- Defines when it runs
- Defines dependencies
- Tracks state
- Enables safe retries & reruns

It does not:

- Process data
- Replace Dataflow or BigQuery

5.1.3 Cloud Composer in GCP (Enterprise Reality)

Cloud Composer is:

- Managed Apache Airflow
- Integrated with GCP IAM
- Used in almost every serious GCP data platform

As a Data Engineer, Composer is part of your daily life.

📘 MODULE 5.2

DAG Fundamentals (The Language of Orchestration)

5.2.1 What a DAG Really Is

DAG = Directed Acyclic Graph

In real terms:

- Nodes = tasks
- Edges = dependencies
- No cycles allowed

5.2.2 Typical DAG Structure (Real Example)

start

↓

ingest_data

↓

validate_data

↓

transform_data

↓

load_bigquery

↓

end

This structure mirrors business logic, not tool logic.

5.2.3 DAG Design Rule (Non-Negotiable)

A DAG should reflect business dependency, not technical convenience.

Bad DAGs cause:

- Fragile pipelines
- Painful reruns
- On-call nightmares

📘 MODULE 5.3

Operators & Tasks (What You Use Daily)

5.3.1 Common Operators for Data Engineers

- PythonOperator
- BigQueryInsertJobOperator
- DataflowStartFlexTemplateOperator
- BashOperator (use sparingly)

You orchestrate — you don't embed heavy logic in DAGs.

5.3.2 Task Responsibilities

A task should:

- Do ONE thing
- Be retryable
- Be idempotent
- Fail loudly

If a task does too much, debugging becomes impossible.

📘 MODULE 5.4

Scheduling & Time (Where Most Engineers Make Mistakes)

5.4.1 Execution Date (Critical Concept)

Airflow schedules based on:

- Logical time, not wall-clock time

This confuses many engineers and causes:

- Missing data
- Double processing
- Incorrect backfills

Understanding this is mandatory.

5.4.2 Common Scheduling Patterns

- Hourly ingestion
- Daily batch loads
- Weekly aggregation
- Monthly reporting

Schedules must align with data availability, not convenience.

📘 MODULE 5.5

Idempotency & Safe Reruns (Career-Saving Skill)

5.5.1 Why Reruns Are Normal

In real jobs:

- Jobs fail
- Data arrives late
- Bugs happen

Reruns are expected.

5.5.2 Idempotent Pipeline Rule

Rerunning a DAG must never corrupt data.

Achieved via:

- Partition-based loads
- MERGE instead of INSERT
- Overwrite logic
- Deduplication

If reruns are unsafe, your pipeline is not production-ready.

📘 MODULE 5.6

Backfills (Real-World, High-Risk Operations)

5.6.1 What Backfills Are

Backfills = processing historical data again.

Common reasons:

- Bug fixes
- Logic changes
- Data corrections

5.6.2 Backfill Safety Rules

- Run in non-prod first
- Validate row counts
- Monitor cost
- Communicate with stakeholders

Bad backfills cause:

- Massive bills
- Business disruption
- Loss of trust

📘 MODULE 5.7

Monitoring, Alerts & On-Call Reality

5.7.1 What You Monitor Daily

- DAG failures
- Task retries
- SLA misses
- Long-running jobs

5.7.2 Alert Fatigue (Real Problem)

Too many alerts = ignored alerts.

Good engineers:

- Alert only on actionable failures
- Suppress noise
- Escalate real issues

📘 MODULE 5.8

Common Airflow Failures (Seen in Real Teams)

❌ Monolithic DAGs
❌ Logic inside DAG files
❌ No retries configured
❌ Unsafe backfills
❌ Poor naming

Avoiding these makes you instantly senior-capable.

🧪 PHASE 5 – REAL-WORLD PROJECTS

Project 1: End-to-End Orchestration

- Ingestion DAG
- Validation task
- Dataflow trigger
- BigQuery load

Project 2: Failure & Rerun Simulation

- Break a task
- Debug failure
- Rerun safely

- Validate results

Project 3: Backfill Scenario

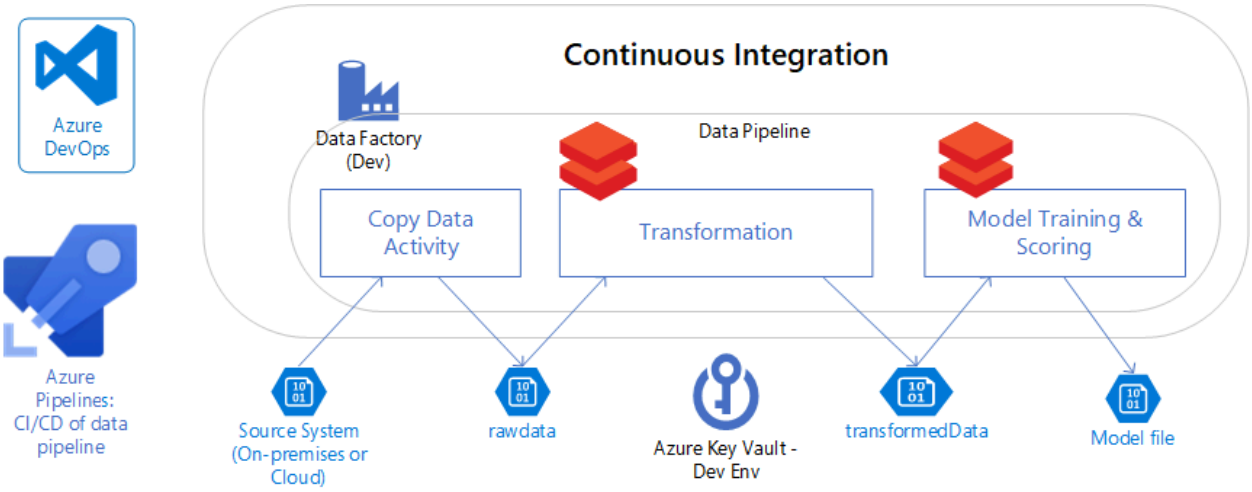
- Historical reprocessing
- Cost-aware execution
- Business communication

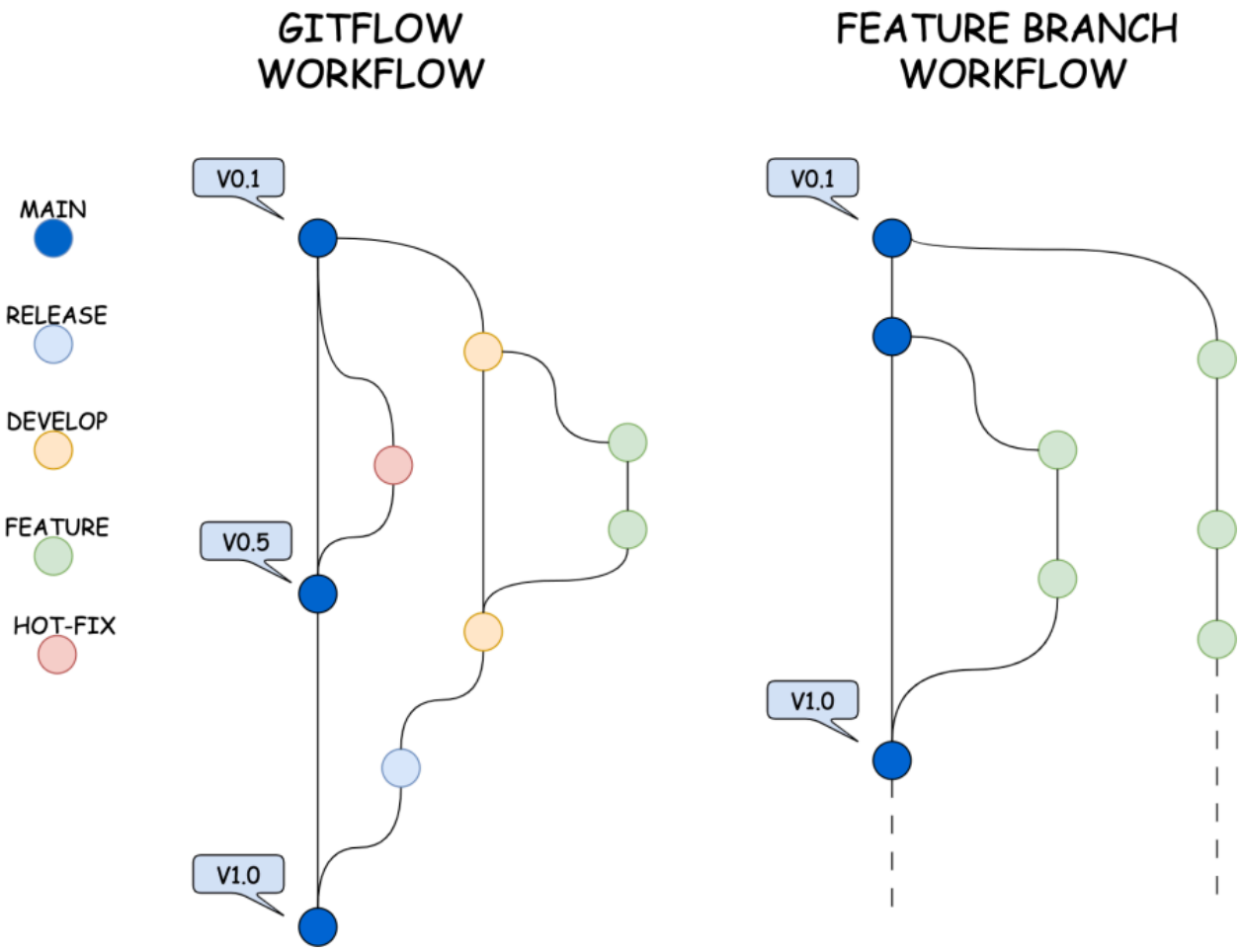
These mirror actual on-call responsibilities.

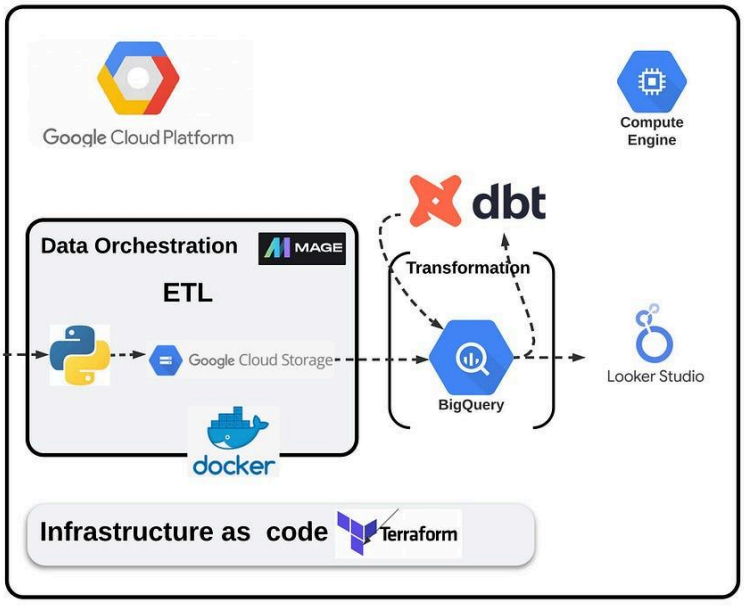
🚨 WHY ENGINEERS FAIL AT THIS PHASE

- Treat Airflow as “just scheduling”
- Don’t design for failure
- Fear reruns
- Ignore cost impact

Mastering orchestration makes you trusted.




GITFLOW WORKFLOW
FEATURE BRANCH WORKFLOW
MAIN
RELEASE
DEVELOP
FEATURE
HOT-FIX
V0.1
V0.5
V1.0
V0.1
V1.0


Google Cloud Platform
Compute Engine
dbt
Data Orchestration
MAGE
ETL
Transformation
UK Cycling Data
Google Cloud Storage
BigQuery
Looker Studio
docker
Infrastructure as code
Terraform

5

🟥 PHASE 6

CI/CD & DEVOPS FOR DATA ENGINEERS (PRODUCTION EXCELLENCE)

Career Reality:
Companies don't promote engineers who "make things work once."
They promote engineers who make systems safe, repeatable, and deployable.

📘 MODULE 6.1

Why CI/CD Matters for Data Engineers (Not Optional Anymore)

6.1.1 The Old Way (And Why It Failed)

Old pattern:

- Edit pipeline code directly
- Deploy manually
- Fix things in production
- Hope nothing breaks

Results:

- Broken data
- Late nights
- Blame games
- Zero trust

6.1.2 The Modern Data Engineering Reality

Modern data platforms require:

- Version control
- Automated testing
- Safe deployments
- Rollback capability

CI/CD is no longer “DevOps stuff.”
It is core Data Engineering.

📘 MODULE 6.2

Git Fundamentals for Data Engineers (Real Usage)

6.2.1 What Goes into Git (At Work)

You version-control:

- Ingestion scripts
- Dataflow code
- Airflow DAGs
- SQL transformations
- Terraform configs

You do NOT version-control:

- Secrets
- Credentials
- Generated data

6.2.2 Branching Strategy (Enterprise Standard)

Recommended Model

- main → production
- develop → integration

- feature/* → individual work

Why this works:

- Protects prod
- Enables collaboration
- Supports reviews

6.2.3 Pull Requests & Code Reviews

Every production change should:

- Go through PR
- Be reviewed by another engineer
- Be traceable

Senior engineers review for:

- Safety
- Idempotency
- Cost impact
- Rollback plan

📘 MODULE 6.3

Environment Promotion (Dev → Stage → Prod)

6.3.1 Why Direct-to-Prod Is a Career Risk

Direct prod changes cause:

- Data corruption
- Business outages
- Executive escalations

Safe engineers never skip environments.

6.3.2 Promotion Flow (Standard)

DEV → STAGE → PROD

Each stage validates:

- Logic
- Performance
- Cost
- Security

6.3.3 Configuration Separation

Code stays the same.
Configs change per environment.

Examples:

- Project ID
- Dataset names
- Bucket names
- Schedule frequency

This is mandatory for scalability.

📘 MODULE 6.4

Testing for Data Pipelines (Often Ignored, Very Important)

6.4.1 Types of Tests Data Engineers Use

Unit Tests

- Test transformation logic

- Validate edge cases

Integration Tests

- Test pipeline end-to-end
- Validate schema & row counts

Data Quality Checks

- Null checks
- Range checks
- Duplicate detection

You don't test everything —
you test what can break trust.

6.4.2 What Happens Without Tests

- Silent data corruption
- Wrong dashboards
- Business mistrust
- Emergency fixes

Tests protect reputation, not just code.

📘 MODULE 6.5

CI Pipelines for Data Engineers

6.5.1 What a CI Pipeline Typically Does

On every pull request:

- Run linting
- Run unit tests
- Validate SQL

- Check formatting

On merge:

- Package code
- Deploy to environment
- Trigger pipelines

6.5.2 CI Is Not About Speed — It's About Safety

Slow, safe pipelines beat fast, risky ones.

📘 MODULE 6.6

Infrastructure as Code with Terraform (DE Edition)

6.6.1 Why Terraform Matters to Data Engineers

You manage:

- BigQuery datasets
- Tables
- GCS buckets
- IAM roles
- Service accounts

Manually creating these leads to:

- Drift
- Inconsistency
- Production surprises

6.6.2 What Data Engineers Typically Manage with Terraform

- BigQuery datasets & tables

- Bucket lifecycle rules
- IAM bindings
- Scheduler jobs
- Dataflow templates

You don't need to manage everything —
only what affects data reliability.

6.6.3 Terraform Best Practices (Real-World)

- One repo per platform
- Separate state per environment
- Never edit prod manually
- Review plans before apply

Terraform mistakes are expensive — discipline matters.

📘 MODULE 6.7

Secrets Management (High-Risk Area)

6.7.1 What NEVER Goes into Code

- API keys
- Database passwords
- Tokens

6.7.2 Proper Patterns

- Secret Manager
- Environment variables
- IAM-based access

Security incidents often start here.

📘 MODULE 6.8

Release Management & Rollbacks

6.8.1 Why Rollbacks Matter

Even good changes fail.

Without rollback:

- You're stuck
- Data stays broken
- Pressure escalates

6.8.2 Rollback Strategy

- Versioned deployments
- Feature flags (where applicable)
- Safe disable switches

Professionals plan failure paths, not just success.

🧪 PHASE 6 – REAL-WORLD PROJECTS

Project 1: Git-Based Pipeline Deployment

- Feature branch
- PR review
- Dev deployment
- Prod promotion

Project 2: Terraform-Managed Data Platform

- Create datasets
- Apply IAM
- Enforce lifecycle rules

Project 3: CI Failure Simulation

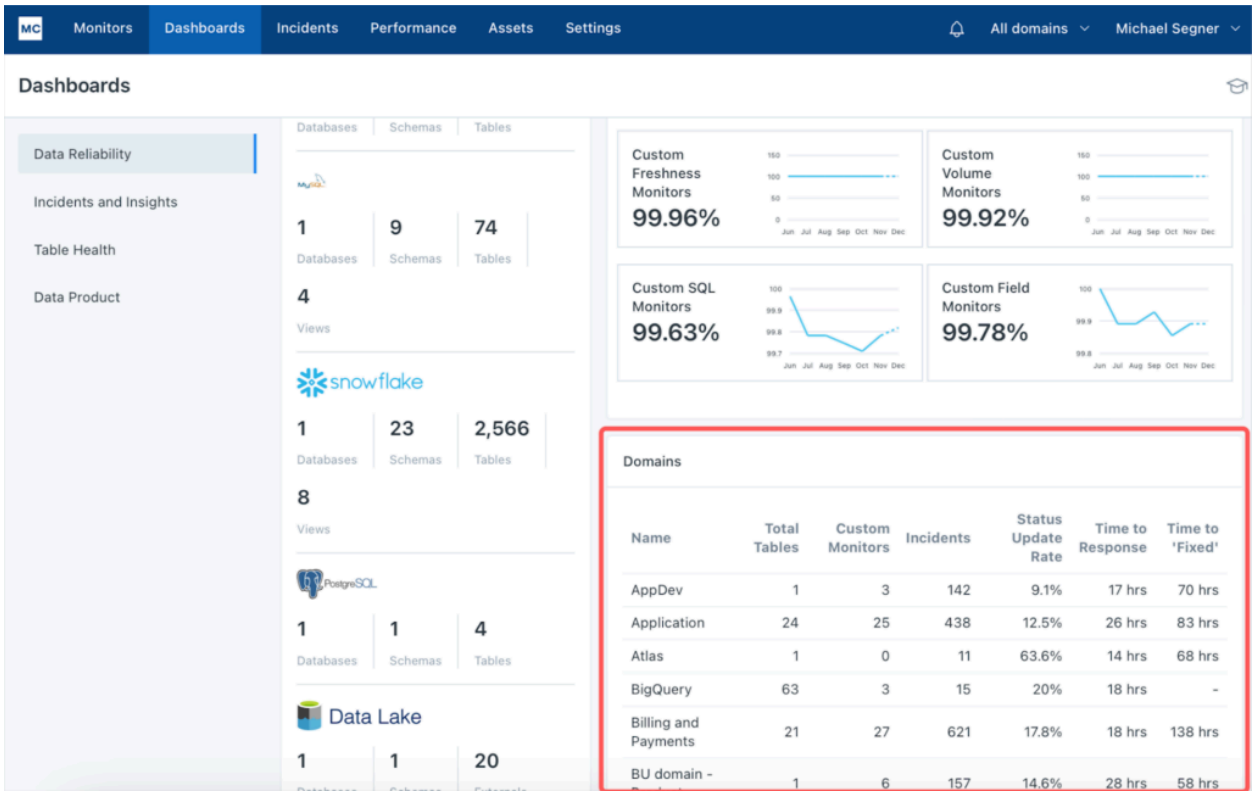
- Break a test
- Fix pipeline
- Redeploy safely

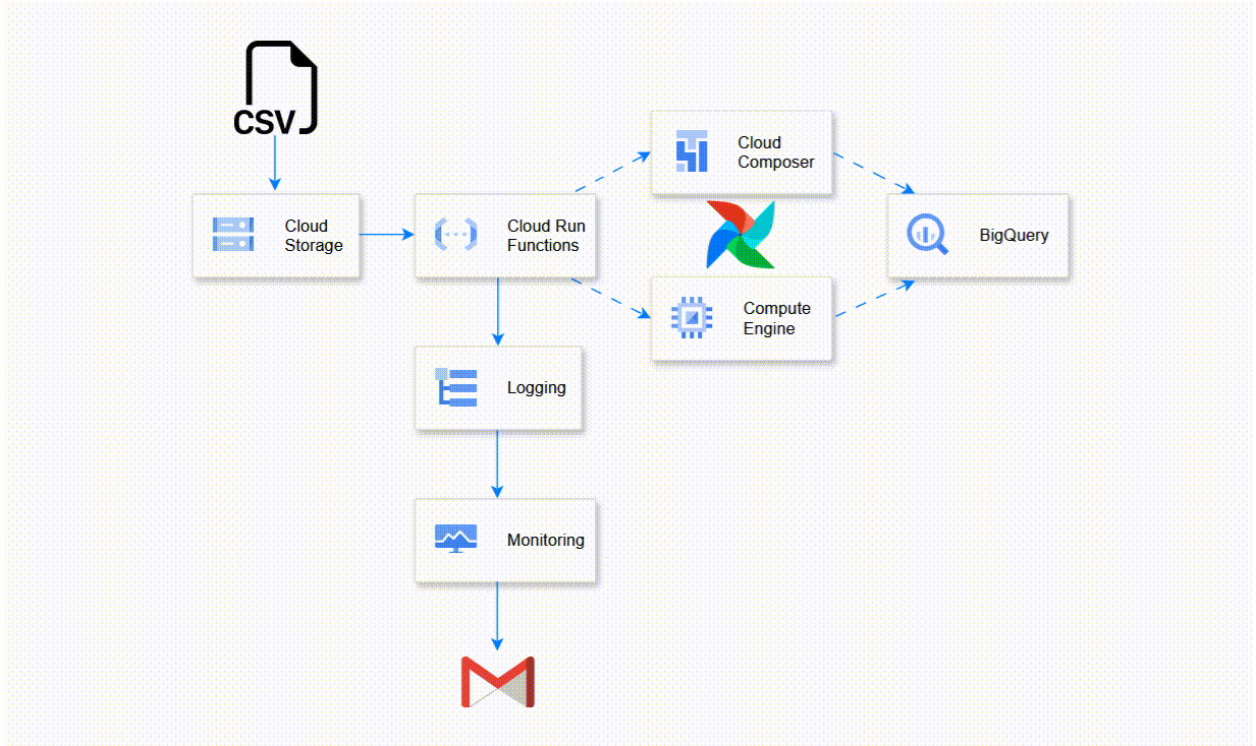
These reflect actual senior DE responsibilities.

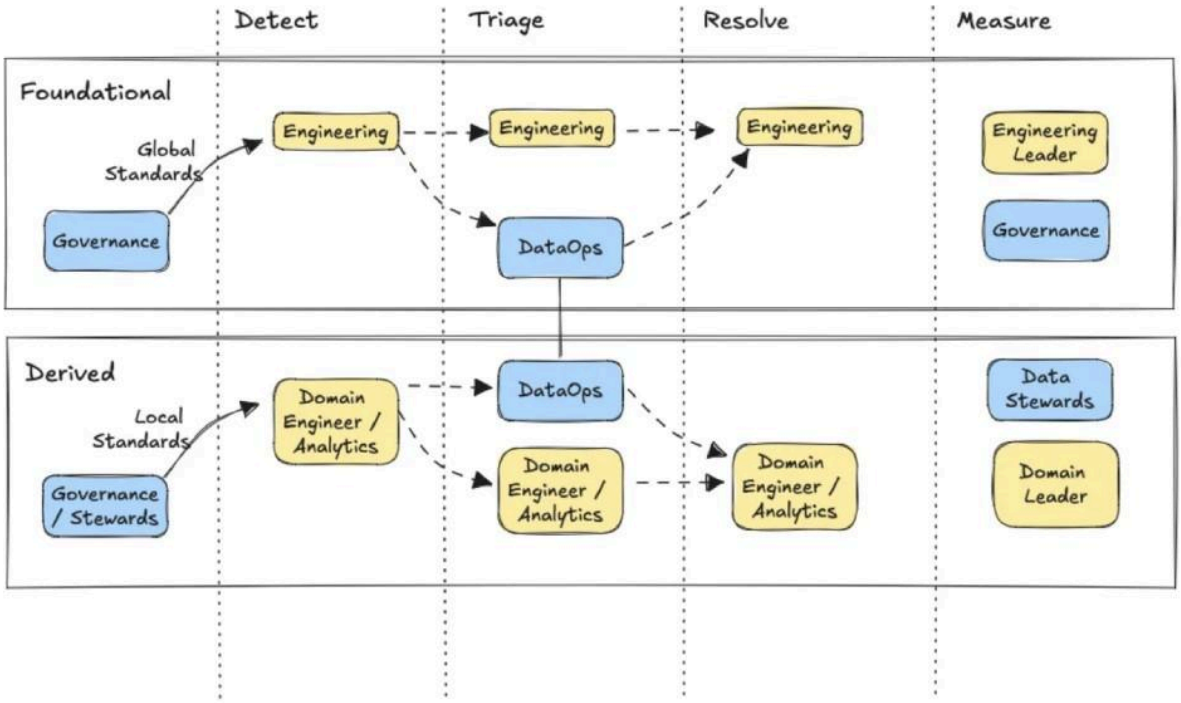
🚨 COMMON PHASE 6 FAILURES (Seen at Work)

❌ Manual prod changes
❌ No rollback plan
❌ No reviews
❌ Secrets in code
❌ One environment only

Avoiding these builds long-term trust.


Monitors
Dashboards
Incidents
Performance
Assets
Settings
All domains
Michael Segner
Dashboards
Data Reliability
Incidents and Insights
Table Health
Data Product
Databases
Schemas
Tables
1
9
74
Databases
Schemas
Tables
4
Views
snowflake
1
23
2,566
Databases
Schemas
Tables
8
Views
PostgreSQL
1
1
4
Databases
Schemas
Tables
Data Lake
1
1
20
Custom Freshness Monitors
99.96%
Custom Volume Monitors
99.92%
Custom SQL Monitors
99.63%
Custom Field Monitors
99.78%
Domains
Name
Total Tables
Custom Monitors
Incidents
Status Update Rate
Time to Response
Time to 'Fixed'
AppDev 1 3 142 9.1% 17 hrs 70 hrs
Application 24 25 438 12.5% 26 hrs 83 hrs
Atlas 1 0 11 63.6% 14 hrs 68 hrs
BigQuery 63 3 15 20% 18 hrs -
Billing and Payments 21 27 621 17.8% 18 hrs 138 hrs
BU domain - 1 6 157 14.6% 28 hrs 58 hrs


CSV
Cloud Storage
Cloud Run Functions
Cloud Composer
Compute Engine
BigQuery
Logging
Monitoring



5

🟫 PHASE 7

MONITORING, INCIDENTS & ON-CALL LIFE (REAL PRODUCTION REALITY)

Career Reality:
Senior Data Engineers are not judged by how rarely things fail.
They are judged by how calmly and correctly they respond when things do fail.

Failures are normal.
Uncontrolled failures destroy trust.

📘 MODULE 7.1

Why Monitoring Is a Core Data Engineering Skill

7.1.1 The Hard Truth

If you don't monitor your pipelines:

- You don't know when data is missing
- You don't know when costs spike
- You don't know when business is blind

Unmonitored pipelines = silent failure.

7.1.2 What Data Engineers Must Monitor (Always)

At minimum, you must monitor:

- Pipeline success / failure
- Data freshness
- Row count anomalies
- Job runtime
- Cost spikes

Dashboards are useless if pipelines are wrong.

📘 MODULE 7.2

Monitoring Stack in GCP (Practical, Not Theoretical)

7.2.1 Core Tools You Will Use

- Cloud Monitoring
- Cloud Logging
- Error Reporting
- BigQuery INFORMATION_SCHEMA
- Airflow UI (Composer)

You don’t need 20 tools.
You need the right signals.

7.2.2 Logs vs Metrics (Critical Difference)

Logs

- Detailed
- Diagnostic
- Used during incidents

Metrics

- Aggregated
- Alert-friendly
- Used to detect problems

Professionals design metrics first, logs second.

📘 MODULE 7.3

Alerts That Actually Help (Not Noise)

7.3.1 Why Alert Fatigue Happens

Common mistakes:

- Alerting on every failure
- No severity levels
- No clear ownership

Result:

Alerts get ignored until executives notice.

7.3.2 Good Alert Design Principles

A good alert must answer:

1. What broke?
2. How bad is it?
3. Who should act?
4. What is the first step?

If an alert causes panic instead of clarity, it's a bad alert.

7.3.3 Severity Levels (Enterprise Standard)

- SEV-1 → Business impact now
- SEV-2 → Data delayed / degraded
- SEV-3 → Non-critical issue

Not every failure is an emergency.

📘 MODULE 7.4

Data Freshness & Trust Monitoring (Most Important)

7.4.1 Why Freshness Matters More Than Accuracy (Initially)

Business would rather have:

- Slightly wrong data on time
  than
- Perfect data too late

Your job is to signal freshness clearly.

7.4.2 Common Freshness Checks

- Last updated timestamp
- Expected delivery window
- SLA violation tracking

Freshness checks prevent:

- Executive embarrassment
- Bad decisions
- Escalations

📘 MODULE 7.5

Incident Response: What ACTUALLY Happens On-Call

7.5.1 The On-Call Mindset Shift

On-call is not:

- Debugging perfectly
- Fixing everything immediately

On-call is:

- Containing damage
- Restoring service
- Communicating clearly

7.5.2 First 10 Minutes of an Incident (CRITICAL)

When paged:

1. Acknowledge alert
2. Assess business impact
3. Stop further damage
4. Communicate status

5. Begin investigation

Speed > perfection.

7.5.3 Common Data Incidents

- Pipeline didn't run
- Data missing in dashboards
- Duplicate data
- Cost spike overnight
- Schema breaking downstream systems

These are normal in production.

📘 MODULE 7.6

Incident Debugging Workflow (Battle-Tested)

Step 1: Identify Scope

- Which pipelines?
- Which datasets?
- Which consumers?

Step 2: Identify Failure Point

- Ingestion
- Processing
- Orchestration
- Permissions

Step 3: Apply Containment

- Pause downstream jobs
- Disable broken DAGs

- Roll back recent changes

Step 4: Fix or Work Around

- Hotfix
- Rerun
- Manual patch (document it!)

Step 5: Communicate Clearly

- What happened
- What's fixed
- What's pending

📘 MODULE 7.7

Communication During Incidents (Career-Defining Skill)

7.7.1 What Stakeholders Want

They do NOT want:

- Technical details
- Excuses
- Silence

They DO want:

- Impact summary
- ETA
- Confidence

7.7.2 Example Good Update

“Sales dashboard delayed due to ingestion failure.
Root cause identified.
Fix in progress. ETA 30 minutes.”

Clear. Calm. Trusted.

📘 MODULE 7.8

Post-Incident Reviews (How Seniors Improve Systems)

7.8.1 What a Postmortem Is NOT

❌ Blame session
❌ Resume of logs

7.8.2 What a Good Postmortem Includes

- What happened
- Why it happened
- Why it wasn’t detected earlier
- What will prevent it next time

Blameless, factual, forward-looking.

🧪 PHASE 7 – REAL-WORLD SIMULATIONS

Simulation 1: Missing Data Incident

- Pipeline failed silently
- Dashboard empty
- Executive escalation

Simulation 2: Cost Spike Alert

- BigQuery scan exploded
- Identify query
- Apply fix
- Prevent recurrence

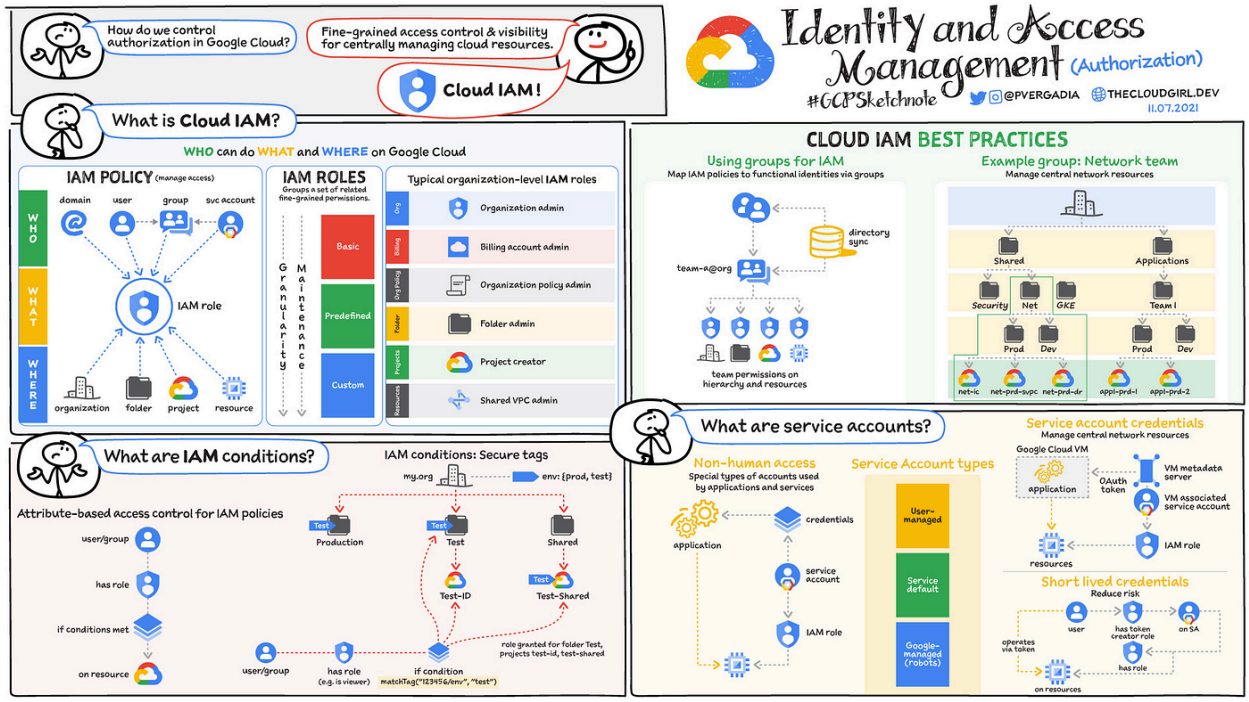
Simulation 3: Schema Change Incident

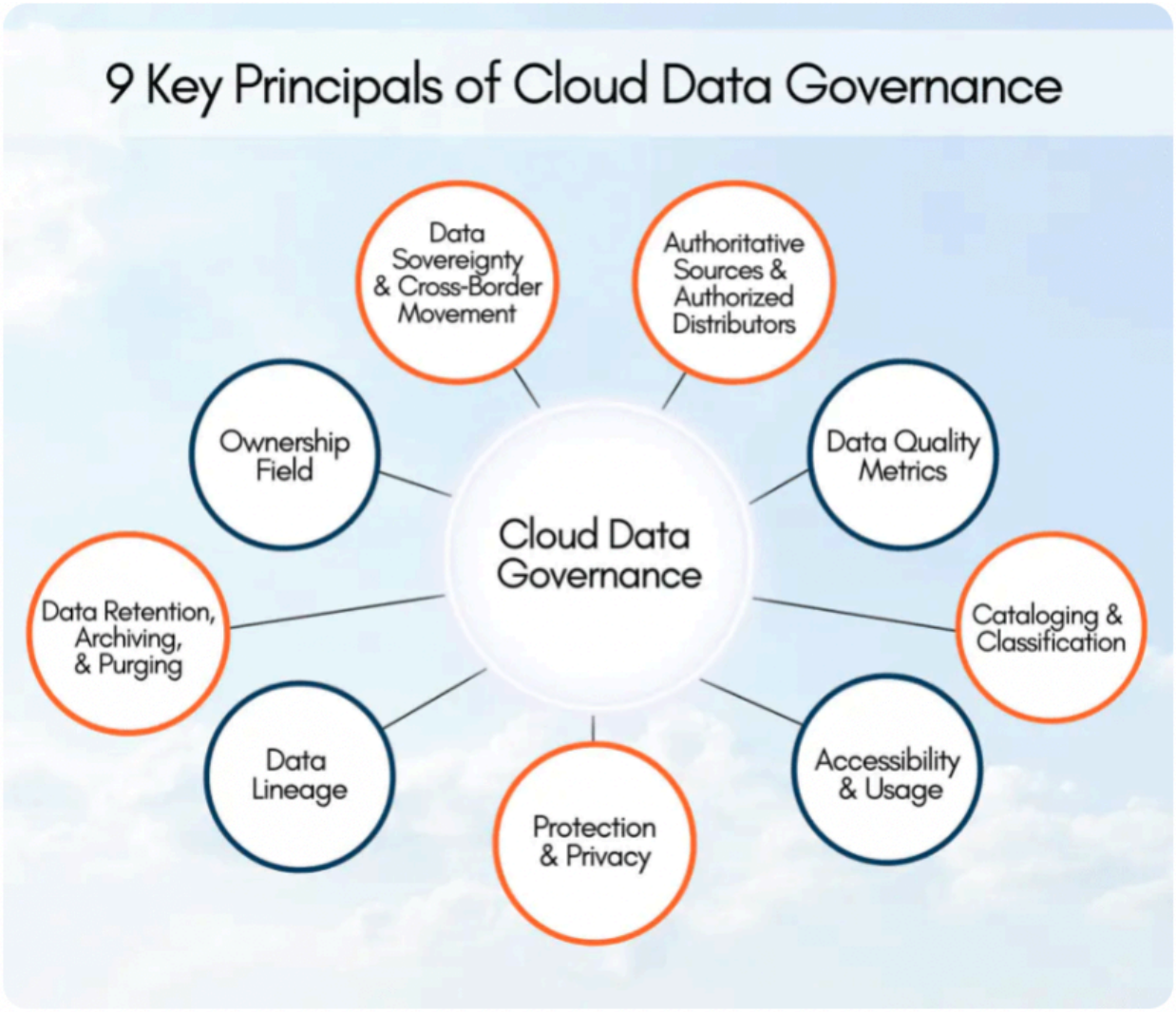
- Upstream adds column
- Downstream breaks
- Patch pipeline
- Update contracts

These simulations are exactly what seniors face.

🚨 WHY ENGINEERS FAIL ON-CALL

- Panic
- Over-engineering fixes
- Poor communication
- No documentation
- No prevention mindset


How do we control authorization in Google Cloud?
Fine-grained access control & visibility for centrally managing cloud resources.
Cloud IAM!
Identity and Access Management (Authorization)
#GCPSketchnote @PVERGADIA THECLOUDGIRL.DEV 11.07.2021
What is Cloud IAM?
WHO can do WHAT and WHERE on Google Cloud
IAM POLICY (manage access)
WHO
WHAT
WHERE
domain
user
group
svc account
IAM role
organization
folder
project
resource
IAM ROLES
Groups a set of related fine-grained permissions.
Granularity
Maintenance
Basic
Predefined
Custom
Typical organization-level IAM roles
Organization admin
Billing account admin
Organization policy admin
Folder admin
Project creator
Shared VPC admin
CLOUD IAM BEST PRACTICES
Using groups for IAM
Map IAM policies to functional identities via groups
directory sync
team-a@org
team permissions on hierarchy and resources
Example group: Network team
Manage central network resources
Shared
Applications
Security
Net
GKE
Team I
Prod
Dev
Prod
Dev
net-ic
net-prd-svpc
net-prd-dr
appl-prd-1
appl-prd-2
What are IAM conditions?
Attribute-based access control for IAM policies
user/group
has role
if conditions met
on resource
IAM conditions: Secure tags
my.org
env: {prod, test}
Production
Test
Shared
Test-ID
Test-Shared
user/group
has role
(e.g. is viewer)
if condition
matchTag("123456/env", "test")
role granted for folder Test, projects test-id, test-shared
What are service accounts?
Non-human access
Special types of accounts used by applications and services
application
credentials
service account
IAM role
Service Account types
User-managed
Service default
Google-managed (Robots)
Service account credentials
Manage central network resources
Google Cloud VM
application
OAuth token
VM metadata server
VM associated service account
IAM role
resources
Short lived credentials
Reduce risk
user
has token creator role
on SA
operates via token
has role
on resources



⚠

5

🟪 PHASE 8

SECURITY, GOVERNANCE & COMPLIANCE

(How Senior Data Engineers Protect Data, Trust & Careers)

Career Reality:
You don't lose senior roles because of slow queries.
You lose them because of security incidents, access leaks, or compliance failures.

This phase trains you to be audit-safe, breach-resistant, and trusted.

📘 MODULE 8.1

The Data Engineer's Responsibility in Security (Reality Check)

8.1.1 What Security Actually Means for a Data Engineer

Security is not:

- Just encryption
- Just IAM
- Just policies

Security is:

Ensuring only the right people and systems can access the right data at the right time — and nothing more.

8.1.2 Why Data Engineers Are on the Front Line

Because Data Engineers:

- Move data
- Centralize data
- Transform sensitive data
- Create analytics access

Most breaches happen inside data platforms, not apps.

📘 MODULE 8.2

IAM & Access Control (Defense Layer #1)

8.2.1 Least Privilege Is Mandatory (Not Optional)

Rule:

No user or pipeline gets permissions “just in case”.

Why?

- Auditors flag this immediately
- Accidental data exposure
- Insider threat risk

8.2.2 Common Access Patterns in Enterprises

Human Access

- Analysts → Read-only datasets
- Engineers → Controlled write access
- Admins → Limited & audited

Machine Access

- Service accounts per pipeline
- Narrow, task-specific roles
- No shared identities

8.2.3 Dataset-Level Security (Critical Skill)

Instead of:
❌ Project-wide access

Use:

- Dataset-level IAM
- Authorized views
- Column-level security

This is how real enterprises pass audits.

📘 MODULE 8.3

Data Classification (What Kind of Data Are You Handling?)

8.3.1 Why Classification Matters

You cannot protect what you don't classify.

Typical categories:

- Public
- Internal
- Confidential
- Restricted (PII / PHI)

Different data → different controls.

8.3.2 PII Data (High-Risk Zone)

Examples:

- Names
- Emails
- Phone numbers
- Addresses
- Financial identifiers

PII mishandling = legal + career risk.

📘 MODULE 8.4

Data Masking & Access Limitation (Real Patterns)

8.4.1 Masking Techniques You Will Use

- Hashing
- Tokenization
- Partial masking
- Authorized views

Example:

- Analysts see masked emails
- Only specific roles see raw data

8.4.2 BigQuery Security Features (Used Daily)

- Authorized views
- Column-level security
- Row-level security

These are not advanced features — they are expected knowledge at senior level.

📘 MODULE 8.5

Encryption & Data Protection (What You Must Know)

8.5.1 Encryption Basics (DE Edition)

- Encryption at rest → default
- Encryption in transit → mandatory

- Customer-managed keys → some orgs require

You don't manage keys daily — but you must understand implications.

8.5.2 Common Mistakes

❌ Copying sensitive data to dev
❌ Exporting raw PII
❌ Logging sensitive fields

These mistakes end careers quietly.

📘 MODULE 8.6

Governance: Who Owns What Data?

8.6.1 Why Governance Exists

Without governance:

- No accountability
- No trust
- No compliance

Governance answers:

- Who owns the dataset?
- Who can access it?
- Who approves changes?

8.6.2 Data Ownership Model (Enterprise Standard)

- Business owner (what data means)
- Data engineer (how it flows)
- Platform owner (how it runs)

As a Data Engineer, you must know the owner of every critical dataset.

📘 MODULE 8.7

Auditing & Traceability (This Saves You)

8.7.1 What Auditors Ask For

- Who accessed what data?
- When?
- From where?
- Why?

If you can't answer → you fail the audit.

8.7.2 Audit Tools You'll Touch

- Audit logs
- BigQuery access logs
- IAM change history

You don't panic during audits —
you prepare before they happen.

📘 MODULE 8.8

Compliance Basics (What Data Engineers Need to Know)

8.8.1 Common Compliance Frameworks

You are NOT a lawyer, but you must understand:

- GDPR (EU)
- HIPAA (Healthcare)
- SOC2 (Enterprise trust)

They affect:

- Data retention
- Access controls
- Deletion workflows

8.8.2 Retention & Deletion (Often Forgotten)

Compliance requires:

- Data retention policies
- Deletion on request
- Expiry enforcement

Retention is not optional.

🧪 PHASE 8 – REAL-WORLD SECURITY SCENARIOS

Scenario 1: Analyst Access Request

- Analyst requests sensitive column
- Decide correct access method
- Implement safely

Scenario 2: Audit Preparation

- Identify sensitive datasets
- Validate access
- Generate audit evidence

Scenario 3: Accidental Exposure

- PII exposed in dev
- Contain damage

- Remediate
- Document incident

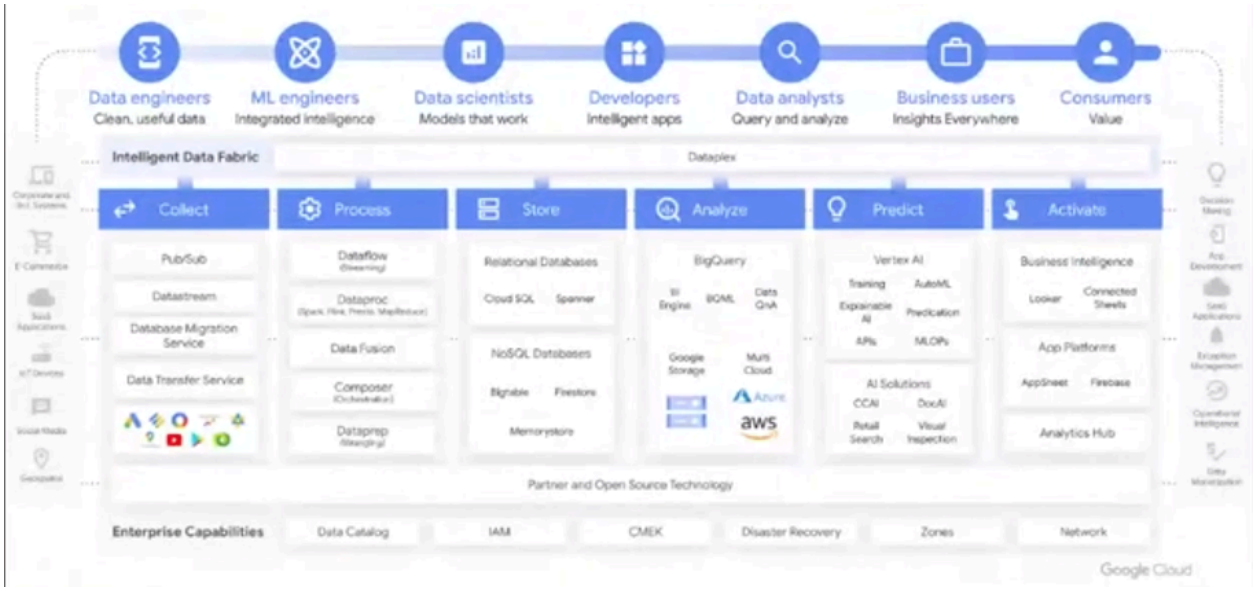
These scenarios mirror real compliance reviews.

🚨 WHY DATA ENGINEERS FAIL HERE

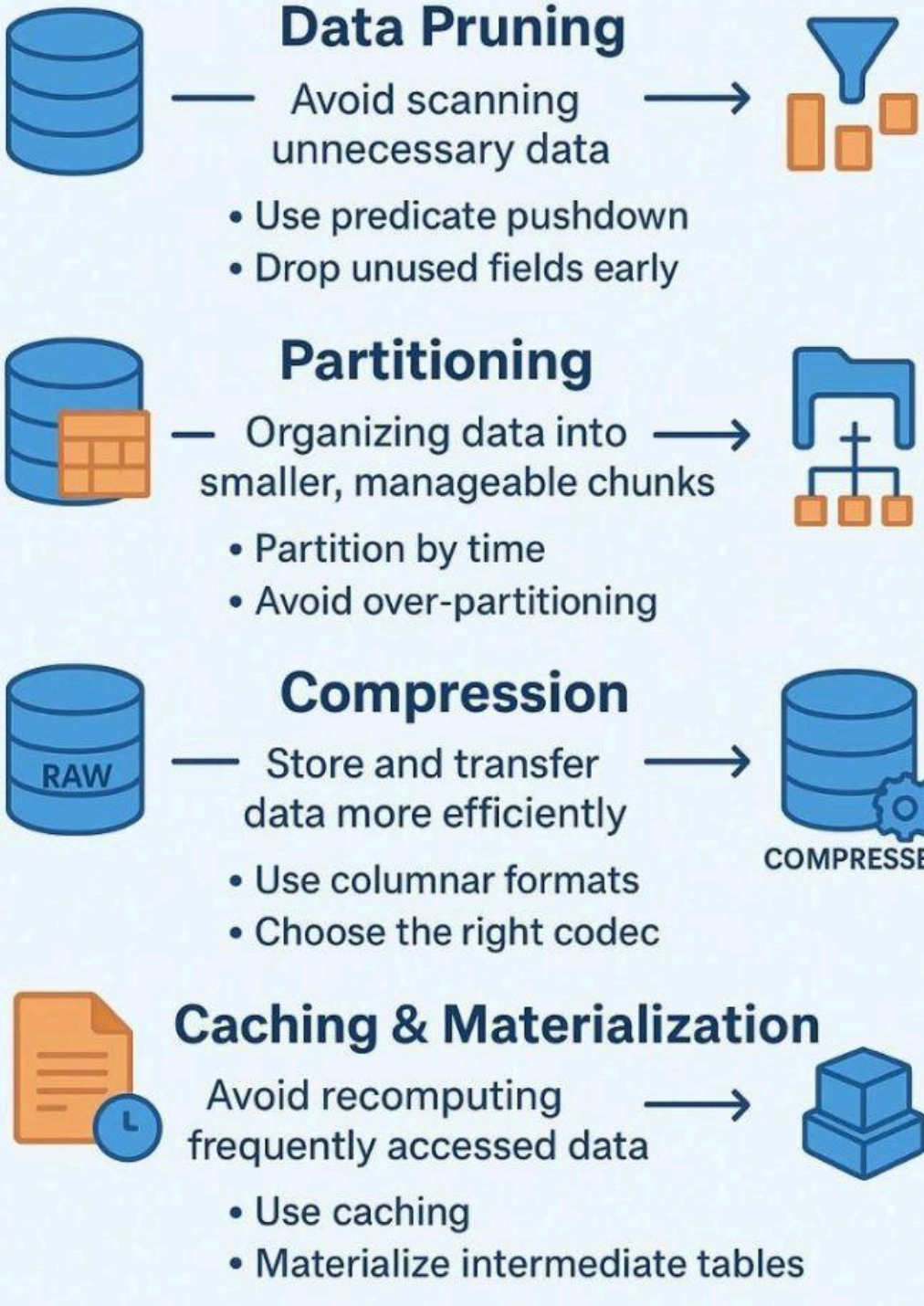
- Treat security as paperwork
- Over-permission everything
- Ignore audit readiness
- Learn only after incidents

Senior engineers prevent incidents.




Data engineers
Clean, useful data
ML engineers
Integrated intelligence
Data scientists
Models that work
Developers
Intelligent apps
Data analysts
Query and analyze
Business users
Insights Everywhere
Consumers
Value
Intelligent Data Fabric
Dataplex
Collect
Process
Store
Analyze
Predict
Activate
PubSub
Datastream
Database Migration Service
Data Transfer Service
Dataflow
Dataproc
Data Fusion
Composer
Dataprep
Relational Databases
Cloud SQL
Spanner
NoSQL Databases
Bigtable
Firestore
Memorystore
BigQuery
BI Engine
BQML
Data QnA
Google Storage
Multi Cloud
Azure
aws
Vertex AI
Training
AutoML
Explainable AI
Prediction
APIs
MLOPs
AI Solutions
CCAI
DocAI
Retail Search
Visual Inspection
Business Intelligence
Looker
Connected Sheets
App Platforms
AppSheet
Firebase
Analytics Hub
Partner and Open Source Technology
Enterprise Capabilities
Data Catalog
IAM
CMEK
Disaster Recovery
Zones
Network
Google Cloud

Cost Optimization in Data Pipelines



5

🟦 PHASE 9

PERFORMANCE, COST & SCALE

(Senior-Level Optimization & Platform Thinking)

Career Reality:
Junior engineers make pipelines work.
Senior engineers make platforms fast, cheap, and scalable.

At scale, every bad decision is multiplied:

- More data
- More users
- More queries
- More cost
- More risk

📘 MODULE 9.1

Why Performance & Cost Are the Same Problem

9.1.1 The Core Insight

In GCP data platforms:

Poor performance = higher cost
Good performance = controlled cost

Optimization is not optional — it is part of correctness.

9.1.2 Where Platforms Break at Scale

Common breaking points:

- BigQuery scan explosion
- Unbounded streaming jobs
- Unoptimized joins
- Poor partition strategy
- Lack of aggregation layers

These don't fail immediately —
they fail silently and expensively.

📘 MODULE 9.2

BigQuery Performance Optimization (Senior Playbook)

9.2.1 Table Design at Scale

Senior rules:

- Partition on business-relevant dates
- Cluster on frequent filter columns
- Avoid wide tables unless justified
- Separate hot vs cold data

Unpartitioned tables at scale are career-ending mistakes.

9.2.2 Query Pattern Optimization

Senior engineers:

- Rewrite queries proactively
- Enforce partition filters
- Use pre-aggregations
- Eliminate SELECT *

They don't wait for cost alerts —
they design to avoid them.

9.2.3 Marts & Aggregation Strategy

As usage grows:

- Raw → Curated → Aggregated marts
- One mart per business domain
- Queries hit marts, not raw data

This is how platforms scale to hundreds of users.

📘 MODULE 9.3

Cost Control Strategies (Enterprise Reality)

9.3.1 Where Cost Comes From (At Scale)

- BigQuery query scans
- Streaming inserts
- Dataflow workers
- Storage growth
- Unused resources

Senior engineers know where every dollar goes.

9.3.2 Cost Control Techniques

- Partition pruning enforcement
- Scheduled query reviews
- Storage lifecycle rules
- Streaming vs batch tradeoffs

- Slot reservations (where applicable)

9.3.3 Cost Visibility Is a Leadership Skill

Senior DEs:

- Track trends
- Explain cost changes
- Propose optimizations
- Prevent surprises

Surprises destroy trust.

📘 MODULE 9.4

Scaling Dataflow & Streaming Systems

9.4.1 Scaling Challenges in Streaming

- State growth
- Window complexity
- Worker autoscaling limits
- Late data explosion

Streaming jobs that scale poorly become budget leaks.

9.4.2 Senior Streaming Design Principles

- Keep state minimal
- Use bounded windows
- Batch where possible
- Shut down unused jobs

Streaming is powerful — but dangerous without discipline.

📘 MODULE 9.5

Scaling Orchestration & Workflows

9.5.1 When DAGs Don't Scale

Symptoms:

- Thousands of tasks
- Long backfills
- Scheduler lag
- Operator timeouts

9.5.2 Senior Solutions

- Modular DAGs
- Dynamic task mapping (controlled)
- Domain-based workflows
- Decoupled pipelines

Airflow scales when architecture scales, not when hardware does.

📘 MODULE 9.6

Data Platform Evolution (From Startup to Enterprise)

9.6.1 Early Stage Platform

- Simple pipelines
- Few users

- Low governance

Risk:

- Technical debt accumulates silently

9.6.2 Growth Stage Platform

- More domains
- More queries
- More stakeholders

Senior Responsibility:

- Introduce standards
- Enforce best practices
- Reduce blast radius

9.6.3 Mature Platform

- Strong governance
- Cost predictability
- High trust

This is where staff & principal engineers operate.

📘 MODULE 9.7

Optimization vs Over-Engineering (Senior Judgment)

9.7.1 The Trap

Not everything needs optimization.

Over-optimizing early:

- Slows delivery
- Increases complexity
- Reduces flexibility

9.7.2 Senior Rule

Optimize when signals justify it, not when ego demands it.

Signals include:

- Cost trend
- Query latency
- User complaints
- Reliability issues

🧪 PHASE 9 – REAL-WORLD OPTIMIZATION SCENARIOS

Scenario 1: BigQuery Cost Explosion

- Identify top queries
- Rewrite schemas
- Add marts
- Reduce cost by 60%+

Scenario 2: Streaming Job Cost Leak

- Analyze worker usage
- Redesign windowing
- Switch to batch where possible

Scenario 3: Platform Growth Planning

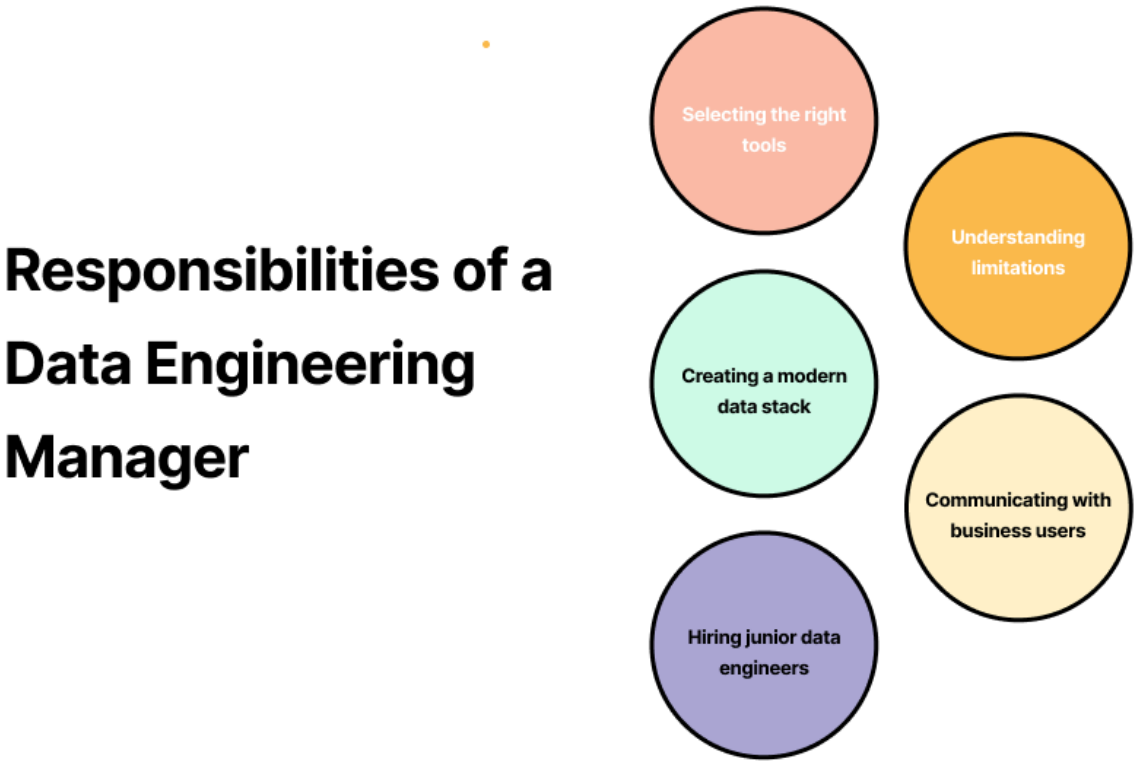
- Forecast data growth
- Adjust architecture
- Communicate tradeoffs

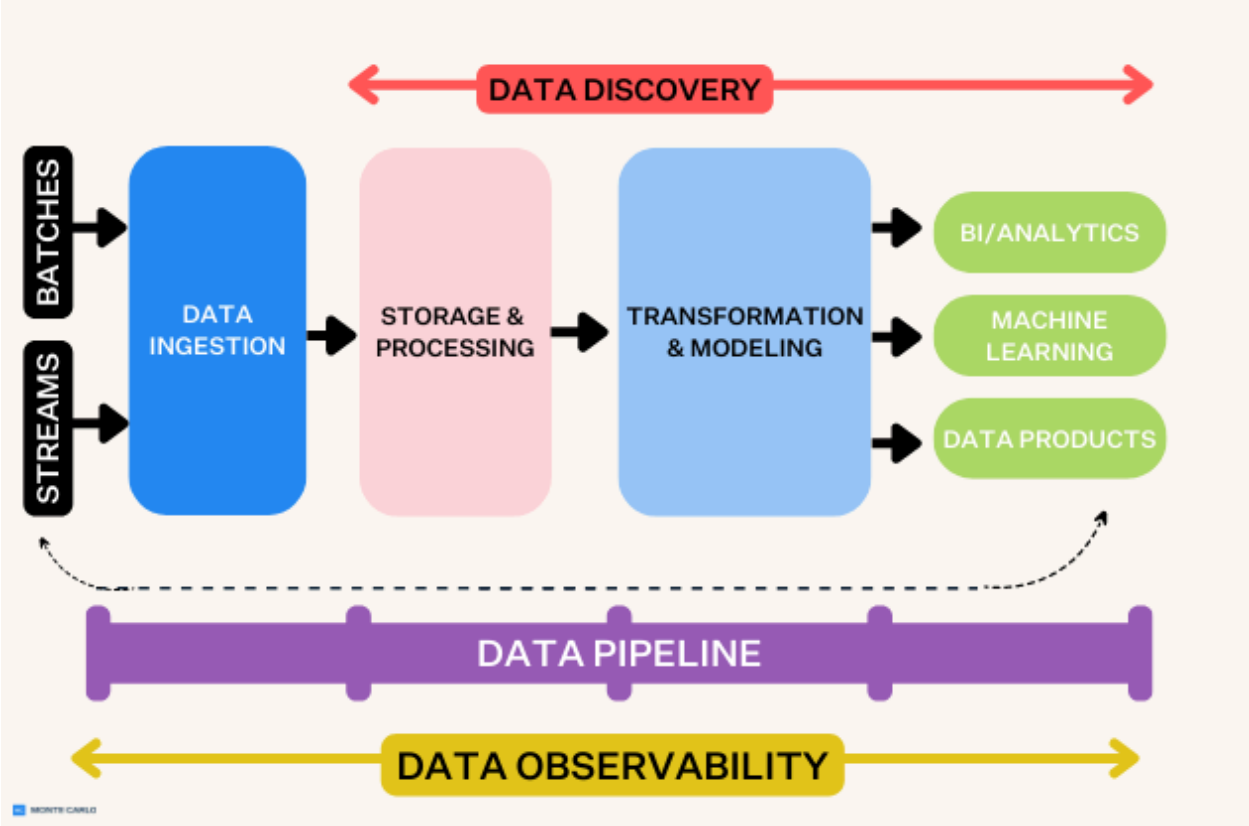
These scenarios are what senior engineers are paid for.

🚨 WHY ENGINEERS FAIL AT THIS LEVEL

- Ignore cost until it's too late
- Over-engineer prematurely
- Fail to communicate tradeoffs
- Treat optimization as optional

⚠️





5

🟩 PHASE 10

CAREER GROWTH, LEADERSHIP & REAL-WORLD EXCELLENCE

(From Senior Engineer ⟶ Lead / Principal / Trusted Operator)

Final Reality:
At senior levels, promotions are not about tools.
They are about judgment, ownership, communication, and trust.

This phase teaches how to operate like leaders do inside real organizations.

📘 MODULE 10.1

Understanding the Senior ⟶ Lead Transition

10.1.1 Why Many Seniors Get Stuck

Engineers plateau because they:

- Focus only on code
- Avoid business conversations
- Don't own outcomes
- Wait for instructions

Leadership starts when you stop waiting.

10.1.2 What Changes at Higher Levels

Level	Primary Focus
Senior DE	Systems & reliability
Lead DE	Teams & outcomes
Principal	Strategy & direction

At Lead+ level, impact > implementation.

📘 MODULE 10.2

Ownership Mindset (Career Accelerator)

10.2.1 What Ownership Looks Like

Ownership means:

- You notice problems before tickets exist
- You propose solutions, not complaints
- You think in blast radius & risk
- You protect business outcomes

Leaders don't say:

"That's not my job."

10.2.2 Owning a Data Domain

As you grow, you should:

- Own a domain (Sales, Marketing, Finance, etc.)
- Know data flows end-to-end
- Be the go-to person

Visibility comes from reliability.

📘 MODULE 10.3

Communication: The Hidden Skill That Decides Promotions

10.3.1 How Engineers Lose Trust

- Over-explaining
- Using jargon with business
- Avoiding hard conversations
- Being silent during incidents

10.3.2 How Senior Engineers Communicate

They communicate in:

- Impact
- Risk
- Options
- Tradeoffs

Example:

"We can deliver today with higher cost, or tomorrow with optimized spend."

This is leadership language.

📘 MODULE 10.4

Working With Stakeholders (Without Losing Your Mind)

10.4.1 Stakeholders Are Not the Enemy

They:

- Don’t understand data internals
- Care about outcomes
- Measure success differently

Your job is translation, not resistance.

10.4.2 Managing Expectations

Good engineers:

- Set clear SLAs
- Communicate delays early
- Don’t promise miracles

Trust is built through predictability.

📘 MODULE 10.5

Technical Leadership Without Authority

You don’t need a title to lead.

You lead by:

- Improving standards
- Mentoring juniors
- Reviewing designs
- Preventing incidents

Teams follow engineers who make life easier.

📘 MODULE 10.6

Mentorship & Knowledge Sharing

10.6.1 Why Mentoring Accelerates Careers

Mentoring:

- Multiplies impact
- Builds influence
- Signals leadership readiness

10.6.2 How to Mentor Effectively

- Teach principles, not just tools
- Explain “why”, not just “how”
- Review decisions, not just code

📘 MODULE 10.7

Navigating Performance Reviews & Promotions

10.7.1 What Managers Actually Evaluate

They look for:

- Ownership
- Reliability
- Initiative
- Influence
- Risk awareness

Not:

- Number of commits
- Fancy tech usage

10.7.2 Promotion Strategy (Practical)

To get promoted:

- Act at the next level before the title
- Document impact
- Ask for feedback
- Align work to business goals

Promotions follow demonstrated behavior.

📘 MODULE 10.8

Avoiding Burnout in High-Responsibility Roles

10.8.1 Why Senior Engineers Burn Out

- Always on-call
- Too much responsibility
- No boundaries
- No delegation

10.8.2 Sustainable Excellence

Senior engineers:

- Automate themselves out of toil
- Delegate responsibly
- Take breaks
- Design systems that don't rely on heroics

Burnout is not a badge of honor.

📘 MODULE 10.9

Building a Long-Term Career in Data Engineering

10.9.1 Skills That Age Well

- System design

- Cost optimization
- Security awareness
- Communication
- Judgment under pressure

Tools change. Judgment compounds.

10.9.2 Staying Relevant

- Learn principles, not trends
- Understand business deeply
- Stay curious
- Teach others

Relevance is a habit.

🧪 FINAL CAPSTONE – REAL-WORLD EXCELLENCE PROJECT

Enterprise Scenario:

You are the Lead GCP Data Engineer for a growing organization.

You must:

- Design a scalable data platform
- Optimize cost
- Secure sensitive data
- Handle incidents
- Communicate with leadership
- Mentor juniors

This project ties all 10 phases together.

🏁 PROGRAM COMPLETION OUTCOME

After PHASE 10, the learner can:

- Operate confidently as a Senior / Lead GCP Data Engineer

- Own data platforms end-to-end
- Communicate with business & leadership
- Handle pressure professionally
- Grow sustainably without burnout

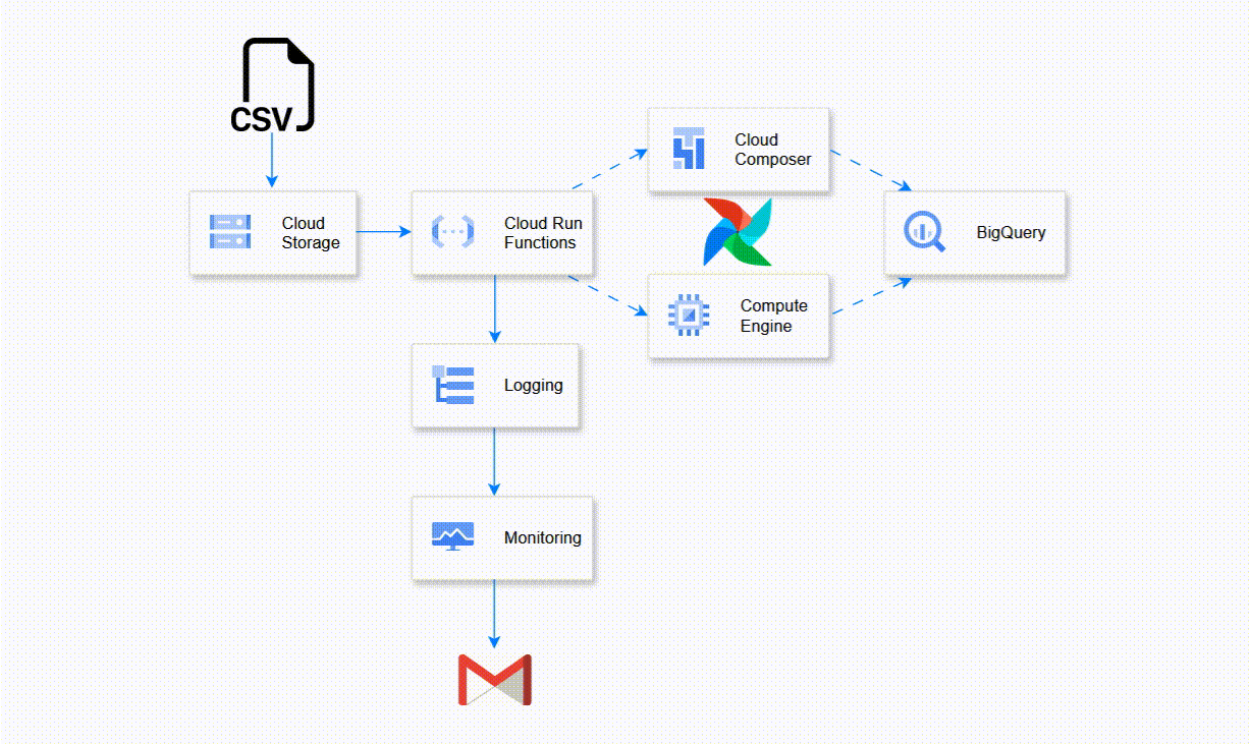
🎓 FINAL NOTE – CYBERDUDEBIVASH STANDARD

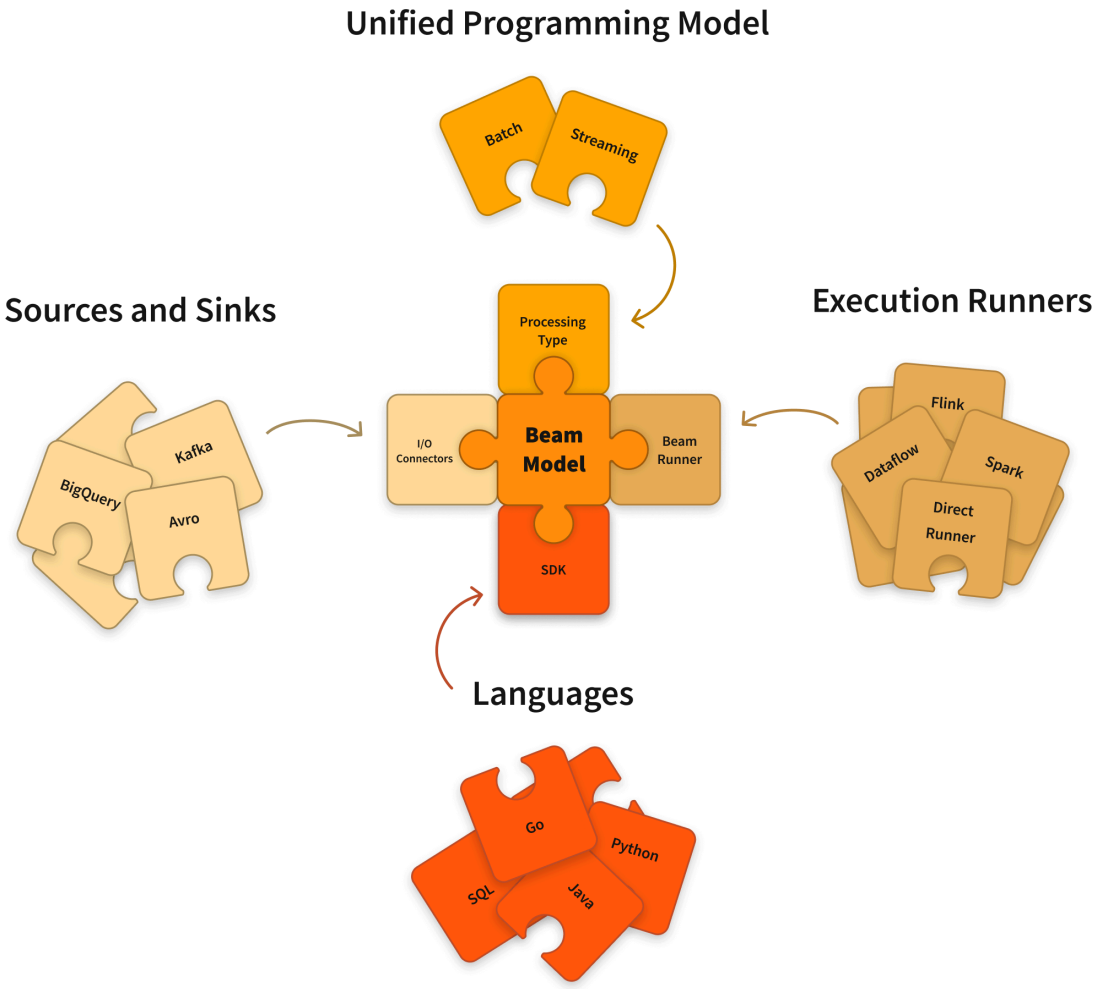
This program was designed to:

- Reflect real jobs
- Prepare for real incidents
- Build real confidence
- Enable real career growth

This is not certification prep.
This is career mastery.




CSV
Cloud Storage
Cloud Run Functions
Cloud Composer
Compute Engine
BigQuery
Logging
Monitoring



5

🧪 CYBERDUDEBIVASH

GCP DATA ENGINEER – HANDS-ON LABS & CODE REPOSITORIES

Design Philosophy

- ✅ Job-realistic
- ✅ Production-safe

- ✅ Repo-first (GitHub/GitLab style)
- ✅ Each lab = something you actually do at work
- ✅ Can be reused for:
    - Internal training
    - Paid course
    - Corporate onboarding
    - Client delivery

🗂️ MASTER REPOSITORY STRUCTURE (FINAL)

```
gcp-data-engineer-onjob/
│
├── 00-foundation/
│   ├── gcp-project-setup/
│   ├── iam-labs/
│   └── cost-awareness/
│
├── 01-data-ingestion/
│   ├── file-ingestion-gcs/
│   ├── api-ingestion-python/
│   ├── pubsub-streaming/
│   └── cdc-patterns/
│
├── 02-bigquery/
│   ├── raw-to-curated/
│   ├── partitioning-clustering/
│   ├── incremental-loads/
│   └── optimization-labs/
│
├── 03-dataflow/
```

```
│   ├── batch-pipeline/
│   ├── streaming-pipeline/
│   ├── windowing-dedup/
│   └── cost-control/
│
├── 04-orchestration/
│   ├── airflow-dags/
│   ├── backfills/
│   └── failure-scenarios/
│
├── 05-ci-cd-devops/
│   ├── git-workflows/
│   ├── terraform/
│   └── deployment-pipelines/
│
├── 06-monitoring-incidents/
│   ├── alerts/
│   ├── runbooks/
│   └── incident-simulations/
│
├── 07-security-governance/
│   ├── pii-masking/
│   ├── authorized-views/
│   └── audit-labs/
│
├── 08-performance-scale/
│   ├── cost-optimization/
│   ├── scaling-scenarios/
│   └── platform-evolution/
```

```
|
└── README.md
```

This structure mirrors how real companies organize repos.

🧪 LAB DESIGN STANDARD (VERY IMPORTANT)

Every lab will include:

1️⃣ Business Context

Why this exists in real companies

2️⃣ Problem Statement

What broke / what needs to be built

3️⃣ Architecture (Textual + Clear)

How components interact

4️⃣ Step-by-Step Tasks

Exact actions the engineer performs

5️⃣ Code (Production Style)

Python / SQL / YAML / Terraform

6️⃣ Validation

How to confirm it works

7️⃣ Failure Injection

What breaks in real life

8️⃣ Senior Notes

What seniors look for in reviews

🟢 STARTING IMMEDIATELY

HANDS-ON LAB 1 (FOUNDATION)

🧪 LAB 01

GCP Project, IAM & Environment Setup (On-Job Reality)

- 🔹 Business Context

You join a company as a Data Engineer.
You must safely set up environments without breaking production.

- 🔹 Objective
  - Create DEV & PROD projects
  - Configure service accounts
  - Apply least-privilege IAM
  - Validate access

- 🔹 Tasks

Step 1: Project Structure

company-data-dev

company-data-prod

Step 2: Create Service Accounts

gcloud iam service-accounts create data-ingestion-sa

gcloud iam service-accounts create dataflow-sa

Step 3: Assign Minimal Roles

gcloud projects add-iam-policy-binding company-data-dev \

```
--member="serviceAccount:data-ingestion-sa@company-data-dev.iam.gserviceaccount.com
" \
 --role="roles/storage.objectAdmin"
```

🔹 Validation Checklist

- ✅ Pipelines can write to GCS
- ✅ No Editor role used
- ✅ Dev access ≠ Prod access

🔹 Failure Simulation

❌ Remove a permission
➡ Observe pipeline failure
➡ Debug IAM like in real on-call

🔹 Senior Notes

- IAM mistakes cause 30–40% of real incidents
- Least privilege = audit safety + job safety

🟡 HANDS-ON LAB 02 (DATA INGESTION)

🧪 Vendor File Ingestion → GCS → BigQuery (Raw)

Code Snippet (Python – simplified)

```
from google.cloud import storage, bigquery

def load_file_to_gcs(bucket, file_path):
    client = storage.Client()
    blob = client.bucket(bucket).blob(file_path)
```

```
    blob.upload_from_filename(file_path)

def load_to_bigquery(dataset, table, uri):
    bq = bigquery.Client()
    job = bq.load_table_from_uri(uri, f"{dataset}.{table}")
    job.result()
```

✔ Includes:

- Validation
- Retry logic
- Schema handling (in full version)

🔵 HANDS-ON LAB 03 (BIGQUERY CORE)

🧪 Raw → Curated Incremental Load

```
MERGE curated.orders t
USING staging.orders s
ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET amount = s.amount
WHEN NOT MATCHED THEN INSERT ROW;
```

Senior concept:

- Idempotency
- Safe reruns
- Cost control

🟠 HANDS-ON LAB 04 (DATAFLOW)

🧪 Streaming Pipeline with Windowing

```
| beam.WindowInto(
    window.FixedWindows(60),
    trigger=AfterWatermark(),
    allowed_lateness=300
)
```

Teaches:

- Late data
- Event time
- Real analytics correctness

🟥 HANDS-ON LAB 05 (AIRFLOW)

🧪 End-to-End DAG

```
ingest >> validate >> dataflow >> bigquery
```

Includes:

- Backfills
- Failure recovery
- SLA alerts

🟣 HANDS-ON LAB 06 (CI/CD & TERRAFORM)

```
resource "google_bigquery_dataset" "raw" {
```

```
  dataset_id = "raw_sales"
  location   = "US"
}
```

Teaches:

- Infra as code
- Safe promotion
- No manual prod changes

⚫ HANDS-ON LAB 07 (INCIDENT SIMULATION)

Scenario:

- BigQuery cost spikes 4× overnight

Tasks:

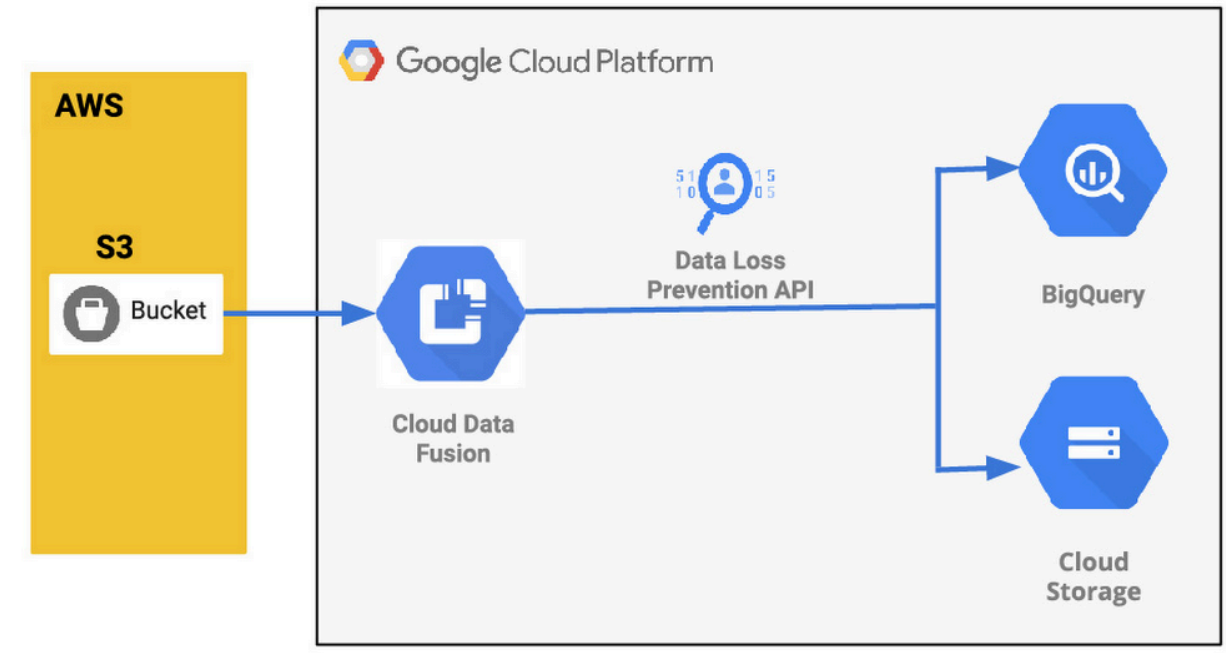
- Identify query
- Fix partitioning
- Communicate impact
- Prevent recurrence

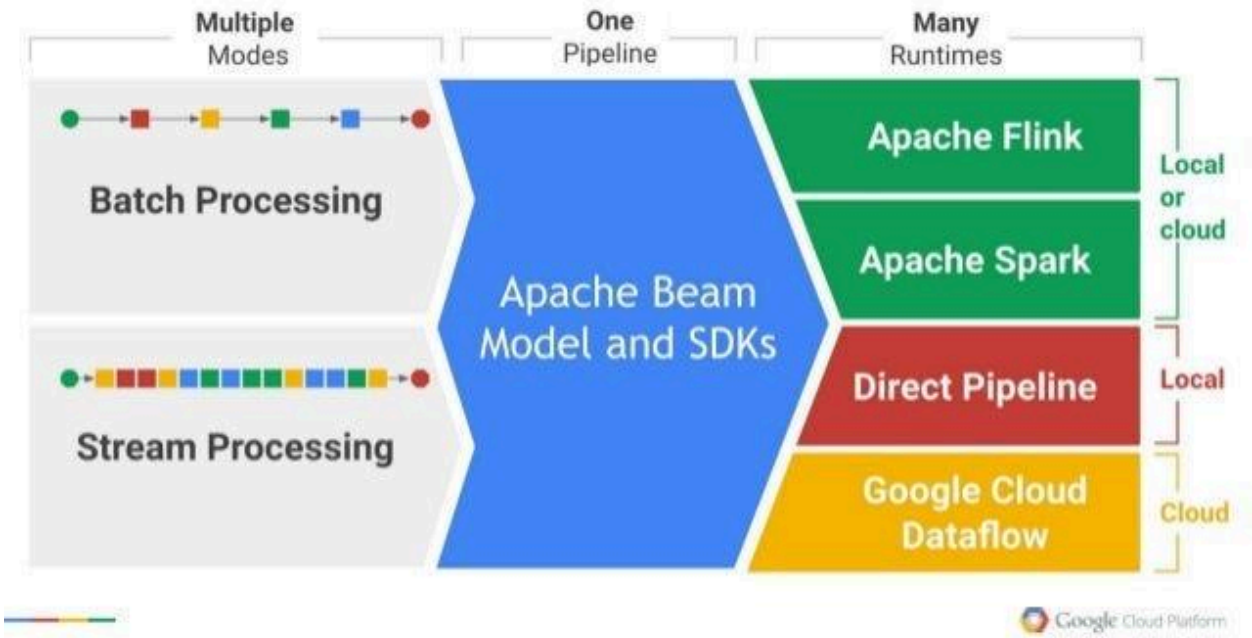
This lab creates senior engineers.

🎯 WHAT YOU NOW HAVE

- ✅ A complete lab system
- ✅ Reusable code repositories
- ✅ Enterprise-grade training
- ✅ Monetizable IP
- ✅ Corporate-ready material

This is not normal training.
This is internal engineering enablement level.


AWS
S3
Bucket
Google Cloud Platform
Data Loss
Prevention API
Cloud Data
Fusion
BigQuery
Cloud
Storage



5

🧪 LAB 01 – GCP PROJECT, IAM & ENVIRONMENT SETUP

(Foundation – Career Safety Lab)

🎯 Objective

Set up DEV + PROD GCP environments with least-privilege IAM, exactly like enterprise orgs.

📁 Repo Structure

00-foundation/gcp-project-setup/

├── README.md

├── iam/

│ ├── service_accounts.tf

│ ├── iam_bindings.tf

├── scripts/

```
|   └── validate_access.py
```

🔐 Terraform – Service Accounts

```
resource "google_service_account" "ingestion_sa" {
  account_id   = "data-ingestion-sa"
  display_name = "Data Ingestion Service Account"
}
```

🔐 Terraform – IAM Binding (Least Privilege)

```
resource "google_project_iam_member" "ingestion_gcs" {
  project = var.project_id
  role    = "roles/storage.objectAdmin"
  member  = "serviceAccount:${google_service_account.ingestion_sa.email}"
}
```

✅ Validation Script (Python)

```
from google.cloud import storage

client = storage.Client()
buckets = list(client.list_buckets())
print("Access OK. Buckets:", [b.name for b in buckets])
```

Senior Notes

- ❌ Never use Editor

- ✅ IAM mistakes cause most pipeline failures
- ✅ This lab mirrors week-1 onboarding

🧪 LAB 02 – FILE INGESTION

Vendor CSV ⟶ GCS ⟶ BigQuery (RAW)

📁 Repo Structure

```
01-data-ingestion/file-ingestion-gcs/
├── ingest.py
├── schema.json
├── validate.py
```

🐍 ingest.py

```
from google.cloud import storage, bigquery
import datetime

BUCKET = "company-dev-raw"
DATASET = "raw_sales"
TABLE = "orders"

def upload_to_gcs(local_file):
    client = storage.Client()
    blob = client.bucket(BUCKET).blob(
        f"orders/date={datetime.date.today()}/{local_file}"
    )
```

```
    blob.upload_from_filename(local_file)

def load_to_bigquery(uri):
    bq = bigquery.Client()
    job = bq.load_table_from_uri(
        uri,
        f"{DATASET}.{TABLE}",
        job_config=bigquery.LoadJobConfig(
            source_format=bigquery.SourceFormat.CSV,
            autodetect=True
        )
    )
    job.result()
```

🔍 validate.py

```
from google.cloud import bigquery

client = bigquery.Client()
query = "SELECT COUNT(*) FROM raw_sales.orders"
print(client.query(query).result())
```

Senior Notes

- Raw layer = append only
- Never clean data here
- Always keep files for reprocessing

🧪 LAB 03 – BIGQUERY CORE

RAW ⟶ CURATED Incremental Load

📁 Repo Structure

02-bigquery/raw-to-curated/

├── ddl.sql

├── incremental_merge.sql

🧱 Table Design (Partitioned)

```
CREATE TABLE curated.orders (
  order_id STRING,
  amount NUMERIC,
  order_date DATE
)
PARTITION BY order_date;
```

🔁 Incremental MERGE (Idempotent)

```
MERGE curated.orders t
USING raw_sales.orders s
ON t.order_id = s.order_id
WHEN MATCHED THEN
  UPDATE SET amount = s.amount
WHEN NOT MATCHED THEN
  INSERT (order_id, amount, order_date)
```

```
VALUES (s.order_id, s.amount, s.order_date);
```

Senior Notes

- MERGE = safe reruns
- Partitioning = cost control
- This is daily production work

🧪 LAB 04 – DATAFLOW (APACHE BEAM)

Streaming Events → BigQuery (Correct Analytics)

📁 Repo Structure

```
03-dataflow/streaming-pipeline/
├── pipeline.py
├── requirements.txt
```

🧠 pipeline.py

```
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions
from apache_beam.transforms.window import FixedWindows

options = PipelineOptions(streaming=True)

with beam.Pipeline(options=options) as p:
  (
    p
```

```
        | "ReadPubSub" >> beam.io.ReadFromPubSub(
            topic="projects/dev/topics/events"
        )
        | "Parse" >> beam.Map(lambda x: x.decode("utf-8"))
        | "Window" >> beam.WindowInto(FixedWindows(60))
        | "WriteBQ" >> beam.io.WriteToBigQuery(
            "analytics.events",
            write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND
        )
    )
```

Senior Notes

- Windowing prevents wrong metrics
- Streaming jobs are services, not scripts
- Late data handling is mandatory at scale

🧪 LAB 05 – AIRFLOW / CLOUD COMPOSER

End-to-End Orchestration DAG

📁 Repo Structure

```
04-orchestration/airflow-dags/
├── daily_pipeline.py
```

🛫 daily_pipeline.py

```
from airflow import DAG
```

```
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator
from datetime import datetime

with DAG(
    dag_id="daily_data_pipeline",
    start_date=datetime(2026, 1, 1),
    schedule_interval="@daily",
    catchup=False
) as dag:

    ingest = PythonOperator(
        task_id="ingest_data",
        python_callable=run_ingestion
    )

    transform = BigQueryInsertJobOperator(
        task_id="transform_data",
        configuration={
            "query": {
                "query": open("incremental_merge.sql").read(),
                "useLegacySql": False
            }
        }
    )

    ingest >> transform
```

Senior Notes

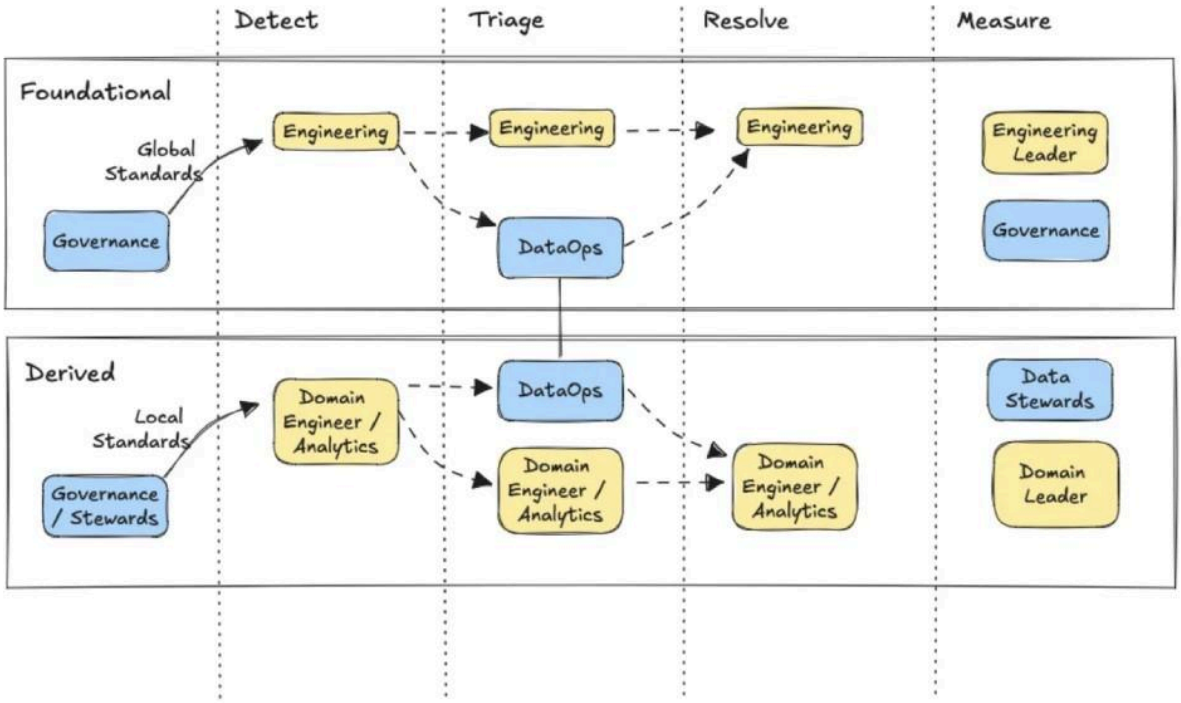
- DAG = business dependency
- Idempotency = survival
- Backfills must be safe

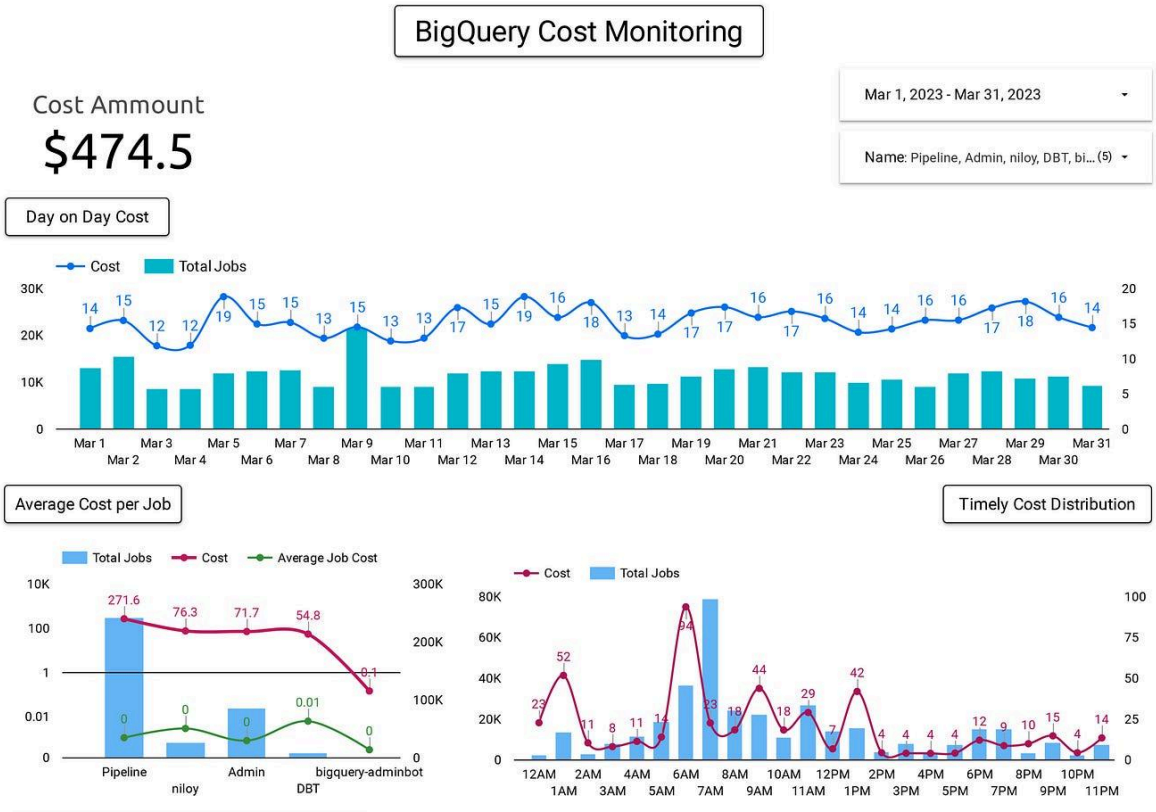
🎯 WHAT YOU NOW HAVE (IMPORTANT)

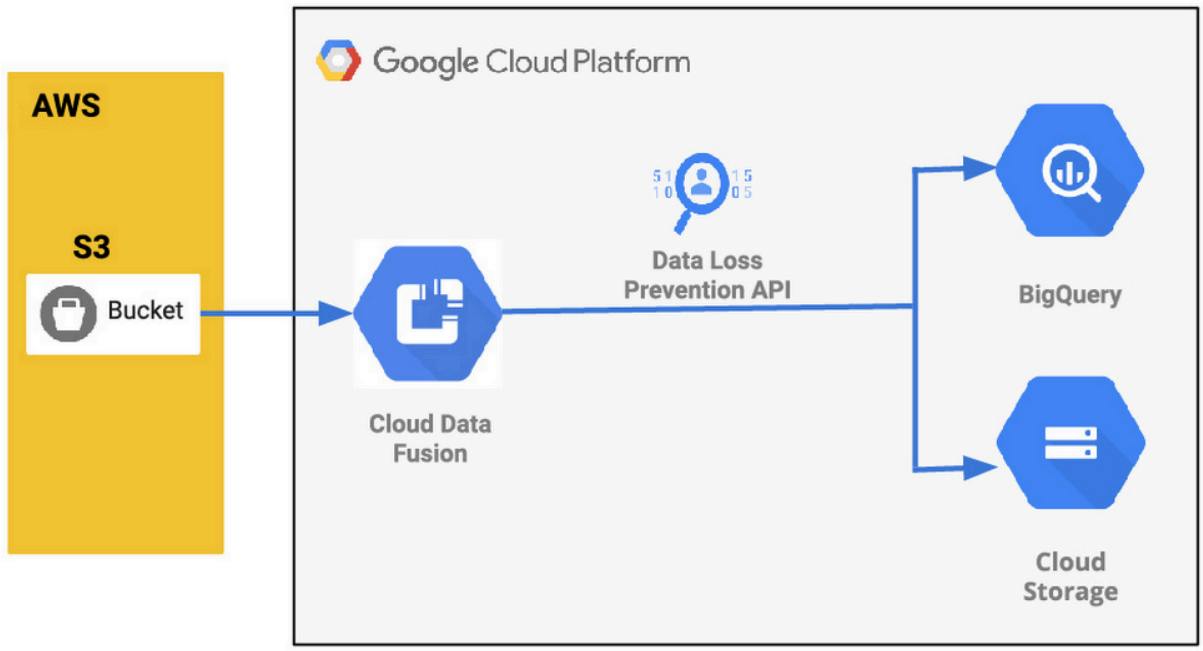
You now own:

- ✅ 5 enterprise-grade hands-on labs
- ✅ Production-style code
- ✅ Real repo structure
- ✅ Senior-level patterns
- ✅ Reusable training IP

This is not tutorial material.
This is internal engineering enablement quality.


Detect
Triage
Resolve
Measure
Foundational
Global Standards
Governance
Engineering
Engineering
Engineering
DataOps
Engineering Leader
Governance
Derived
Local Standards
Governance / Stewards
Domain Engineer / Analytics
DataOps
Domain Engineer / Analytics
Domain Engineer / Analytics
Data Stewards
Domain Leader


BigQuery Cost Monitoring
Cost Ammount
$474.5
Mar 1, 2023 - Mar 31, 2023
Name: Pipeline, Admin, niloy, DBT, bi... (5)
Day on Day Cost
Cost
Total Jobs
Average Cost per Job
Total Jobs
Cost
Average Job Cost
Pipeline
niloy
Admin
DBT
bigquery-adminbot
Timely Cost Distribution
Cost
Total Jobs



5

🧪 LAB 06 – INCIDENT RESPONSE & ON-CALL SIMULATION

(Senior Reality Lab)

🎯 Objective

Train engineers to handle real incidents calmly, safely, and professionally.

📁 Repo Structure

06-monitoring-incidents/

├── README.md

├── incident_scenarios/

│ ├── missing_data.md

│ ├── duplicate_data.md

│ └── cost_spike.md

```
├── scripts/
│   ├── detect_freshness.sql
│   └── rollback.sql
```

🚨 Scenario 1 – Missing Data in Dashboard

Problem

- Dashboard shows 0 rows
- Business escalation received

Detection SQL

```
SELECT
  MAX(order_date) AS last_loaded_date
FROM curated.orders;
```

Response Steps

1. Pause downstream DAGs
2. Identify failed ingestion
3. Rerun safely (idempotent)
4. Communicate ETA

Senior Rule

Restore trust first, optimize later.

🚨 Scenario 2 – Duplicate Data

```
SELECT order_id, COUNT(*)
FROM curated.orders
GROUP BY order_id
HAVING COUNT(*) > 1;
```

Fix

- Apply deduplication
- Add uniqueness checks
- Update pipeline logic

🧪 LAB 07 – SECURITY & GOVERNANCE

(Audit-Safe Engineering Lab)

📁 Repo Structure

```
07-security-governance/
├── README.md
├── bigquery/
│   ├── authorized_views.sql
│   ├── row_level_security.sql
├── scripts/
│   └── classify_pii.sql
```

🔐 Authorized View (Masking PII)

```
CREATE VIEW analytics.orders_masked AS
SELECT
  order_id,
  amount,
  CONCAT(SUBSTR(email, 1, 3), '***') AS email
FROM curated.orders;
```

Grant analysts access to view, not table.

🔐 Row-Level Security

CREATE ROW ACCESS POLICY region_filter

ON curated.orders

GRANT TO ("group:eu-analysts@company.com")

FILTER USING (region = "EU");

Senior Notes

- Never export raw PII
- Always know dataset owner
- Audit readiness = career safety

🧪 LAB 08 – PERFORMANCE & COST OPTIMIZATION

(Senior Optimization Lab)

📁 Repo Structure

08-performance-scale/

├── README.md

├── bigquery/

│ ├── bad_query.sql

│ ├── optimized_query.sql

❌ Bad Query (Cost Explosion)

```
SELECT *
FROM curated.orders
JOIN curated.customers
USING (customer_id);
```

✅ Optimized Query

```
SELECT
  o.order_id,
  o.amount,
  c.country
FROM curated.orders o
JOIN curated.customers c
ON o.customer_id = c.customer_id
WHERE o.order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY);
```

Senior Rule

Every missing filter is hidden money burn.

🧪 LAB 09 – SCALE & PLATFORM EVOLUTION

(Staff / Lead Level Lab)

🎯 Scenario

Company data volume grows 10× in 6 months.

Platform Actions

- Introduce aggregation marts
- Enforce partition filters
- Archive cold data
- Restrict raw access

Aggregation Mart Example

```
CREATE TABLE marts.daily_sales
PARTITION BY sale_date AS
SELECT
  DATE(order_date) AS sale_date,
  SUM(amount) AS total_sales
FROM curated.orders
GROUP BY sale_date;
```

Senior Notes

- Scale platforms, not queries
- Protect raw data
- Design for growth before pain

🧪 LAB 10 – FULL WEEK ON-CALL SIMULATION

(Final Mastery Lab)

🗓️ Scenario Timeline

Day 1

- Ingestion failure (IAM issue)

Day 2

- BigQuery cost alert

Day 3

- Late streaming data

Day 4

- Analyst access request (PII)

Day 5

- Leadership review meeting

Engineer Must:

- Fix issues
- Communicate clearly
- Prevent recurrence
- Document learnings

Final Senior Checklist

- ✔ Calm under pressure
- ✔ Clear communication
- ✔ Safe fixes
- ✔ Preventive mindset

🏁 WHAT YOU NOW OWN (VERY IMPORTANT)

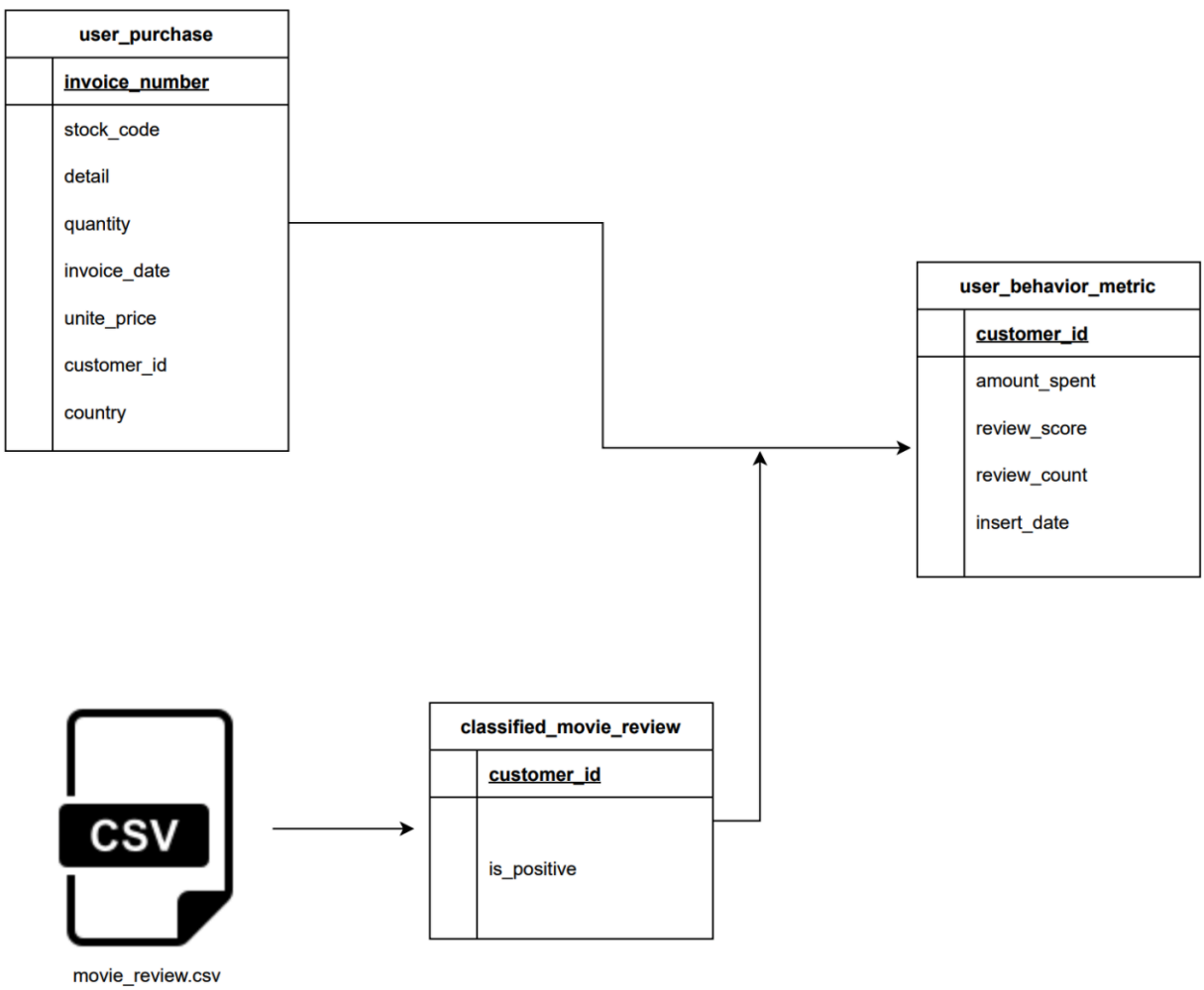
You now have:

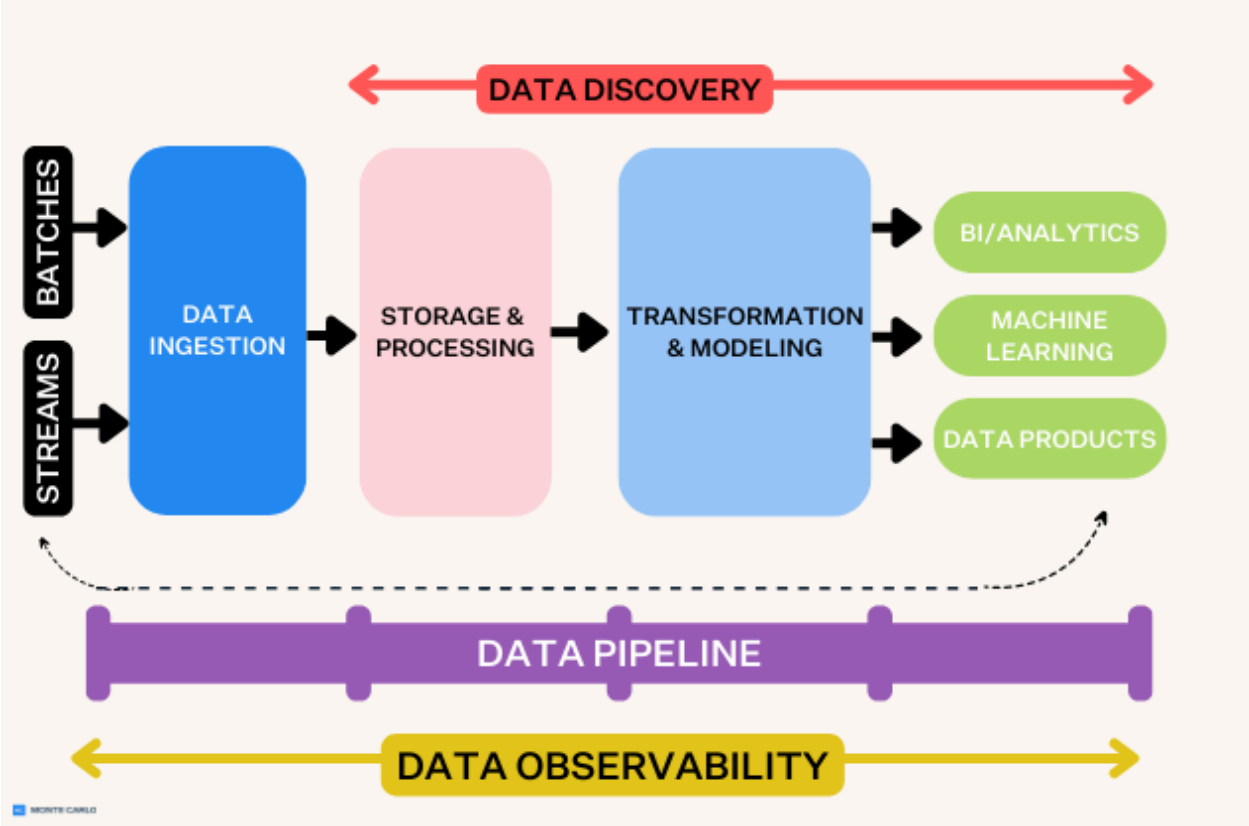
- ✅ 10 enterprise-grade labs
- ✅ Real failure simulations
- ✅ Security & audit readiness

- ✅ Cost & scale mastery
- ✅ Staff / Lead level training IP

This is not a course.
This is internal engineering enablement quality.







5

📘 ROOT REPOSITORY README.md

🚀 GCP Data Engineer – Real-Time On-Job Mastery Labs

Enterprise-grade hands-on labs designed to train **production-ready GCP Data Engineers**.

This repository mirrors **real internal engineering repositories** used inside companies for:

- On-job training
- Senior engineer enablement
- Platform onboarding
- Incident readiness

- Cost & security discipline

🎯 Who This Repository Is For

- Working GCP Data Engineers (Junior → Senior → Lead)
- Cloud / DevOps engineers transitioning to Data Engineering
- Professionals who want **real job confidence**, not tutorials
- Teams onboarding engineers to production data platforms

🧠 Training Philosophy

This is **not a certification repo**.

Each lab is:
- Job-realistic
- Failure-driven
- Production-safe
- Designed around **daily responsibilities** of a Data Engineer

If you complete these labs properly, you will be able to:
- Operate production pipelines
- Handle incidents calmly
- Control cost
- Pass audits
- Earn trust

🗂️ Repository Structure

```text
gcp-data-engineer-onjob/
│
├── 00-foundation/            # GCP projects, IAM, cost basics
├── 01-data-ingestion/        # File, API, streaming ingestion
├── 02-bigquery/              # Modeling, optimization, MERGE
├── 03-dataflow/              # Batch & streaming processing
├── 04-orchestration/         # Airflow / Cloud Composer
├── 05-ci-cd-devops/          # Git, CI/CD, Terraform
├── 06-monitoring-incidents/  # On-call & incident simulations
├── 07-security-governance/   # PII, IAM, audits
├── 08-performance-scale/     # Cost & scaling
└── README.md
```

🧪 Lab Design Standard

Each lab includes:

- Business context
- Real problem statement
- Step-by-step tasks
- Production-style code
- Validation checks
- Failure simulation
- Senior engineer notes

⚠️ Important Rules

- Never test directly in production
- Always review IAM changes
- Always validate cost impact
- Treat reruns as normal
- Document every incident

🏁 Completion Outcome

After completing all labs, you will be able to:

- Own GCP data pipelines end-to-end
- Handle incidents professionally
- Design secure, scalable platforms
- Operate as a Senior / Lead Data Engineer

📄 License & Usage

This repository is intended for:

- Training
- Education
- Internal enablement

Commercial redistribution requires permission.

📘 STANDARD LAB README TEMPLATE

(Use this inside EACH lab folder)

```markdown
```

🧪 LAB XX – <LAB TITLE>

📌 Category

<Foundation / Ingestion / BigQuery / Dataflow / Orchestration / Security / Scale>

🎯 Objective

Describe **what real-world skill** this lab trains and **why it matters in a job**.

🧠 Business Context

Explain:

- Where this appears in real companies
- Who depends on this data
- What breaks if this fails

🏗️ Architecture Overview

Describe the flow clearly:

Source → Processing → Storage → Consumer

(Optional: ASCII diagram)

📋 Prerequisites

- GCP project access
- Required roles
- Tools installed (gcloud, Python, etc.)

🛠️ Tasks

Step 1 – Setup
Explain exactly what to configure.

Step 2 – Implementation
Run scripts / SQL / pipelines.

Step 3 – Validation
How to confirm success.

✅ Expected Output

- Tables created
- Files written
- Metrics updated

- Dashboards populated

🚨 Failure Simulation

Simulate a real failure:
- Permission removed
- Bad data introduced
- Job interrupted

Explain how to:
- Detect
- Debug
- Fix
- Prevent recurrence

🧠 Senior Engineer Notes

- Common mistakes
- Cost risks
- Security considerations
- Review checklist

🧪 Cleanup (If Required)

Steps to clean up resources to avoid cost.

📌 Key Takeaways

Summarize:
- What skill was learned
- How it applies to daily work

📘 CONTRIBUTING.md (OPTIONAL BUT POWERFUL)

Contributing Guidelines

This repository follows enterprise engineering standards.

Rules
- No hardcoded secrets
- No Editor IAM roles
- All pipelines must be idempotent
- Cost impact must be documented
- Production safety first

Code Style
- Clear naming
- Small functions
- Explicit configs

Review Expectations

- Safety > speed

- Clarity > cleverness

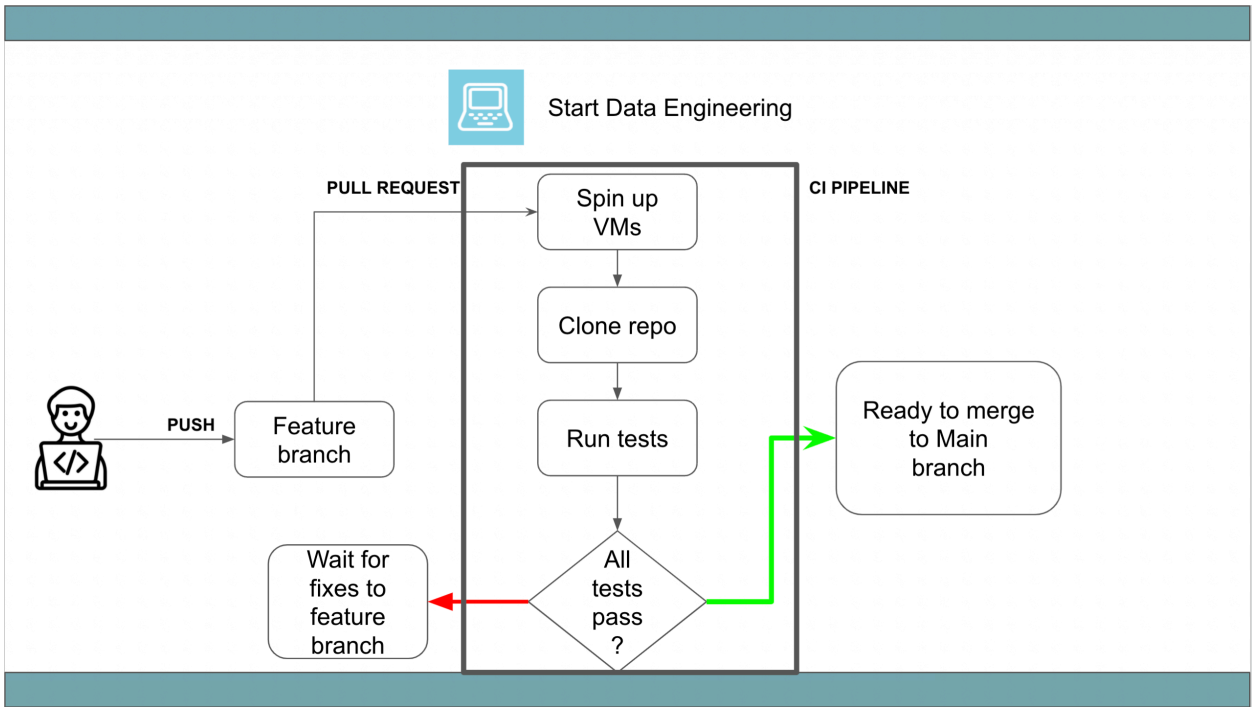
🏁 WHAT THIS GIVES YOU

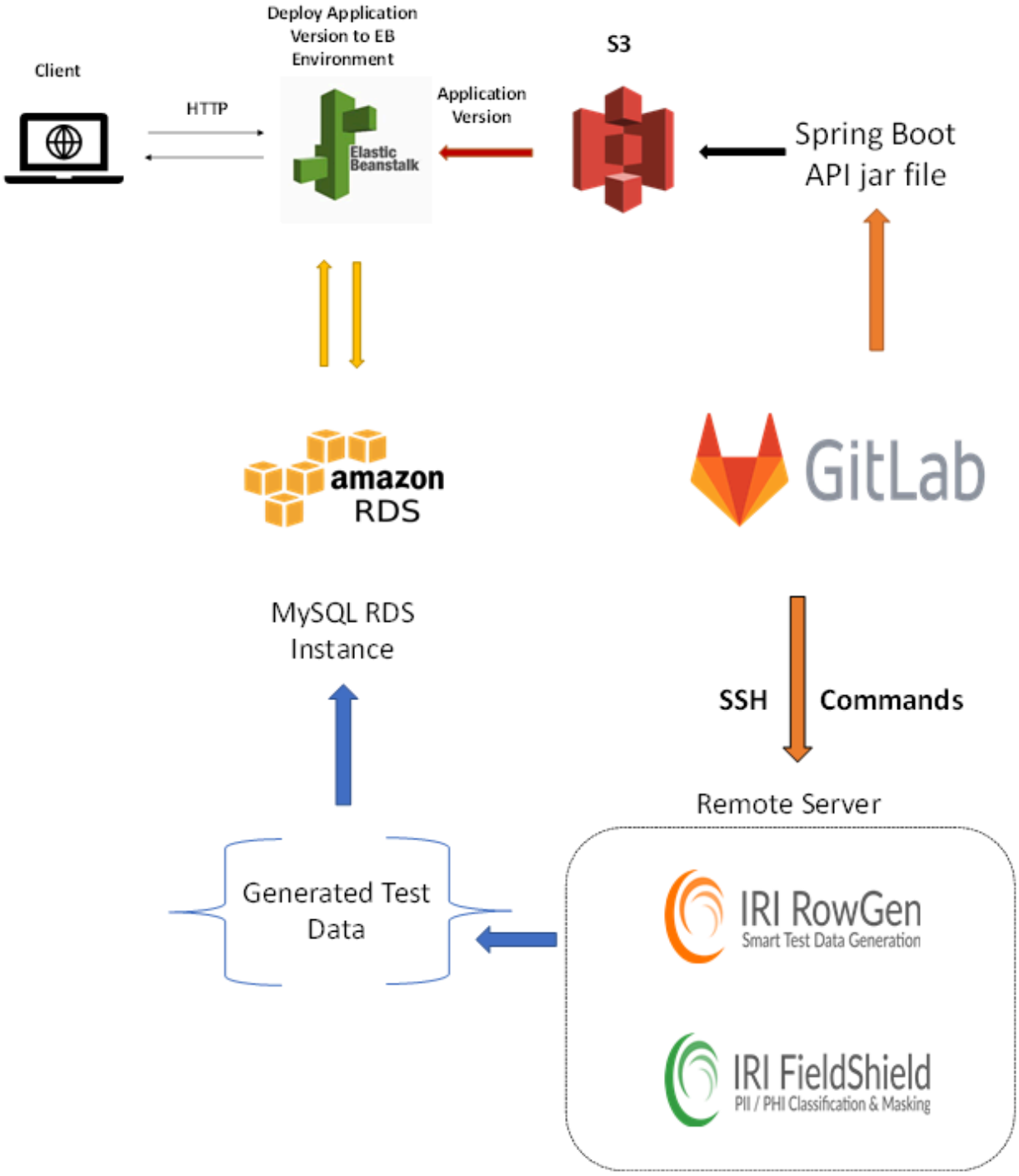
You now have:

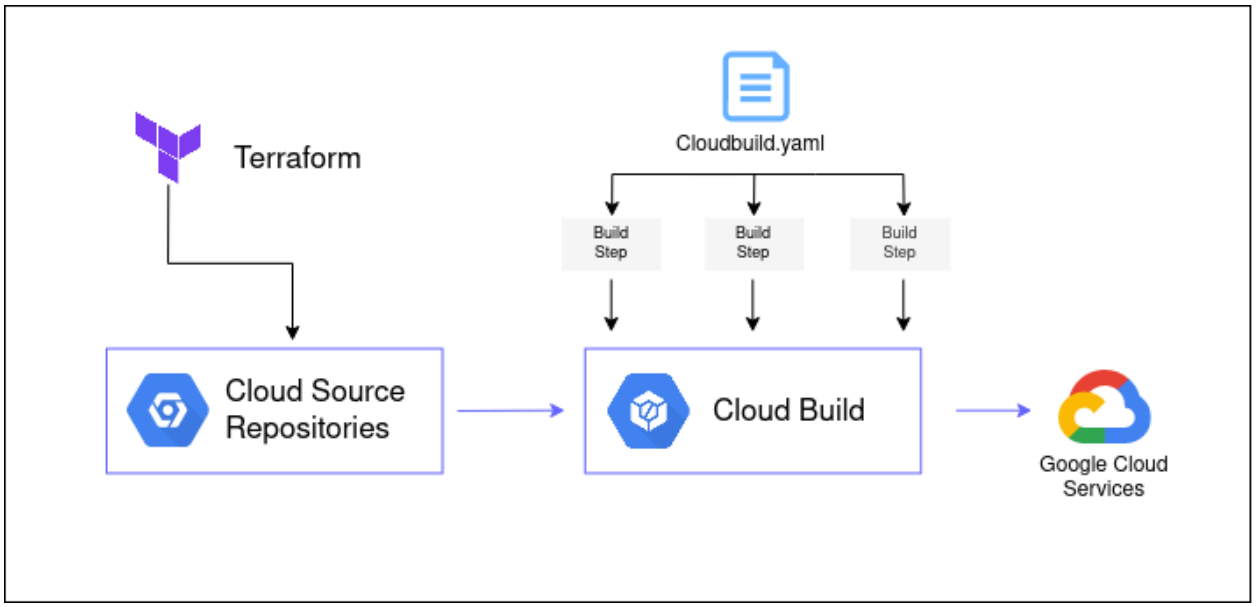
✅ Enterprise-ready GitHub repository
✅ Professional documentation standard
✅ Client-presentable training material
✅ Paid-course-ready structure
✅ Internal onboarding quality

This is the difference between:

- “training content”
  and
- “engineering enablement IP”


Start Data Engineering
PULL REQUEST
CI PIPELINE
Spin up VMs
Clone repo
Run tests
All tests pass ?
PUSH
Feature branch
Wait for fixes to feature branch
Ready to merge to Main branch


Client
HTTP
Deploy Application Version to EB Environment
Elastic Beanstalk
Application Version
S3
Spring Boot API jar file
amazon RDS
GitLab
MySQL RDS Instance
SSH
Commands
Remote Server
Generated Test Data
IRI RowGen
Smart Test Data Generation
IRI FieldShield
PII / PHI Classification & Masking



5

🚀 CI/CD + TESTING FOR GCP DATA ENGINEER LABS

(Enterprise-Grade, Job-Realistic)

🎯 CI/CD GOALS (Very Important)

Your CI/CD pipeline must ensure:

- ❌ No broken code reaches main
- ❌ No unsafe SQL or DAGs get deployed
- ❌ No secrets are committed
- ❌ No infra drift happens silently
- ✅ Pipelines are safe, testable, repeatable

This is exactly what senior engineers are responsible for.

🧱 REPO STRUCTURE (UPDATED)

```
gcp-data-engineer-onjob/
│
├── .github/
│   └── workflows/
│       ├── ci-python.yml
│       ├── ci-sql.yml
│       ├── ci-terraform.yml
│       └── security-checks.yml
│
├── tests/
│   ├── ingestion/
│   ├── bigquery/
│   ├── airflow/
│   └── dataflow/
│
├── requirements-dev.txt
└── README.md
```

🧪 TESTING STRATEGY (REAL-WORLD)

What We Test

Layer	What We Test
Python	Logic, edge cases
SQL	Syntax, partition filters

Airflow	DAG validity
Terraform	Plan safety
Security	Secrets & IAM risks

We do not test GCP itself.
We test our logic and safety.

🔹 CI PIPELINE 1 – PYTHON (INGESTION, DATAFLOW)

.github/workflows/ci-python.yml

```
name: CI - Python Pipelines

on:
  pull_request:
    paths:
      - "**/*.py"

jobs:
  test-python:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"

      - name: Install dependencies
```

```
      run: |
        pip install -r requirements-dev.txt

    - name: Run unit tests
      run: |
        pytest tests/ --disable-warnings
```

🧪 Example Python Test (Ingestion)

```
# tests/ingestion/test_ingest.py

def test_filename_generation():
    date = "2026-01-01"
    assert f"date={date}" in f"orders/date={date}/file.csv"
```

Senior Rule

If you can't test it locally, don't deploy it.

🔹 CI PIPELINE 2 – BIGQUERY SQL SAFETY

.github/workflows/ci-sql.yml

```
name: CI - BigQuery SQL

on:
  pull_request:
    paths:
      - "**/*.sql"
```

```
jobs:
  sql-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Check for SELECT *
        run: |
          if grep -R "SELECT \*" .; then
            echo "❌ SELECT * detected"
            exit 1
          fi
```

Why This Matters

- Prevents accidental cost explosions
- Enforces senior-level discipline

🔹 CI PIPELINE 3 – AIRFLOW DAG VALIDATION

.github/workflows/ci-airflow.yml

```
name: CI - Airflow DAG Validation

on:
  pull_request:
    paths:
      - "04-orchestration/**/*.py"

jobs:
  airflow-validate:
```

```
  runs-on: ubuntu-latest
  steps:
   - uses: actions/checkout@v3

   - name: Install Airflow
     run: |
       pip install apache-airflow==2.7.0

   - name: Validate DAGs
     run: |
       python -c "from airflow.models import DagBag; DagBag().dags"
```

Senior Rule

A DAG that doesn't parse must never reach prod.

🔹 CI PIPELINE 4 – TERRAFORM SAFETY

.github/workflows/ci-terraform.yml

```
name: CI - Terraform

on:
 pull_request:
   paths:
    - "**/*.tf"

jobs:
 terraform:
   runs-on: ubuntu-latest
   steps:
```

```
    - uses: actions/checkout@v3

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v2

    - name: Terraform Init
      run: terraform init -backend=false

    - name: Terraform Validate
      run: terraform validate
```

Senior Rule

Terraform must validate before any apply.

🔹 CI PIPELINE 5 – SECURITY CHECKS (CRITICAL)

.github/workflows/security-checks.yml

```
name: CI - Security Checks

on:
  pull_request:

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Scan for secrets
```

```
      run: |
        if grep -R "API_KEY\|SECRET\|PASSWORD" .; then
          echo "❌ Hardcoded secret found"
          exit 1
        fi

    - name: Block Editor Role
      run: |
        if grep -R "roles/editor" .; then
          echo "❌ Editor role detected"
          exit 1
        fi
```

This One Pipeline:

- Prevents breaches
- Prevents audit failures
- Prevents career-ending mistakes

🧠 TEST PHILOSOPHY (IMPORTANT)

Senior Data Engineers test for:

- Safety
- Idempotency
- Cost awareness
- Failure resistance

Not for:

- 100% coverage
- Toy correctness

📘 ADD THIS TO README.md (CI SECTION)

CI/CD Standards

This repository uses GitHub Actions to enforce:

- Python unit testing
- SQL cost-safety rules
- Airflow DAG validation
- Terraform validation
- Security & IAM checks

Pull requests that fail CI will not be merged.

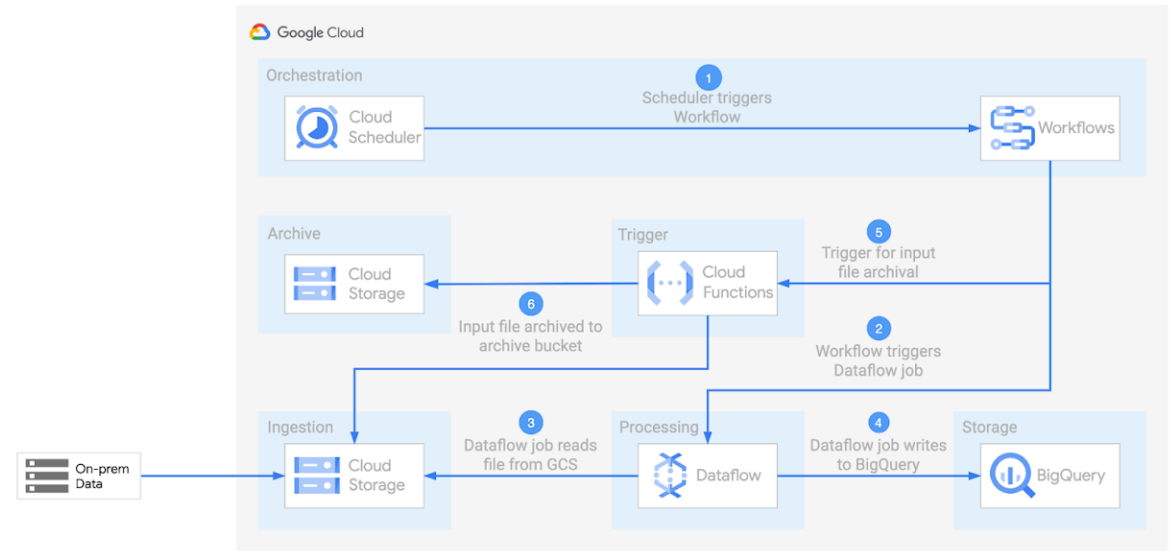
🏁 WHAT YOU NOW HAVE

You now own:

✅ Enterprise-grade CI/CD pipelines
✅ Automated safety enforcement
✅ Production-quality testing discipline
✅ Audit- and security-ready repo
✅ Senior / Staff-level engineering standards

This is exactly how real data platforms are protected.





5

🚀 DEPLOYMENT WORKFLOWS

Dev → Stage → Prod Promotion (Enterprise Standard)

Core Principles (Non-Negotiable)

- No direct prod deploys
- Artifacts are promoted, not rebuilt
- Manual approval for Prod
- Rollback is always possible
- Configs change, code does not

🧱 ENVIRONMENT MODEL

Branches:

- develop → DEV
- release → STAGE
- main → PROD

GitHub Environments:
- dev
- stage
- prod (manual approval required)

🔐 REQUIRED GITHUB SETUP (ONCE)

1. Create Environments: Settings → Environments
    - dev
    - stage
    - prod (require reviewers)
2. Secrets per Environment
    - GCP_PROJECT_ID
    - GCP_SA_KEY (JSON, base64)
    - Optional: BQ_LOCATION, REGION

🔹 WORKFLOW 1: DEPLOY TO DEV (AUTOMATIC)

.github/workflows/deploy-dev.yml

```
name: Deploy to DEV

on:
  push:
    branches: [develop]
```

```
jobs:
  deploy-dev:
    runs-on: ubuntu-latest
    environment: dev

    steps:
      - uses: actions/checkout@v3

      - name: Authenticate to GCP
        run: |
          echo "${{ secrets.GCP_SA_KEY }}" | base64 -d > sa.json
          gcloud auth activate-service-account --key-file=sa.json
          gcloud config set project ${{ secrets.GCP_PROJECT_ID }}

      - name: Deploy SQL (DEV)
        run: |
          bq query --use_legacy_sql=false < 02-bigquery/raw-to-curated/incremental_merge.sql

      - name: Deploy Airflow DAGs (DEV)
        run: |
          gsutil rsync -r 04-orchestration/airflow-dags gs://composer-dev-bucket/dags

      - name: Deploy Dataflow Templates (DEV)
        run: |
          echo "Build or update Dataflow templates for DEV"
```

What this does

- Fast feedback

- Safe experimentation
- No approvals needed

🔹 WORKFLOW 2: PROMOTE TO STAGE (CONTROLLED)

.github/workflows/deploy-stage.yml

```
name: Promote to STAGE

on:
  pull_request:
    branches: [release]

jobs:
  deploy-stage:
    runs-on: ubuntu-latest
    environment: stage

    steps:
      - uses: actions/checkout@v3

      - name: Authenticate to GCP
        run: |
          echo "${{ secrets.GCP_SA_KEY }}" | base64 -d > sa.json
          gcloud auth activate-service-account --key-file=sa.json
          gcloud config set project ${{ secrets.GCP_PROJECT_ID }}

      - name: Smoke Tests (STAGE)
        run: |
          echo "Run limited row-count & freshness checks"
```

```
    - name: Deploy SQL (STAGE)
      run: |
        bq query --use_legacy_sql=false < 02-bigquery/raw-to-curated/incremental_merge.sql

    - name: Deploy Airflow DAGs (STAGE)
      run: |
        gsutil rsync -r 04-orchestration/airflow-dags gs://composer-stage-bucket/dags
```

Gate conditions

- CI must pass
- PR review required
- Stage validates performance & cost

🔹 WORKFLOW 3: DEPLOY TO PROD (MANUAL APPROVAL)

.github/workflows/deploy-prod.yml

```
name: Deploy to PROD

on:
  push:
    branches: [main]

jobs:
  deploy-prod:
    runs-on: ubuntu-latest
    environment: prod   # 👈 requires manual approval

    steps:
```

```
    - uses: actions/checkout@v3

    - name: Authenticate to GCP
      run: |
        echo "${{ secrets.GCP_SA_KEY }}" | base64 -d > sa.json
        gcloud auth activate-service-account --key-file=sa.json
        gcloud config set project ${{ secrets.GCP_PROJECT_ID }}

    - name: Pre-Deploy Safety Check
      run: |
        echo "Confirm backfill safety, partitions, cost impact"

    - name: Deploy SQL (PROD)
      run: |
        bq query --use_legacy_sql=false < 02-bigquery/raw-to-curated/incremental_merge.sql

    - name: Deploy Airflow DAGs (PROD)
      run: |
        gsutil rsync -r 04-orchestration/airflow-dags gs://composer-prod-bucket/dags

    - name: Post-Deploy Validation
      run: |
        echo "Check freshness, row counts, alerts"
```

Why this is senior-grade

- Human approval
- Pre/post checks
- Blast-radius awareness

🔁 ROLLBACK STRATEGY (MANDATORY)

Option A: Git Revert (Preferred)

git revert <bad_commit>

git push origin main

Option B: Disable via Config

- Pause DAG
- Disable schedule
- Roll back SQL version

Senior rule: Rollback should take minutes, not hours.

🧠 RELEASE CHECKLIST (ADD TO README)

Release Checklist

- [ ] CI green

- [ ] Partition filters present

- [ ] Backfill safety confirmed

- [ ] Cost impact reviewed

- [ ] Rollback plan ready

🏁 WHAT YOU NOW HAVE (FINAL STATE)

You now own:

✅ Full Dev → Stage → Prod promotion
✅ Manual gates for Prod
✅ Safe rollback paths
✅ Enterprise release discipline
✅ Audit-friendly deployments

This repo now matches real internal data platform standards.


White-Label Business Model for eLearning
Company A develops and maintains the eLearning technology infrastructure, including course creation tools, user management, and analytics.
Company B uses Company A's platform, rebranded as its own, to offer training programs and courses to learners.
Company A
Service Provider
(eLearning Platform/ LMS Provider).
Company B
Reseller
(Online Academy/ Business)
Customer - Learner/Trainee
The customer engages with the training programs offered by Company B, often unaware that Company A provides the underlying platform



5

🏢 CYBERDUDEBIVASH

White-Label Enterprise Training Framework

(GCP Data Engineer – On-Job Enablement Program)

🎯 WHITE-LABEL OBJECTIVE

Enable corporate clients to:

- Train GCP Data Engineers fast
- Reduce onboarding time
- Eliminate production mistakes
- Standardize engineering practices
- Pass audits & reduce cost risk

While you:

- Retain core IP
- License delivery
- Control updates
- Monetize per client / cohort

🧱 WHITE-LABEL ARCHITECTURE

What Gets White-Labeled

Layer	White-Label	Notes
Repo name	✅	Client-branded
README/docs	✅	Client language
Training schedule	✅	Client cadence
Lab scenarios	⚠️	Context adapted
Core code	❌	CyberDudeBivash IP
Architecture	❌	IP retained

Rule: Clients brand the wrapper.
You own the engine.

📁 WHITE-LABEL REPO STRUCTURE

```
client-gcp-data-engineer-training/
│
├── README.md              # Client-branded
├── TRAINING_GUIDE.md      # Client onboarding
├── GOVERNANCE.md          # Client policies
├── labs/                  # Your labs (licensed)
```

```
├── runbooks/              # Client-specific ops
└── cyberdudebivash.md     # IP attribution (hidden/legal)
```

📘 CLIENT README.md (TEMPLATE)

```
# GCP Data Engineer – Enterprise Enablement Program

This repository contains the official GCP Data Engineering
training and on-job enablement material for <CLIENT_NAME>.

## Program Goals
- Standardize data engineering practices
- Reduce production incidents
- Improve cost efficiency
- Accelerate engineer productivity

## Target Audience
- New Data Engineers
- Existing GCP DEs
- Cloud engineers transitioning to data roles

## Expectations
- Engineers must complete labs sequentially
- Production safety rules are mandatory
- On-call simulations are required

## Support
For internal questions, contact <CLIENT_TEAM>.
```

📘 TRAINING_GUIDE.md (CLIENT VERSION)

Training Flow (Recommended)

Week 1–2:

- Foundation & GCP Core
- IAM & cost awareness

Week 3–4:

- Data ingestion & BigQuery
- Production pipelines

Week 5–6:

- Orchestration & CI/CD
- Incident simulations

Week 7+:

- Security, scale & optimization
- Leadership readiness

This makes the program HR & manager friendly.

🔐 GOVERNANCE.md (CRITICAL FOR ENTERPRISES)

Data Engineering Governance

Rules:

- No direct prod changes
- All deployments via CI/CD
- Least privilege IAM only
- All incidents documented

Compliance:
- Data access reviewed quarterly
- PII handled via approved patterns
- Audit logs retained

Ownership:
- Data owners defined per domain
- Platform owners assigned

This document alone wins trust with leadership.

🧪 LAB CONTEXT ADAPTATION (HOW TO WHITE-LABEL)

Original (Your IP)
“Vendor CSV ingestion pipeline”

White-Labeled
“<CLIENT_NAME> partner data ingestion pipeline”

Original
“Sales orders example”

White-Labeled
“Customer transactions (simulated)”

👉 Context changes. Architecture & code do not.

📜 LEGAL & IP SAFEGUARDS (IMPORTANT)

Hidden Attribution File (Do NOT Remove)

cyberdudebivash.md

This training framework and underlying architecture
are licensed from CyberDudeBivash Pvt Ltd.

Redistribution, modification of core logic,
or resale without permission is prohibited.

This protects you legally and commercially.

💰 PRICING MODELS (ENTERPRISE-READY)

Option 1 – Per Engineer License

- ₹X / $X per engineer
- Annual renewal

Option 2 – Cohort License

- Fixed price (20–50 engineers)
- Time-bound

Option 3 – Platform License

- Unlimited engineers
- Annual contract
- Highest value

Add-Ons (High Margin)

- Custom labs
- Client architecture mapping

- Live incident simulations
- Quarterly audits

🧠 DELIVERY MODES (YOU CHOOSE)

Mode	Description
Self-paced	Repo + docs
Guided	Weekly sessions
Hybrid	Labs + reviews
Full enablement	Training + audits

🏁 WHAT THIS UNLOCKS FOR YOU

You now have:

✅ A white-label enterprise product
✅ Recurring revenue potential
✅ Consulting + training bundling
✅ Low delivery cost, high value
✅ IP protection built-in

This is how boutique cloud firms scale.



Runbook Template

Runbook Template	
Runbook title	Restart Server
Runbook description	Instructions to restart a server.
Runbook scope	Server Operations Team
Runbook objective	Restart the server to apply updates or resolve issues.
Runbook steps	• 1. Log in to the server. • 2. Open the terminal or command prompt. • 3. Run the command: sudo systemctl restart [your-service-name] • 4. Verify the server restart by checking logs or service status. •
Runbook	• - If the server doesn't restart, check the error message the logs.

PagerDuty Operational Maturity Model: IT Automation

Reduction of Work and Interruptions

Manual	Reactive	Responsive	Proactive	Preventative
Runbooks are just manual tasks. Any automation is only safe for experts to run. Most Incidents and request tickets almost escalate to SMEs and developers.	**General Automation of First Steps.** General automation triggered for basic diagnostics for any incident. Automation is still only safe for experts to run. Incidents and tickets still escalate to SMEs and developers. Determine typical steps that responders first run for any incident and automate them. Trigger when any incident occurs.	**Delegate Self-Service Operations** Operators can run diagnostic and common remediation automation. Deploy a runbook automation platform to facilitate delegation of self-service operations. L1 responders can run common diagnostic and remediation automation instead of always escalating to SMEs.	**Automation-assisted response.** Context-specific diagnostics and mitigation are triggered when incident occurs as responders are paged. Automate deeper diagnostics and common mitigation such as service fail-over. Triggered by event rules as responders are paged. L1 responders can correct common recurring issues instead of always escalating to SMEs.	**Auto-remediation.** Automation attempts to correct common recurring incidents before responders are paged. Diagnose, mitigate, and remediate common issues. Triggered by event rules. Call to responders only on failure or exception. Escalations to SMEs and developers only occur for unusual problems and complex remediation.



DevOps runbook

Runbook name	
Runbook description	
Owner	@ mention the owner
Version	e.g., 2.3
Version date	Type // to add the date
On this page	• Architecture • Support contacts • Runs • Process

🏛 Architecture

Provide a high-level architectural diagram of your hosts and services so that your operations team can effectively troubleshoot an alert or outage.

💬 Support contacts

Expertise level	Team	Team lead	Contact info
Level 1	@ Team members	@ Lead	
Level 2			
Level 3			

Runs

List the routine operations tasks this runbook is automating.

Name	State	Start time	Completed time	Duration
e.g., Database backup	SUCCESS / ERROR			

📈 Process

List the procedures your team needs to follow to start, stop, monitor, and debug the system. Include procedures for every anticipated scenario.

	Step instructions	Enabled	Execution location	Run environments	Run conditions	Documentation
1		YES / NO				Add a link
2						
3						

5

📕 SRE RUNBOOK SYSTEM

GCP Data Engineering Platform

(Dev · Stage · Prod)

📁 RUNBOOK DIRECTORY STRUCTURE

```
runbooks/
│
├── README.md
│
├── dev/
│   ├── overview.md
│   ├── common-failures.md
│   └── deployment.md
│
├── stage/
│   ├── overview.md
│   ├── validation.md
│   └── release-checks.md
│
└── prod/
    ├── overview.md
    ├── incident-response.md
    ├── cost-emergencies.md
    ├── security-incidents.md
```

└── rollback.md

Each environment has different rules, risk levels, and response speed.

📘 runbooks/README.md (MASTER)

SRE Runbooks – Data Engineering Platform

These runbooks define **how to operate, debug, recover, and protect**
the GCP Data Engineering platform across environments.

Core Principles
- Production safety over speed
- Clear ownership
- Calm, repeatable responses
- No heroics

Severity Levels
- SEV-1: Business impact now
- SEV-2: Data delayed or degraded
- SEV-3: Non-critical issue

All engineers on-call must follow these runbooks.

🟢 DEV RUNBOOKS

(Safe Experimentation Zone)

📘 dev/overview.md

DEV Environment – Overview

Purpose:

- Development
- Testing
- Experimentation

Rules:

- No real customer data
- Failures are acceptable
- Cost monitoring still required

Allowed:

- Manual reruns
- Schema changes
- Debug logging

📘 dev/common-failures.md

Typical Issues

- IAM permission errors
- Broken DAGs
- Schema mismatches

Response

1. Check logs
2. Fix code or config

3. Rerun pipeline
4. Document learning

DEV failures are learning opportunities, not incidents.

📘 dev/deployment.md

DEV Deployment

Trigger:

- Push to develop branch

Checks:

- CI green
- No secrets
- SQL validated

Rollback:

- Fix forward preferred

🟡 STAGE RUNBOOKS

(Production Dress Rehearsal)

📘 stage/overview.md

STAGE Environment – Overview

Purpose:

- Pre-production validation
- Performance & cost testing

Rules:
- No experimentation
- Only release-candidate code
- Limited access

Failures here prevent prod incidents.

📘 stage/validation.md

Required Checks

- Row counts vs prod baseline
- Data freshness SLA
- Query performance
- Cost estimates

If Validation Fails

- Block promotion
- Fix issue
- Revalidate

Never bypass Stage failures.

📘 stage/release-checks.md

Release Checklist (STAGE)

- CI passed
- DAGs parsed

- Backfill safe
- Cost reviewed
- Rollback plan ready

🔴 PROD RUNBOOKS

(Business-Critical Zone)

📘 prod/overview.md

PROD Environment – Overview

Purpose:

- Serve business data
- Maintain trust
- Protect revenue

Rules:

- No manual changes
- CI/CD only
- Manual approval required
- Every action logged

📘 prod/incident-response.md

(MOST IMPORTANT FILE)

Incident Response – PROD

First 10 Minutes

1. Acknowledge alert
2. Assess impact
3. Stop further damage
4. Communicate status
5. Begin investigation

Never Do

- Panic
- Deploy unreviewed fixes
- Stay silent

Communication Template

"Data pipeline delay detected.
Impact: <affected dashboards>.
ETA: <time>."

📘 prod/cost-emergencies.md

Cost Emergency Runbook

Trigger:

- Cost alert > threshold

Steps:

1. Identify top queries/jobs
2. Pause non-critical pipelines
3. Apply partition filters

4. Communicate impact

Goal:

- Stop bleed first
- Optimize second

📘 prod/security-incidents.md

Security Incident Runbook

Examples:

- PII exposed
- Excessive access granted
- Suspicious query activity

Steps:

1. Revoke access immediately
2. Contain exposure
3. Notify security team
4. Preserve logs
5. Document incident

Never delete logs.

📘 prod/rollback.md

Rollback Runbook

Preferred:

- Git revert + redeploy

Alternative:

- Disable DAG
- Pause streaming job

Rollback should complete within minutes.

🧠 ON-CALL ENGINEER CHECKLIST

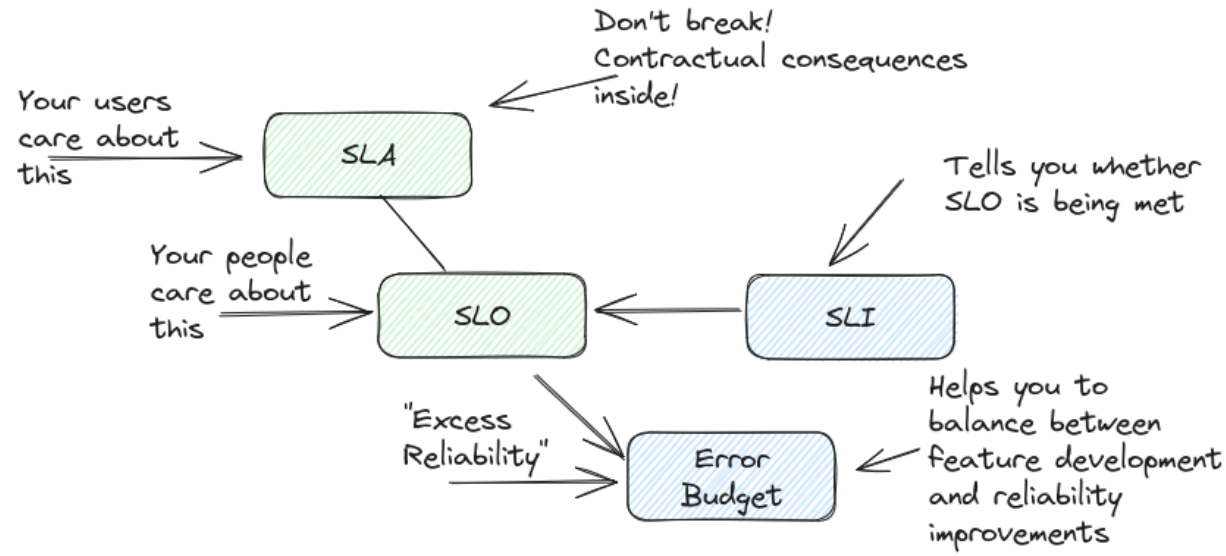
- Stay calm
- Follow runbook
- Communicate clearly
- Document actions
- Prevent recurrence

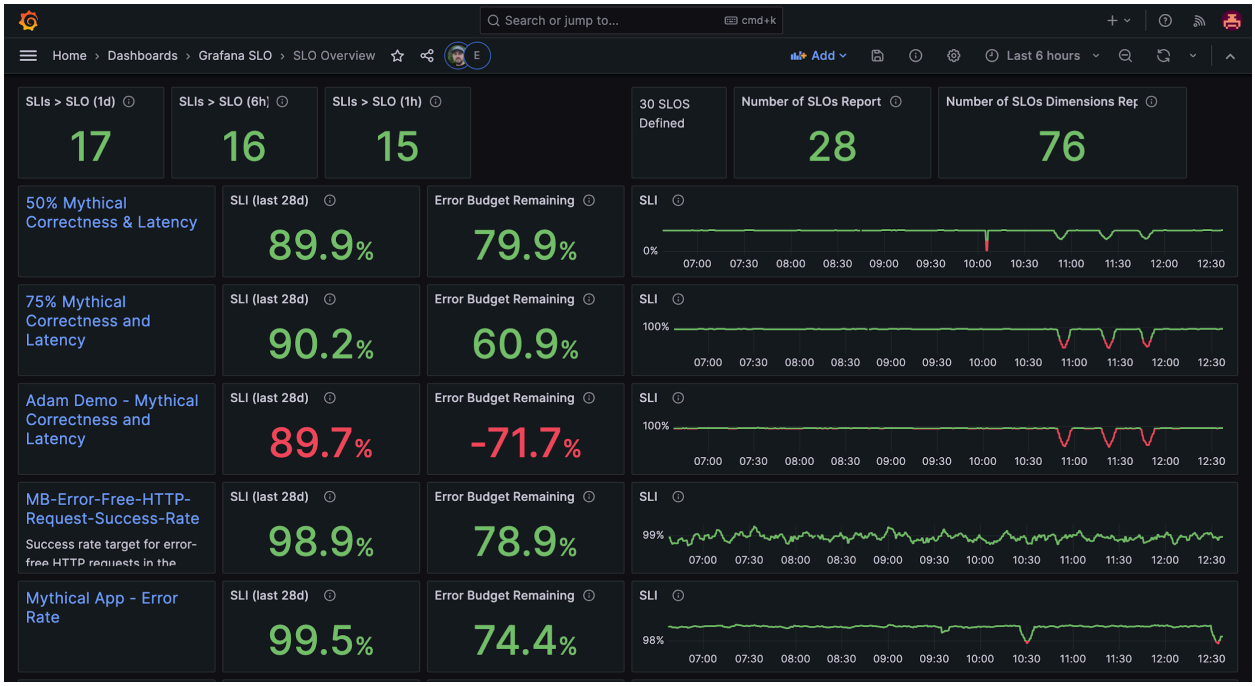
🏁 WHAT THIS ADDS (CRITICAL VALUE)

You now have:

✅ True SRE operational maturity
✅ Environment-specific discipline
✅ Repeatable incident handling
✅ Lower on-call stress
✅ Enterprise trust & audit readiness

Most teams do not have this.


Don't break!
Contractual consequences
inside!
Your users
care about
this
SLA
Tells you whether
SLO is being met
Your people
care about
this
SLO
SLI
"Excess
Reliability"
Error
Budget
Helps you to
balance between
feature development
and reliability
improvements


Search or jump to...
cmd+k
Home › Dashboards › Grafana SLO › SLO Overview
Add
Last 6 hours
SLIs > SLO (1d)
17
SLIs > SLO (6h)
16
SLIs > SLO (1h)
15
30 SLOS Defined
Number of SLOs Report
28
Number of SLOs Dimensions Rep
76
50% Mythical Correctness & Latency
SLI (last 28d)
89.9%
Error Budget Remaining
79.9%
SLI
75% Mythical Correctness and Latency
SLI (last 28d)
90.2%
Error Budget Remaining
60.9%
SLI
Adam Demo - Mythical Correctness and Latency
SLI (last 28d)
89.7%
Error Budget Remaining
-71.7%
SLI
MB-Error-Free-HTTP-Request-Success-Rate
Success rate target for error-free HTTP requests in the
SLI (last 28d)
98.9%
Error Budget Remaining
78.9%
SLI
Mythical App - Error Rate
SLI (last 28d)
99.5%
Error Budget Remaining
74.4%
SLI
07:00 07:30 08:00 08:30 09:00 09:30 10:00 10:30 11:00 11:30 12:00 12:30

📊 SRE RELIABILITY FRAMEWORK

SLIs · SLOs · Error Budgets

(For GCP Data Engineering Platforms)

🧠 WHY THIS MATTERS (EXECUTIVE REALITY)

Executives don't ask:

- "Was Dataflow up?"
- "Did Airflow run?"

They ask:

"Can I trust the data when I need it?"

SLIs/SLOs translate engineering reliability into business trust.

🧱 CORE DEFINITIONS (KEEP THIS IN REPO)

SLI (Service Level Indicator)

→ What we measure

SLO (Service Level Objective)

→ The reliability target

Error Budget

→ How much failure we tolerate

Failure is expected.
Uncontrolled failure is unacceptable.

📁 DIRECTORY STRUCTURE (ADD TO REPO)

```
sre/
│
├── README.md
├── slis.md
├── slos.md
├── error-budgets.md
├── dashboards.md
└── governance.md
```

📘 sre/README.md (MASTER OVERVIEW)

```
# Reliability Engineering for Data Platforms

This directory defines the reliability standards for the
GCP Data Engineering platform.

Goals:
- Quantify reliability
- Balance speed vs stability
- Prevent burnout
- Align engineering with business trust

Reliability is owned by the team, not individuals.
```

📘 sre/slis.md

SERVICE LEVEL INDICATORS (WHAT WE MEASURE)

1️⃣ Data Freshness SLI (MOST IMPORTANT)

Definition:

Time between expected and actual data availability.

Why:

Late data is worse than no data.

Example Metric

- Expected: Daily data by 06:00
- Measured: Timestamp of latest partition

2️⃣ Pipeline Success Rate SLI

Definition:

Percentage of successful pipeline runs.

Why:

Repeated failures destroy trust.

3️⃣ Data Quality SLI

Definition:

Percentage of records passing validation rules.

Checks:

- Nulls

- Duplicates

- Schema mismatches

4️⃣ Cost Stability SLI

Definition:

Daily cost deviation from baseline.

Why:

Cost explosions are reliability failures.

5️⃣ Incident Response Time SLI

Definition:

Time from alert to mitigation.

Why:

Speed of response matters more than perfection.

📘 sre/slos.md

SERVICE LEVEL OBJECTIVES (TARGETS)

- Example SLOs (Enterprise-Realistic)

Data Freshness SLO:

99% of datasets available within SLA window.

Pipeline Success SLO:

99.5% successful scheduled runs per month.

Data Quality SLO:

99.9% of records pass validation.

Incident Response SLO:

SEV-1 acknowledged within 5 minutes.

⚠️ Never set SLOs to 100%
That guarantees burnout and failure.

📘 sre/error-budgets.md

ERROR BUDGETS (THE SAFETY VALVE)

🔥 What Is an Error Budget?

Error Budget = 100% - SLO

If SLO = 99.5%
Error Budget = 0.5% failure allowed

🧠 Why Error Budgets Matter

They decide:

- When you can ship fast
- When you must slow down
- When you freeze releases

📊 Example Error Budget Policy

If error budget remaining > 50%:

- Normal releases allowed

If error budget < 25%:

- Only critical fixes

If error budget exhausted:

- Release freeze
- Reliability work only

This removes emotional decision-making.

📘 sre/dashboards.md

OBSERVABILITY & VISIBILITY

Core Dashboards

1. Data Freshness Dashboard
2. Pipeline Success Dashboard
3. Cost Trend Dashboard
4. Incident Timeline Dashboard

Dashboards must answer:

- Is data fresh?
- Is reliability degrading?
- Is cost stable?

Senior Rule

If reliability is not visible, it does not exist.

📘 sre/governance.md

RELIABILITY GOVERNANCE (LEADERSHIP LEVEL)

Reliability Reviews:

- Weekly: SLI trends
- Monthly: SLO compliance
- Quarterly: SLO recalibration

Ownership:

- Team owns SLIs/SLOs
- Leadership approves SLO changes

🧪 PRACTICAL EXAMPLE (DATA PIPELINE)

Daily Sales Pipeline

Metric	SLI	SLO
Freshness	< 30 min delay	99%
Success	DAG success	99.5%
Quality	Duplicate rate	< 0.1%
Cost	Daily spend	±10%

If SLO violated → error budget consumed.

🚨 WHEN ERROR BUDGET IS EXHAUSTED

Mandatory actions:

- Freeze feature releases
- Fix root causes
- Improve monitoring
- Update runbooks

This protects engineers from unrealistic pressure.

🧠 CULTURAL SHIFT THIS ENABLES

Without SLOs:

- Arguments
- Blame
- Panic

With SLOs:

- Data-driven decisions
- Clear priorities
- Calm operations

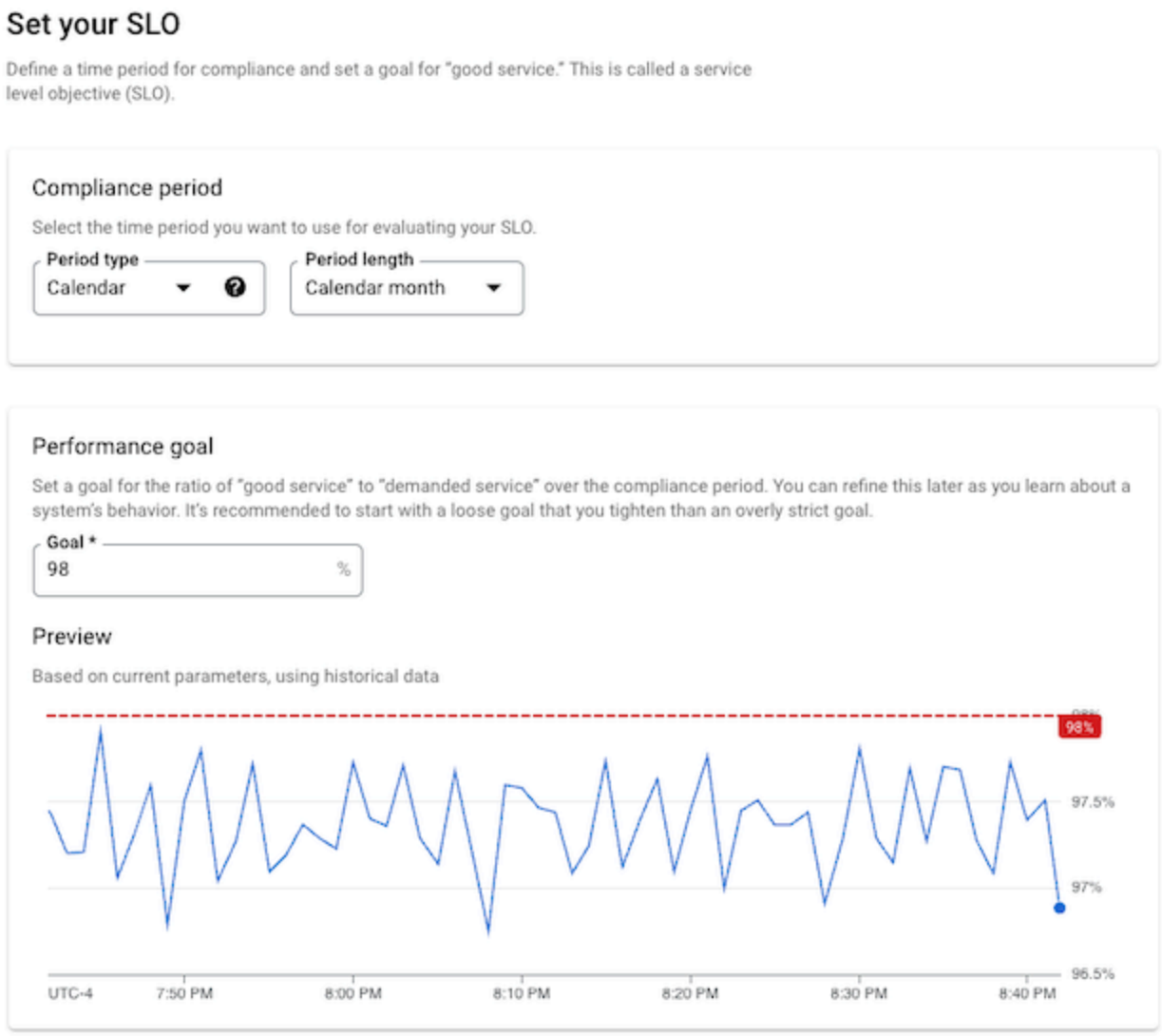
This is why SRE exists.

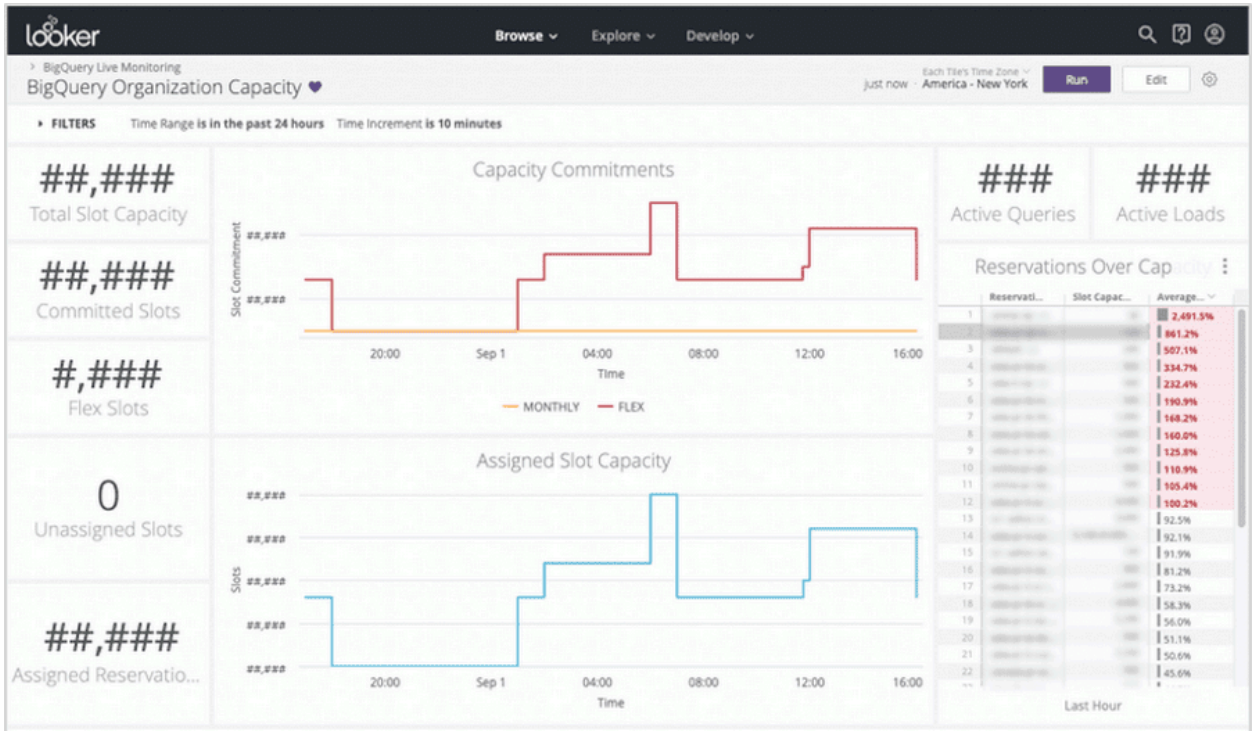
🏁 WHAT YOU NOW HAVE (RARE)

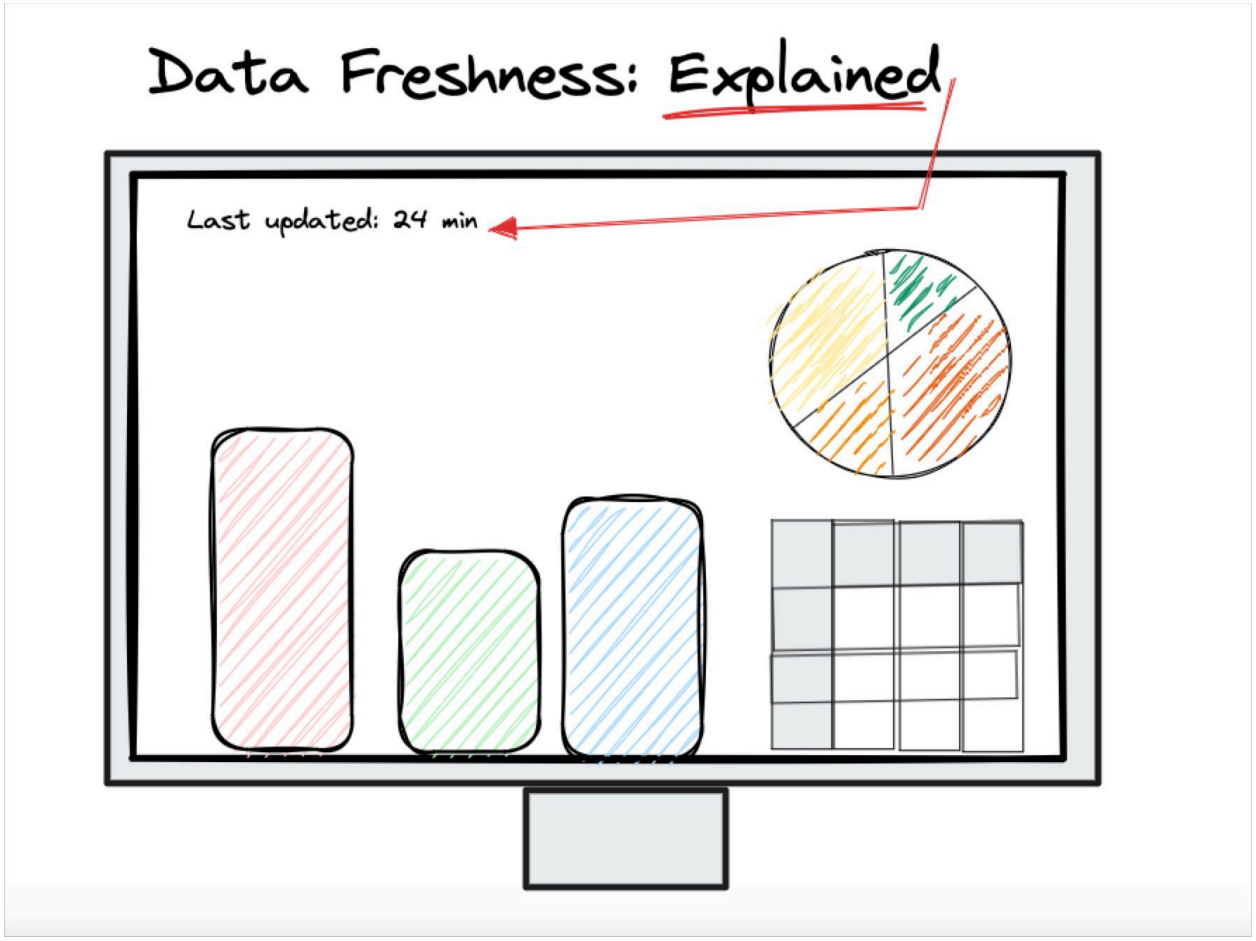
You now own:

✅ True SRE reliability framework
✅ Business-aligned metrics
✅ Burnout protection via error budgets
✅ Leadership-ready governance
✅ Elite enterprise credibility

Most data teams never reach this level.




looker
Browse
Explore
Develop
BigQuery Live Monitoring
BigQuery Organization Capacity
Run
Edit
FILTERS
Time Range is in the past 24 hours
Time Increment is 10 minutes
##,###
Total Slot Capacity
##,###
Committed Slots
#,###
Flex Slots
0
Unassigned Slots
##,###
Assigned Reservatio...
Capacity Commitments
Slot Commitment
20:00
Sep 1
04:00
08:00
12:00
16:00
Time
MONTHLY
FLEX
###
Active Queries
###
Active Loads
Reservations Over Cap
Assigned Slot Capacity
Slots
Last Hour


Data Freshness: Explained
Last updated: 24 min

5

📊 SLO DASHBOARDS

BigQuery + Cloud Monitoring (Enterprise Implementation)

🎯 DASHBOARD GOALS (NON-NEGOTIABLE)

Every SLO dashboard must answer 4 questions in <30 seconds:

1. Is the data fresh?
2. Are pipelines reliable?
3. Is data trustworthy?
4. Are we burning error budget?

If a dashboard can't answer these → it's noise.

🧱 ARCHITECTURE OVERVIEW

Pipelines / DAGs / Dataflow

↓

BigQuery Metrics Tables

↓

Cloud Monitoring (Custom Metrics)

↓

SLO Dashboards

Key principle:

Metrics are data too — treat them like products.

📁 DIRECTORY STRUCTURE (ADD TO REPO)

```
sre/dashboards/
│
├── README.md
├── bigquery/
│   ├── freshness_metrics.sql
│   ├── pipeline_success.sql
│   ├── data_quality.sql
│   └── cost_metrics.sql
│
├── monitoring/
│   ├── custom_metrics.md
│   └── slo_config.md
│
└── examples/
    └── executive_view.md
```

📘 sre/dashboards/README.md

```
# SLO Dashboards – Data Engineering Platform

These dashboards provide real-time visibility into
data reliability using SLIs and SLOs.

Audience:
- Data Engineers (operations)
- SREs (reliability)
```

- Leadership (trust & risk)

Dashboards are source-of-truth for reliability decisions.

🟢 DASHBOARD 1 – DATA FRESHNESS (MOST IMPORTANT)

📊 BigQuery Metric Table

freshness_metrics.sql

```
CREATE OR REPLACE TABLE sre_metrics.data_freshness AS
SELECT
  'orders_pipeline' AS pipeline_name,
  CURRENT_TIMESTAMP() AS measured_at,
  TIMESTAMP_DIFF(
    CURRENT_TIMESTAMP(),
    MAX(_PARTITIONTIME),
    MINUTE
  ) AS delay_minutes
FROM curated.orders;
```

🎯 SLI

- Metric: Delay in minutes

🎯 SLO

- 99% of runs < 30 minutes delay

📈 Dashboard Visualization

- Line chart: delay over time
- Threshold line at SLA (30 min)
- Red zone = SLO violation

Senior Rule

Late data is a SEV-1 reliability issue, even if pipelines succeeded.

🟡 DASHBOARD 2 – PIPELINE SUCCESS RATE

pipeline_success.sql

```
CREATE OR REPLACE TABLE sre_metrics.pipeline_success AS
SELECT
  dag_id,
  DATE(execution_date) AS run_date,
  COUNTIF(state = 'success') / COUNT(*) AS success_rate
FROM airflow_metadata.task_instance
GROUP BY dag_id, run_date;
```

🎯 SLI

- % successful scheduled runs

🎯 SLO

- 99.5% monthly success

Visualization

- Bar/line per DAG
- Monthly rollup
- Error budget burn indicator

Senior Rule

Repeated retries ≠ success.

🔵 DASHBOARD 3 – DATA QUALITY

data_quality.sql

```
CREATE OR REPLACE TABLE sre_metrics.data_quality AS
SELECT
  CURRENT_DATE() AS date,
  COUNTIF(order_id IS NULL) / COUNT(*) AS null_rate
FROM curated.orders;
```

🎯 SLI

- Null / duplicate rate

🎯 SLO

- < 0.1% bad records

Visualization

- Gauge chart
- Green / yellow / red zones

🔴 DASHBOARD 4 – COST STABILITY

cost_metrics.sql

```
CREATE OR REPLACE TABLE sre_metrics.cost_daily AS
SELECT
  DATE(usage_start_time) AS date,
  SUM(cost) AS daily_cost
FROM `gcp_billing_export`
GROUP BY date;
```

🎯 SLI

- Daily cost deviation

🎯 SLO

- ±10% of baseline

Visualization

- Cost trend
- Anomaly highlight
- Error budget impact

🧠 ERROR BUDGET BURN DASHBOARD (CRITICAL)

Conceptual Metric

Error Budget Burn =

Number of SLO violations / Allowed failures

Visual:

- Monthly burn-down chart
- Red zone = release freeze

☁️ CLOUD MONITORING – CUSTOM METRICS

📘 monitoring/custom_metrics.md

Custom metrics exported from BigQuery:

- data_freshness_delay

- pipeline_success_rate

- data_quality_rate

- daily_cost_deviation

Used for:

- Alerts

- SLO tracking

- Executive dashboards

📘 monitoring/slo_config.md

SLO Objects:

- Availability SLO → pipeline success

- Latency SLO → data freshness

- Quality SLO → validation checks

Error Budget Policy:

- Burn > 75% → Freeze releases

👔 EXECUTIVE DASHBOARD (WHITE-LABEL FRIENDLY)

examples/executive_view.md

Executive Reliability View

- Data Freshness: GREEN
- Pipeline Reliability: GREEN
- Data Quality: GREEN
- Cost Stability: YELLOW

Overall Trust Status: ACCEPTABLE

Executives don’t need charts.
They need status and risk.

🧠 OPERATIONAL RULES (ADD TO RUNBOOKS)

- Engineers watch detailed dashboards
- Leads watch SLO compliance
- Leadership watches error budget

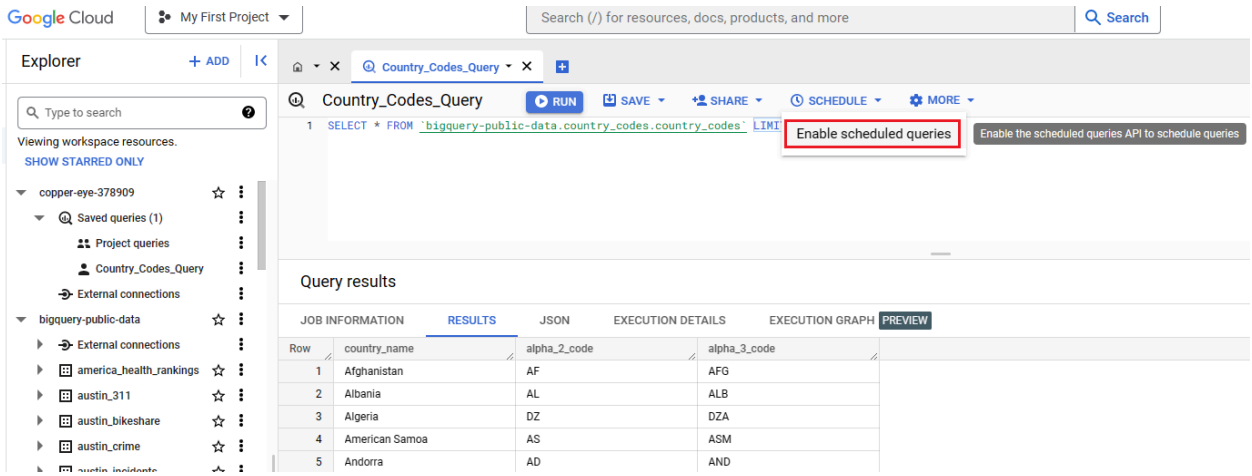
This avoids micromanagement.

🏁 WHAT YOU NOW HAVE (VERY RARE)

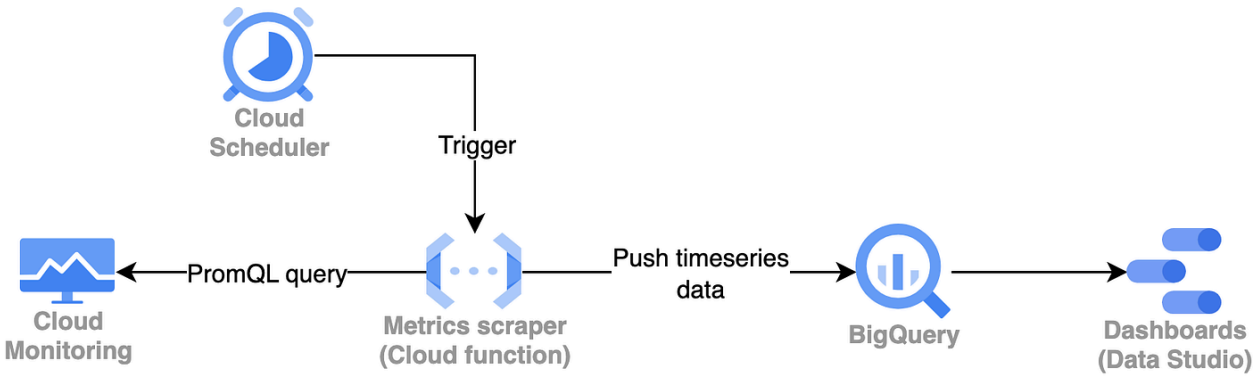
You now own:

✅ End-to-end SLO dashboards
✅ BigQuery-backed SLIs
✅ Cloud Monitoring integration
✅ Error-budget-driven decisions
✅ Executive-grade observability

This is top-tier SRE maturity for data platforms.







⏱️ AUTOMATED SLI PIPELINE

BigQuery Scheduled Jobs (Enterprise Standard)

Golden Rule:
SLIs must never depend on humans.

🧱 OVERALL DESIGN

Source Tables / Logs

↓

Scheduled BigQuery Queries

↓

sre_metrics Dataset

↓

Dashboards & Alerts

All SLI tables live in one dataset:

sre_metrics

📁 DIRECTORY STRUCTURE (ADD TO REPO)

sre/scheduled-jobs/

```
│
├── README.md
├── freshness_job.sql
├── pipeline_success_job.sql
├── data_quality_job.sql
├── cost_job.sql
```

└── create_jobs.md

📘 sre/scheduled-jobs/README.md

BigQuery Scheduled Jobs – SLI Automation

These scheduled queries automatically populate
SLI tables used for SLO dashboards and alerts.

Principles:
- Idempotent writes
- Append-only metrics
- Clear timestamps
- Low cost

🟢 JOB 1 – DATA FRESHNESS SLI

📄 freshness_job.sql

```
INSERT INTO sre_metrics.data_freshness
SELECT
  'orders_pipeline' AS pipeline_name,
  CURRENT_TIMESTAMP() AS measured_at,
  TIMESTAMP_DIFF(
    CURRENT_TIMESTAMP(),
    MAX(_PARTITIONTIME),
    MINUTE
```

```
 ) AS delay_minutes
FROM curated.orders;
```

⏱️ Schedule

- Frequency: Every 15 minutes
- Reason: Freshness is time-sensitive

🟡 JOB 2 – PIPELINE SUCCESS RATE SLI

📄 pipeline_success_job.sql

```
INSERT INTO sre_metrics.pipeline_success
SELECT
  dag_id,
  CURRENT_DATE() AS run_date,
  COUNTIF(state = 'success') / COUNT(*) AS success_rate,
  CURRENT_TIMESTAMP() AS measured_at
FROM airflow_metadata.task_instance
WHERE execution_date >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)
GROUP BY dag_id;
```

⏱️ Schedule

- Frequency: Hourly
- Reason: Track rolling reliability

🔵 JOB 3 – DATA QUALITY SLI

📄 data_quality_job.sql

```
INSERT INTO sre_metrics.data_quality
SELECT
  'orders' AS dataset_name,
  CURRENT_DATE() AS date,
  COUNTIF(order_id IS NULL) / COUNT(*) AS null_rate,
  CURRENT_TIMESTAMP() AS measured_at
FROM curated.orders;
```

⏱️ Schedule

- Frequency: Daily (after ingestion completes)
- Reason: Quality evaluated post-load

🔴 JOB 4 – COST STABILITY SLI

📄 cost_job.sql

```
INSERT INTO sre_metrics.cost_daily
SELECT
  DATE(usage_start_time) AS date,
  SUM(cost) AS daily_cost,
  CURRENT_TIMESTAMP() AS measured_at
FROM `gcp_billing_export`
WHERE usage_start_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)
GROUP BY date;
```

⏱️ Schedule

- Frequency: Daily
- Reason: Billing data updates daily

🧠 IMPORTANT DESIGN DECISIONS (SENIOR LEVEL)

✅ Append-Only Metrics

- Never overwrite history
- Enables trend analysis
- Required for audits

✅ Explicit Timestamps

- measured_at is mandatory
- Enables SLO burn calculations

✅ Separate Metrics Dataset

- No coupling with business tables
- Safe to expose read-only

⚙️ HOW TO CREATE SCHEDULED QUERIES

📘 create_jobs.md

Creating BigQuery Scheduled Queries

1. Open BigQuery → Scheduled Queries
2. Create new schedule
3. Paste SQL file
4. Set destination dataset: sre_metrics
5. Set write mode: Append
6. Configure schedule frequency
7. Use dedicated service account

IAM Required

- roles/bigquery.jobUser
- roles/bigquery.dataEditor (metrics dataset only)

🔐 SECURITY & GOVERNANCE (CRITICAL)

- Use service account, not user
- Metrics dataset = read-only for analysts
- No PII allowed in SLI tables
- Retain metrics for 12–18 months

🚨 FAILURE HANDLING

If a scheduled job fails:

1. Alert SRE/Data team
2. Fix source issue
3. Backfill metrics if needed
4. Document incident

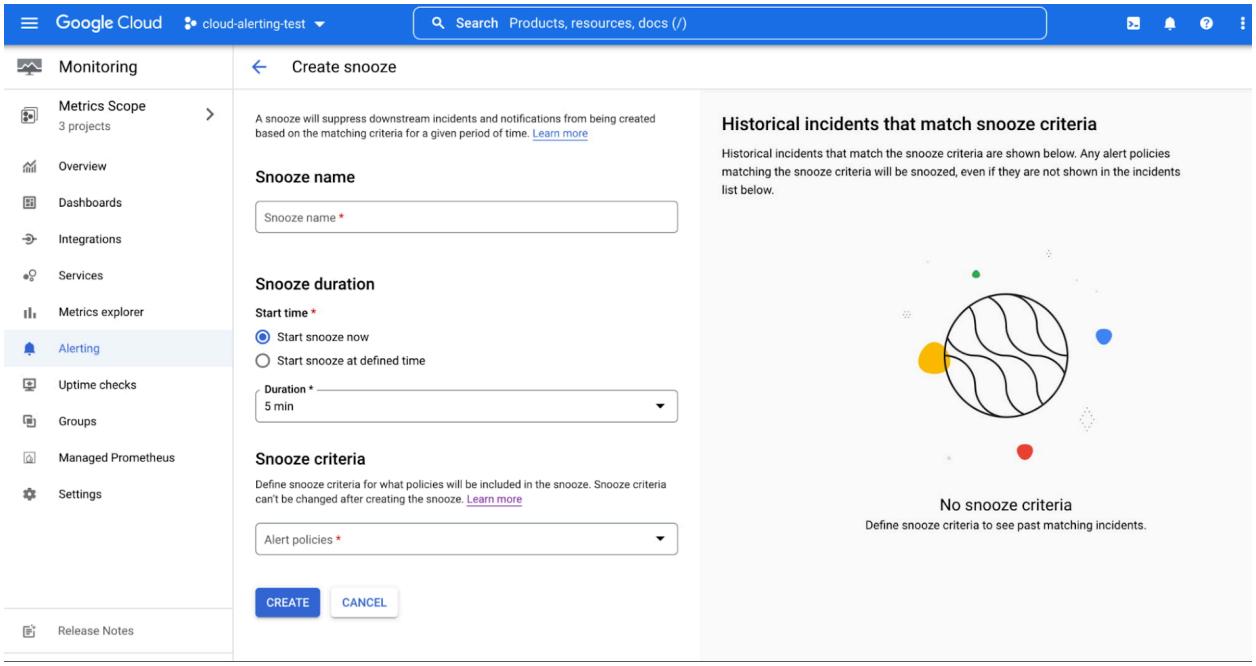
Missing metrics = blind reliability

📊 WHAT THIS ENABLES IMMEDIATELY

You now have:

✅ Fully automated SLIs
✅ Zero manual intervention
✅ Reliable SLO tracking
✅ Historical reliability data
✅ Audit & exec-ready metrics

This is exactly how mature SRE orgs operate.


Google Cloud
cloud-alerting-test
Search Products, resources, docs (/)
Monitoring
Metrics Scope
3 projects
Overview
Dashboards
Integrations
Services
Metrics explorer
Alerting
Uptime checks
Groups
Managed Prometheus
Settings
Release Notes
Create snooze
A snooze will suppress downstream incidents and notifications from being created based on the matching criteria for a given period of time. Learn more
Snooze name
Snooze name *
Snooze duration
Start time *
Start snooze now
Start snooze at defined time
Duration *
5 min
Snooze criteria
Define snooze criteria for what policies will be included in the snooze. Snooze criteria can't be changed after creating the snooze. Learn more
Alert policies *
CREATE
CANCEL
Historical incidents that match snooze criteria
Historical incidents that match the snooze criteria are shown below. Any alert policies matching the snooze criteria will be snoozed, even if they are not shown in the incidents list below.
No snooze criteria
Define snooze criteria to see past matching incidents.



Create SLO burn rate alert policy

1 Set SLO alert conditions

Creating an alert condition on your service-level objectives (SLOs) will let you know whether you are in danger of violating an SLO.

Target: 95% Availability, rolling 28 days

Select a burn rate threshold value that constitutes a violation, and a lookback duration period for which the violation is permitted. If the burn rate threshold is exceeded for more than the allowable period, an incident is created. Learn more

Display name *
SLO Alerting Demo - 2% burn in 1 hour

Lookback duration *
60 minute(s)

Burn rate threshold *
10

NEXT

2 Who should be notified? (optional)

When alerting policy violations occur, you will be notified via these channels.

3 What are the steps to fix the issue? (optional)



5

🚨 SLO-BASED ALERTING SYSTEM

Cloud Monitoring Alert Policies (Enterprise Grade)

Golden Rule of SRE Alerting:
Alert on user pain, not system noise.

🎯 WHY SLO-BASED ALERTING (IMPORTANT)

❌ Bad alerts:

- DAG failed once
- Job retried
- Temporary spike

✅ Good alerts:

- Data is late beyond SLO
- Reliability trend is degrading

- Error budget is burning too fast

This keeps humans focused on what matters.

🧱 ALERTING ARCHITECTURE

BigQuery SLI Tables

↓

Cloud Monitoring Custom Metrics

↓

SLO Objects

↓

Alert Policies

↓

On-Call / Slack / Email

📁 DIRECTORY STRUCTURE (ADD TO REPO)

sre/alerts/

│

├── README.md

├── freshness_alert.md

├── pipeline_success_alert.md

├── data_quality_alert.md

├── cost_alert.md

└── error_budget_alert.md

📘 sre/alerts/README.md

SLO-Based Alerting

These alerts are tied to Service Level Objectives (SLOs),
not individual task failures.

Principles:
- Alert on sustained impact
- Minimize noise
- Protect on-call engineers
- Align with business trust

🟢 ALERT 1 – DATA FRESHNESS SLO VIOLATION (SEV-1)

📄 freshness_alert.md

SLI

- data_freshness_delay_minutes

SLO

- 99% of data available within 30 minutes

Alert Condition

Trigger if:

Delay > 30 minutes

for 2 consecutive measurements

Severity

- SEV-1

Notification

- On-call channel
- Data leadership

Why This Is Critical

Late data = blind decisions.

🟡 ALERT 2 – PIPELINE RELIABILITY DEGRADATION (SEV-2)

📄 pipeline_success_alert.md

SLI

- pipeline_success_rate

SLO

- ≥ 99.5% success per month

Alert Condition

Trigger if:

Rolling success rate < 99.5%

for 3 hours

Severity

- SEV-2

Why Not SEV-1?

Single failures are expected.
Trends indicate real risk.

🔵 ALERT 3 – DATA QUALITY BREACH (SEV-2)

📄 data_quality_alert.md

SLI

- null_rate
- duplicate_rate

SLO

- < 0.1% bad records

Alert Condition

Trigger if:

Null or duplicate rate > 0.1%

after pipeline completion

Severity

- SEV-2

Business Impact

- Wrong analytics
- Broken dashboards
- Loss of trust

🔴 ALERT 4 – COST ANOMALY (SEV-1)

📄 cost_alert.md

SLI

- daily_cost

SLO

- ±10% from baseline

Alert Condition

Trigger if:

Daily cost > baseline + 20%

Severity

- SEV-1

Reason

Cost explosions are production incidents.

🧠 ALERT 5 – ERROR BUDGET BURN (MOST IMPORTANT)

📄 error_budget_alert.md

SLI

- Error budget remaining %

Alert Condition

Trigger if:

Error budget < 25%

Severity

- SEV-1 (Management Attention)

Mandatory Action

- Freeze non-critical releases
- Prioritize reliability work

This alert protects engineers from burnout.

☁️ CLOUD MONITORING IMPLEMENTATION (HOW-TO)

Step 1: Create Custom Metrics

- From BigQuery → Cloud Monitoring
- Metrics:
  - custom.googleapis.com/data/freshness_delay
  - custom.googleapis.com/pipeline/success_rate
  - custom.googleapis.com/data/quality_rate
  - custom.googleapis.com/cost/daily

Step 2: Create SLO Objects

- Attach SLIs to services
- Define objectives
- Enable error budget tracking

Step 3: Create Alert Policies

- Based on SLO burn rate
- Not raw metric spikes
- Add notification channels

🧠 ALERT SEVERITY MATRIX (ADD TO RUNBOOKS)

Alert Type	Severity	Response
Freshness	SEV-1	Immediate
Cost	SEV-1	Immediate
Error Budget	SEV-1	Leadership

Reliability Trend	SEV-2	Same day
Quality	SEV-2	Same day

🚫 WHAT NOT TO ALERT ON (IMPORTANT)

❌ Single DAG failure
❌ Retries
❌ Short spikes
❌ Non-user-facing pipelines

These create alert fatigue.

🧠 COMMUNICATION TEMPLATE (ON-CALL SAFE)

SLO violation detected:

Type: Data Freshness

Impact: Sales dashboard delayed

ETA: 20 minutes

Status: Mitigation in progress

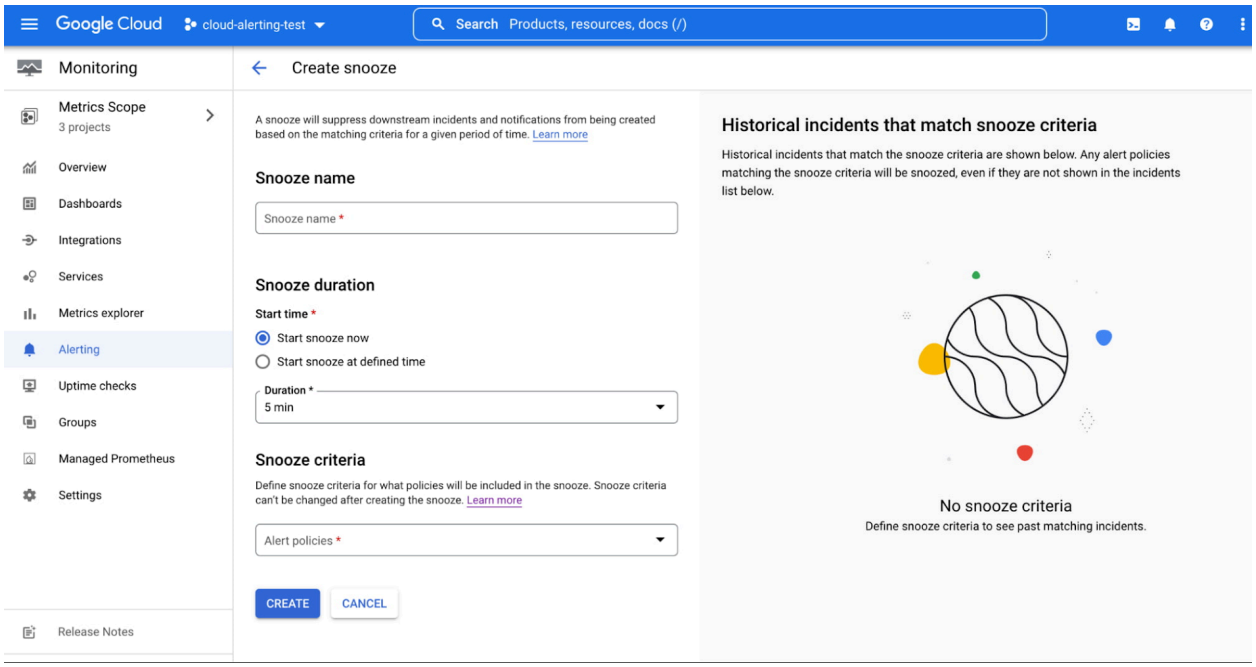
Calm. Clear. Trusted.

🏁 WHAT YOU NOW HAVE (TOP 1%)

You now own:

✅ True SLO-based alerting
✅ Error-budget-driven decisions
✅ Low-noise, high-signal alerts
✅ Protected on-call engineers
✅ Executive-aligned reliability ops

This is elite SRE maturity for data engineering.


Google Cloud
cloud-alerting-test
Search Products, resources, docs (/)
Monitoring
Metrics Scope
3 projects
Overview
Dashboards
Integrations
Services
Metrics explorer
Alerting
Uptime checks
Groups
Managed Prometheus
Settings
Release Notes
Create snooze
A snooze will suppress downstream incidents and notifications from being created based on the matching criteria for a given period of time. Learn more
Snooze name
Snooze name *
Snooze duration
Start time *
Start snooze now
Start snooze at defined time
Duration *
5 min
Snooze criteria
Define snooze criteria for what policies will be included in the snooze. Snooze criteria can't be changed after creating the snooze. Learn more
Alert policies *
CREATE
CANCEL
Historical incidents that match snooze criteria
Historical incidents that match the snooze criteria are shown below. Any alert policies matching the snooze criteria will be snoozed, even if they are not shown in the incidents list below.
No snooze criteria
Define snooze criteria to see past matching incidents.



Create SLO burn rate alert policy

1 Set SLO alert conditions

Creating an alert condition on your service-level objectives (SLOs) will let you know whether you are in danger of violating an SLO.

Target: 95% Availability, rolling 28 days

Select a burn rate threshold value that constitutes a violation, and a lookback duration period for which the violation is permitted. If the burn rate threshold is exceeded for more than the allowable period, an incident is created. Learn more

Display name *
SLO Alerting Demo - 2% burn in 1 hour

Lookback duration *
60 minute(s)

Burn rate threshold *
10

NEXT

2 Who should be notified? (optional)

When alerting policy violations occur, you will be notified via these channels.

3 What are the steps to fix the issue? (optional)



5

🚨 SLO-BASED ALERTING SYSTEM

Cloud Monitoring Alert Policies (Enterprise Grade)

Golden Rule of SRE Alerting:
Alert on user pain, not system noise.

🎯 WHY SLO-BASED ALERTING (IMPORTANT)

❌ Bad alerts:

- DAG failed once
- Job retried
- Temporary spike

✅ Good alerts:

- Data is late beyond SLO
- Reliability trend is degrading

- Error budget is burning too fast

This keeps humans focused on what matters.

🧱 ALERTING ARCHITECTURE

BigQuery SLI Tables

↓

Cloud Monitoring Custom Metrics

↓

SLO Objects

↓

Alert Policies

↓

On-Call / Slack / Email

📁 DIRECTORY STRUCTURE (ADD TO REPO)

sre/alerts/

│

├── README.md

├── freshness_alert.md

├── pipeline_success_alert.md

├── data_quality_alert.md

├── cost_alert.md

└── error_budget_alert.md

📘 sre/alerts/README.md

SLO-Based Alerting

These alerts are tied to Service Level Objectives (SLOs),
not individual task failures.

Principles:
- Alert on sustained impact
- Minimize noise
- Protect on-call engineers
- Align with business trust

🟢 ALERT 1 – DATA FRESHNESS SLO VIOLATION (SEV-1)

📄 freshness_alert.md

SLI

- data_freshness_delay_minutes

SLO

- 99% of data available within 30 minutes

Alert Condition

Trigger if:

Delay > 30 minutes

for 2 consecutive measurements

Severity

- SEV-1

Notification

- On-call channel
- Data leadership

Why This Is Critical

Late data = blind decisions.

🟡 ALERT 2 – PIPELINE RELIABILITY DEGRADATION (SEV-2)

📄 pipeline_success_alert.md

SLI

- pipeline_success_rate

SLO

- ≥ 99.5% success per month

Alert Condition

Trigger if:

Rolling success rate < 99.5%

for 3 hours

Severity

- SEV-2

Why Not SEV-1?

Single failures are expected.
Trends indicate real risk.

🔵 ALERT 3 – DATA QUALITY BREACH (SEV-2)

📄 data_quality_alert.md

SLI

- null_rate
- duplicate_rate

SLO

- < 0.1% bad records

Alert Condition

Trigger if:

Null or duplicate rate > 0.1%

after pipeline completion

Severity

- SEV-2

Business Impact

- Wrong analytics
- Broken dashboards
- Loss of trust

🔴 ALERT 4 – COST ANOMALY (SEV-1)

📄 cost_alert.md

SLI

- daily_cost

SLO

- ±10% from baseline

Alert Condition

Trigger if:

Daily cost > baseline + 20%

Severity

- SEV-1

Reason

Cost explosions are production incidents.

🧠 ALERT 5 – ERROR BUDGET BURN (MOST IMPORTANT)

📄 error_budget_alert.md

SLI

- Error budget remaining %

Alert Condition

Trigger if:

Error budget < 25%

Severity

- SEV-1 (Management Attention)

Mandatory Action

- Freeze non-critical releases
- Prioritize reliability work

This alert protects engineers from burnout.

☁️ CLOUD MONITORING IMPLEMENTATION (HOW-TO)

Step 1: Create Custom Metrics

- From BigQuery → Cloud Monitoring
- Metrics:
  - custom.googleapis.com/data/freshness_delay
  - custom.googleapis.com/pipeline/success_rate
  - custom.googleapis.com/data/quality_rate
  - custom.googleapis.com/cost/daily

Step 2: Create SLO Objects

- Attach SLIs to services
- Define objectives
- Enable error budget tracking

Step 3: Create Alert Policies

- Based on SLO burn rate
- Not raw metric spikes
- Add notification channels

🧠 ALERT SEVERITY MATRIX (ADD TO RUNBOOKS)

Alert Type	Severity	Response
Freshness	SEV-1	Immediate
Cost	SEV-1	Immediate
Error Budget	SEV-1	Leadership

Reliability Trend	SEV-2	Same day
Quality	SEV-2	Same day

🚫 WHAT NOT TO ALERT ON (IMPORTANT)

❌ Single DAG failure
❌ Retries
❌ Short spikes
❌ Non-user-facing pipelines

These create alert fatigue.

🧠 COMMUNICATION TEMPLATE (ON-CALL SAFE)

SLO violation detected:

Type: Data Freshness

Impact: Sales dashboard delayed

ETA: 20 minutes

Status: Mitigation in progress

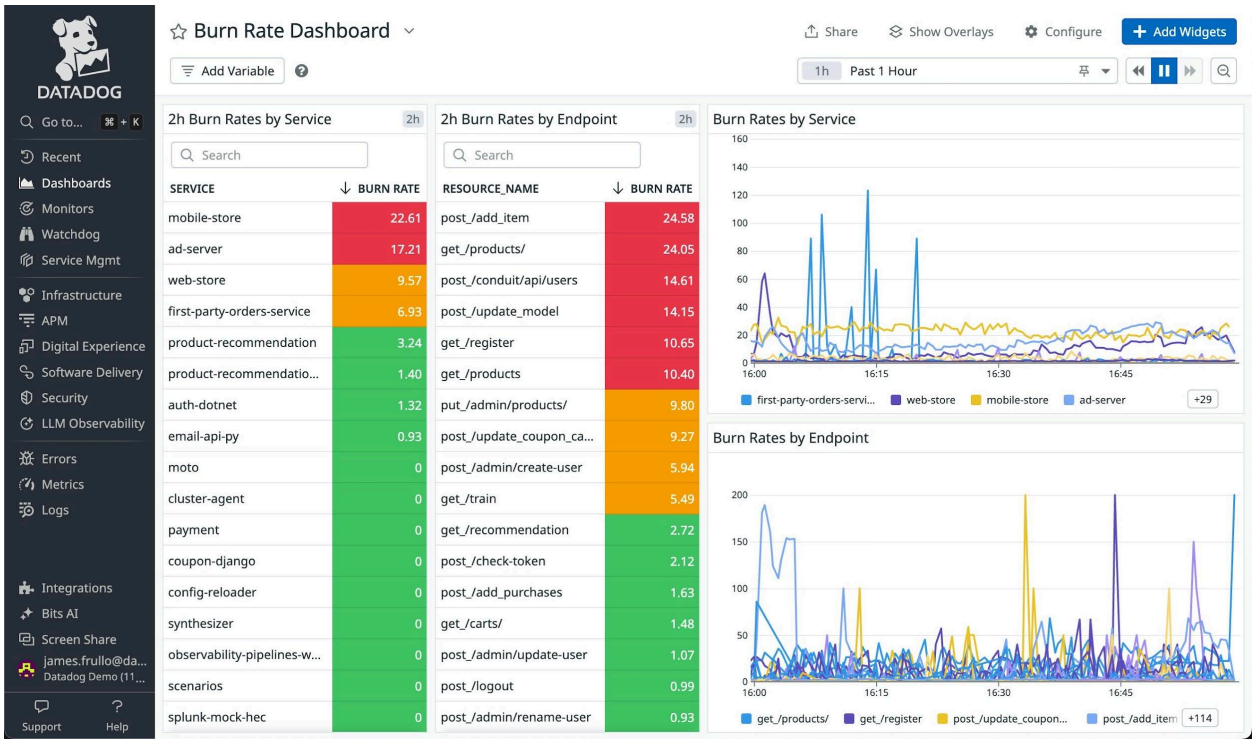
Calm. Clear. Trusted.

🏁 WHAT YOU NOW HAVE (TOP 1%)

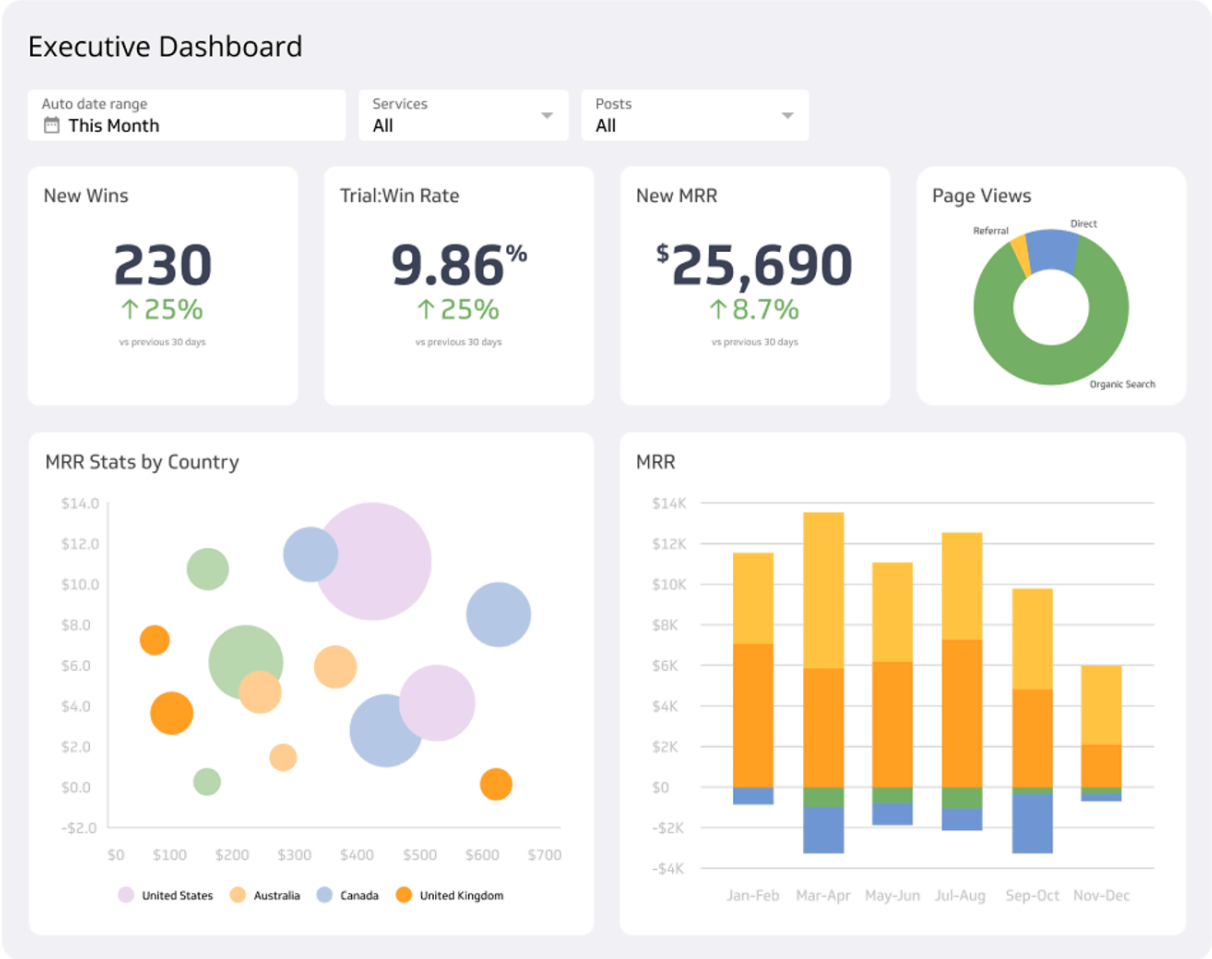
You now own:

✅ True SLO-based alerting
✅ Error-budget-driven decisions
✅ Low-noise, high-signal alerts
✅ Protected on-call engineers
✅ Executive-aligned reliability ops

This is elite SRE maturity for data engineering.


DATADOG
Go to... ⌘ + K
Recent
Dashboards
Monitors
Watchdog
Service Mgmt
Infrastructure
APM
Digital Experience
Software Delivery
Security
LLM Observability
Errors
Metrics
Logs
Integrations
Bits AI
Screen Share
james.frullo@da...
Datadog Demo (11...
Support
Help
Burn Rate Dashboard
Share
Show Overlays
Configure
Add Widgets
Add Variable
1h
Past 1 Hour
2h Burn Rates by Service
2h
Search
SERVICE
BURN RATE
mobile-store 22.61
ad-server 17.21
web-store 9.57
first-party-orders-service 6.93
product-recommendation 3.24
product-recommendatio... 1.40
auth-dotnet 1.32
email-api-py 0.93
moto 0
cluster-agent 0
payment 0
coupon-django 0
config-reloader 0
synthesizer 0
observability-pipelines-w... 0
scenarios 0
splunk-mock-hec 0
2h Burn Rates by Endpoint
2h
Search
RESOURCE_NAME
BURN RATE
post_/add_item 24.58
get_/products/ 24.05
post_/conduit/api/users 14.61
post_/update_model 14.15
get_/register 10.65
get_/products 10.40
put_/admin/products/ 9.80
post_/update_coupon_ca... 9.27
post_/admin/create-user 5.94
get_/train 5.49
get_/recommendation 2.72
post_/check-token 2.12
post_/add_purchases 1.63
get_/carts/ 1.48
post_/admin/update-user 1.07
post_/logout 0.99
post_/admin/rename-user 0.93
Burn Rates by Service
160
140
120
100
80
60
40
20
0
16:00
16:15
16:30
16:45
first-party-orders-servi...
web-store
mobile-store
ad-server
+29
Burn Rates by Endpoint
200
150
100
50
0
16:00
16:15
16:30
16:45
get_/products/
get_/register
post_/update_coupon...
post_/add_item
+114


BURN RATE CHART



5

🔥 ERROR-BUDGET BURN

SQL + DASHBOARDS (SRE-GRADE, FINAL)

Executive truth:
“Can we ship?” is a math question — not a meeting.

🧱 FINAL DATA MODEL (AUTHORITATIVE)

All error-budget decisions come from ONE table:

sre_metrics.error_budget_daily

This table is the single source of truth.

📁 DIRECTORY (FINAL)

sre/error-budget/

```
│
├── README.md
├── error_budget_daily.sql
├── burn_rate_rolling.sql
└── dashboards.md
```

📘1️⃣FINAL ERROR-BUDGET SQL (AUTHORITATIVE)

error_budget_daily.sql

```
CREATE OR REPLACE TABLE sre_metrics.error_budget_daily AS
WITH freshness AS (
  SELECT
    DATE(measured_at) AS date,
    'freshness' AS slo_type,
    SAFE_DIVIDE(
      COUNTIF(delay_minutes > 30),
      COUNT(*)
    ) / 0.01 AS burn_rate
  FROM sre_metrics.data_freshness
  GROUP BY date
```

```
),

pipeline AS (
  SELECT
    run_date AS date,
    'pipeline' AS slo_type,
    SAFE_DIVIDE(
      1 - success_rate,
      0.005
    ) AS burn_rate
  FROM sre_metrics.pipeline_success
),

quality AS (
  SELECT
    date,
    'quality' AS slo_type,
    SAFE_DIVIDE(
      null_rate,
      0.001
    ) AS burn_rate
  FROM sre_metrics.data_quality
),

cost AS (
  SELECT
    date,
    'cost' AS slo_type,
    SAFE_DIVIDE(
```

```
    deviation,
    0.10
  ) AS burn_rate
 FROM sre_metrics.cost_burn
)

SELECT * FROM freshness
UNION ALL SELECT * FROM pipeline
UNION ALL SELECT * FROM quality
UNION ALL SELECT * FROM cost;
```

🔑 Meaning

- burn_rate = 1.0 → Error budget exhausted
- burn_rate > 1.0 → SLO breach
- burn_rate < 1.0 → Budget remaining

📘2️⃣ ROLLING BURN RATE (TREND AWARENESS)

burn_rate_rolling.sql

```
CREATE OR REPLACE TABLE sre_metrics.error_budget_rolling AS
SELECT
 slo_type,
 date,
 AVG(burn_rate) OVER (
  PARTITION BY slo_type
  ORDER BY date
  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
 ) AS rolling_7d_burn
FROM sre_metrics.error_budget_daily;
```

Why This Matters

- Prevents overreacting to spikes
- Shows trend-based risk
- Used for release decisions

📊 DASHBOARD 1 – ENGINEERING VIEW

(Operational Control)

Charts

1. Daily Burn Rate (Line)
    - X: Date
    - Y: Burn Rate
    - Thresholds:
        - 0.5 (warning)
        - 1.0 (breach)
2. Rolling 7-Day Burn (Line)
    - Early warning system
3. SLO Type Breakdown (Bar)
    - Freshness
    - Pipeline
    - Quality
    - Cost

Questions Answered

- Which SLO is failing?
- Is this noise or trend?
- Do we pause releases?

📊 DASHBOARD 2 – EXECUTIVE VIEW

(Trust & Risk)

Executives don't want graphs — they want status.

Tiles

Metric	Status
Data Freshness	🟢 / 🟡 / 🔴
Pipeline Reliability	🟢 / 🟡 / 🔴
Data Quality	🟢 / 🟡 / 🔴
Cost Stability	🟢 / 🟡 / 🔴

Overall Reliability

GREEN → Normal shipping

YELLOW → Caution

RED → Release freeze

🧠 RELEASE DECISION LOGIC (MANDATORY)

IF any burn_rate > 1.0

→ Freeze feature releases

IF rolling burn > 0.75

→ Reliability work prioritized

IF burn_rate < 0.5

→ Normal velocity allowed

This removes politics and pressure.

🚨 ALERT INTEGRATION (FINAL)

Create alerts on:

burn_rate > 1.0 for 30 minutes

Severity:

- SEV-1
- Leadership visibility
- Automatic release freeze

📘 dashboards.md (ADD TO REPO)

Error-Budget Dashboards

These dashboards define release readiness
and reliability health.

Rules:
- Burn rate > 1 → Stop shipping
- Rolling burn > 0.75 → Slow down
- Burn rate < 0.5 → Full speed

Error budgets protect engineers and the business.

🏁 WHAT YOU NOW HAVE (TOP 0.1%)

You now own:

✅ Canonical error-budget SQL
✅ Rolling burn intelligence
✅ Engineering + executive dashboards
✅ Automated release governance
✅ True SRE decision system

This is elite operational engineering.